

光线跟踪

高林

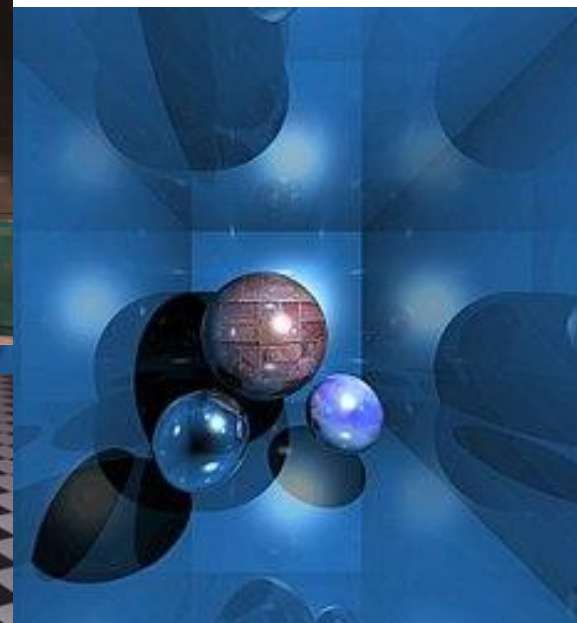
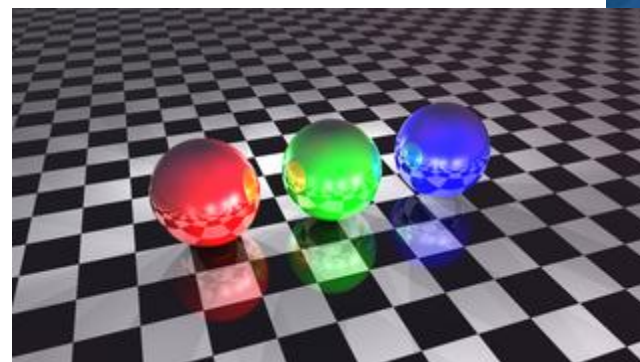
中国科学院计算技术研究所

什么是光线跟踪

- 一种用复杂的光线相互作用来绘制三维图形的渲染算法，追踪光线从来源开始照射到物体上，再由物体反射的光线“路径”

- 效果

- ◆ 模拟真实世界的光源
 - 自然光、漫反射光、折射光、...
- ◆ 模拟真实世界的物体的材质
 - 镜面、透明、半透明、...
- ◆ 模拟真实世界的阴影效果
 - 软阴影、硬阴影、...



光线跟踪算法的背景

- 使用光线追踪技术，可以合成具有极高真实感的图像

斯坦福大学的cs348b渲染竞赛



使用L系统对树木程序建模，使用元球对雪进行程序化建模，地下散射算法使雪具有逼真的外观



冰洞图像，物理模拟冰的融化和重新冻结，模拟由于融化的径流导致的冰的侵蚀

光线跟踪算法的背景

- CG与电影行业中，光线追踪被大量运用，渲染出令人震撼的画面效果
Pixar公司RenderMan的R19版本以后转向RIS，基于光线追踪技术
Disney公司动画部门的渲染器Hyperion，基于光线追踪，功能强大



光线跟踪算法的背景

- CG与电影行业中，光线追踪被大量运用，渲染出令人震撼的画面效果
使用光线追踪技术来生成或强化特效，包括逼真的反射、折射和阴影效果
使电影中的虚拟角色、火焰、烟雾和爆炸场景更加真实



光线跟踪算法的背景

- 游戏行业，光线追踪技术逐渐得到应用

Unity 2019.3正式支持实时光线追踪特性，部分3A游戏已加入实时光线追踪



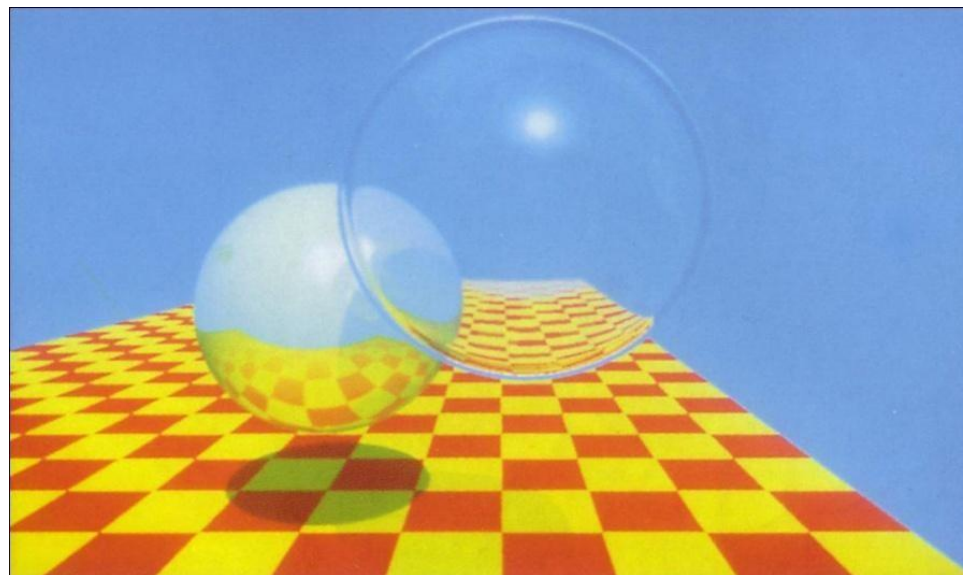
NVIDIA宣布了可加速硬件中光线追踪速度的新架构Turing



第一款搭载RTX实时混合光线追踪技术的游戏《战地5（Battlefield V）》正式面世

光线跟踪算法的历史

- Turner Whitted 于 1980 年首次提出一个包含光反射和折射效果的模型：Whitted 模型，并第一次给出 光线跟踪算法的范例，是计算机图形学历史上的里程碑。
- Turner Whitted , An improved illumination model for shaded display, Communications of the ACM, v.23 n.6, p.343-349, June 1980.



光线跟踪算法的历史

- Turner Whitted 于 1978 年在北卡罗来纳州立大学 (NCSU) 获得 Ph.D., 之后来到贝尔实验室, 提出了震惊世界的光线跟踪算法。Turner Whitted 一共只发表过19篇论文, 其中光线跟踪算法是他的第一篇论文。
- Turner Whitted 于 2003 年当选美国工程院院士。



Dr. J. Turner Whitted ([Print This](#))
Senior Researcher

Primary Work Institution: Microsoft Research
Election Year: 2003
Primary Membership Section: 05. Computer Science & Engineering

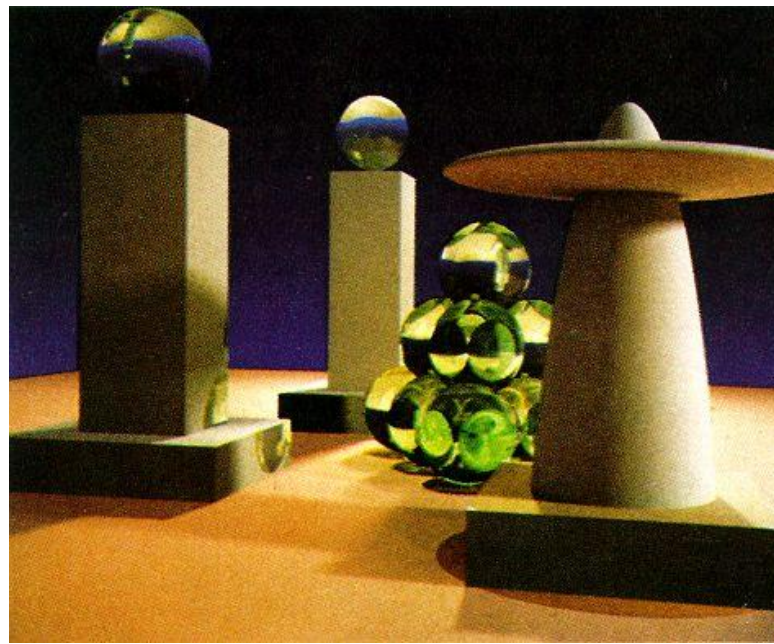
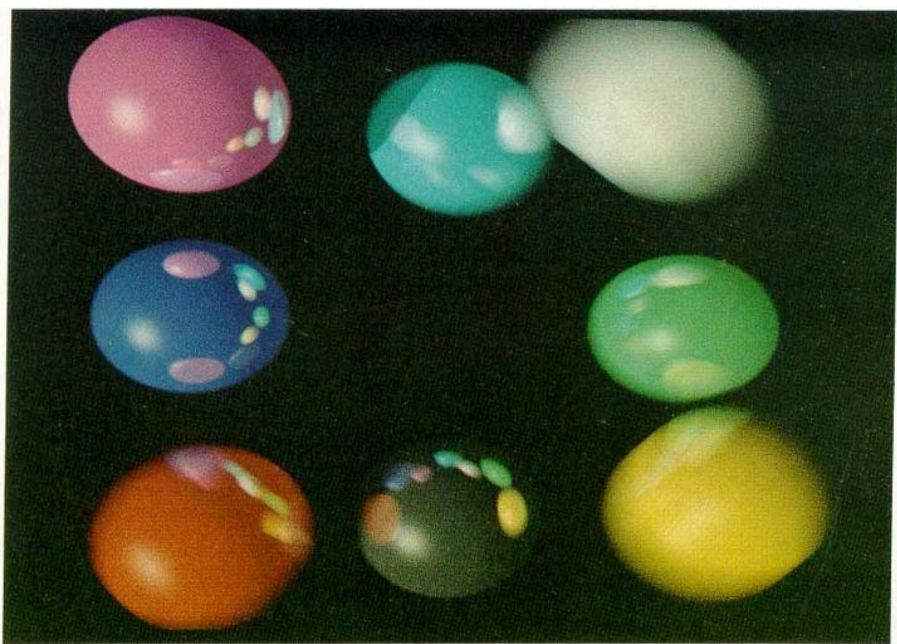
Country: United States
State: WA

Member Type: Member

Election Citation:
For contributions to computer graphics, notably recursive ray-tracing.

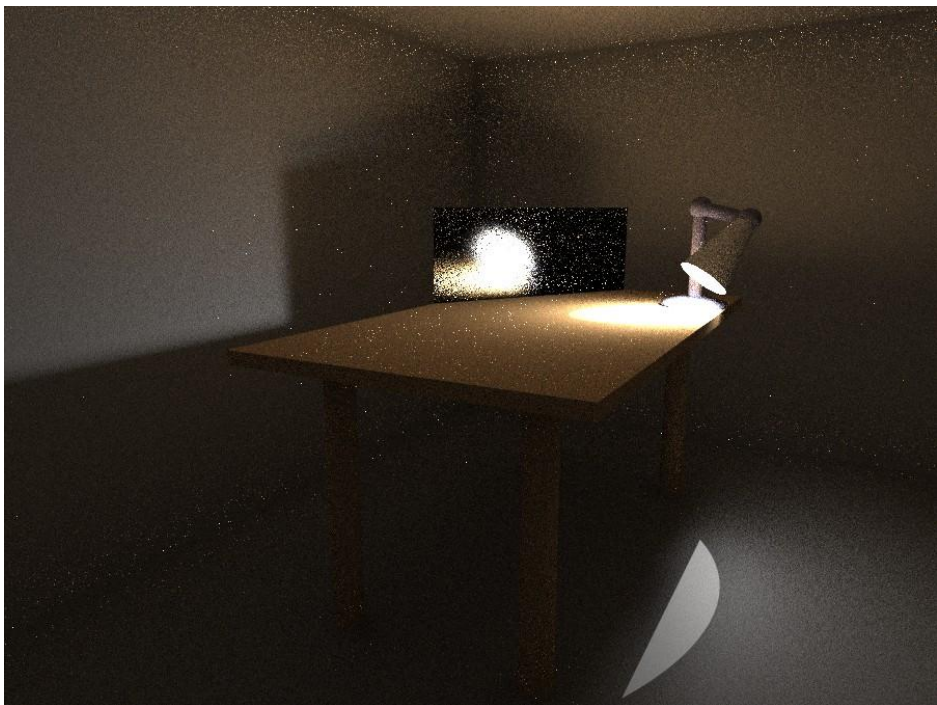
光线跟踪算法的历史

- 1984年, Cook 等人提出了分布式光线追踪 (distributed or distribution ray tracing, DRT), 使用了蒙特卡洛方法
- 1986年, Kajiya等人提出了渲染方程(rendering equation), 同时也提出了最原始路径追踪算法。



光线跟踪算法的历史

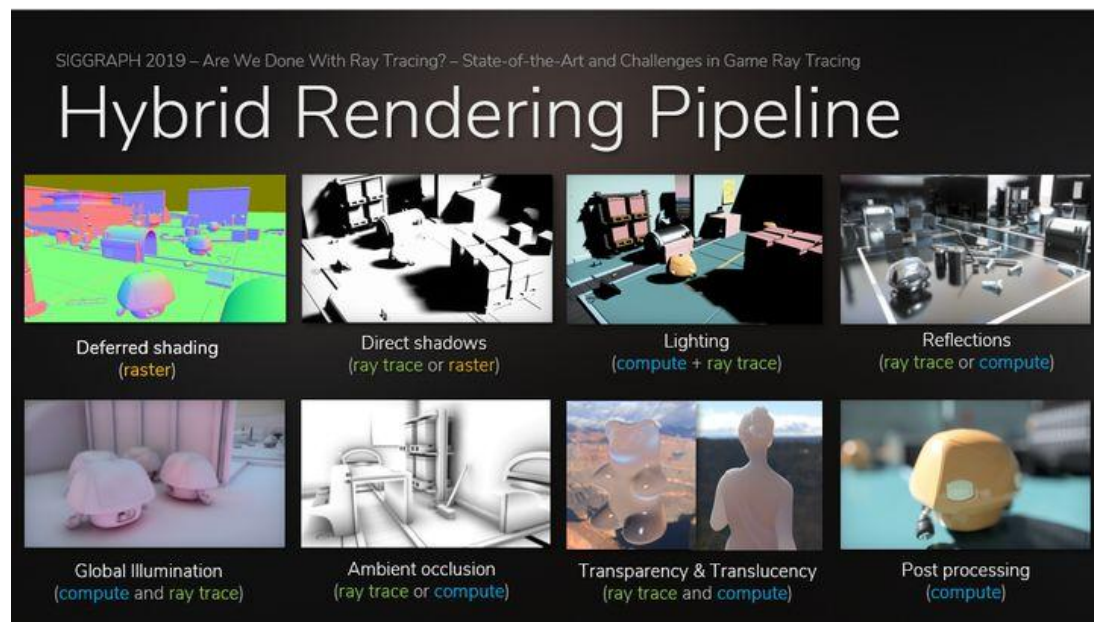
- 双向路径追踪：由Lafortune和 Willemms提出， Veach则对双向路径追踪做了详细的描述。
- 梅特波利斯光照传输：1997年 Veach首次将最早应用在计算物理领域的 Metropolis 采样方法引入图形学



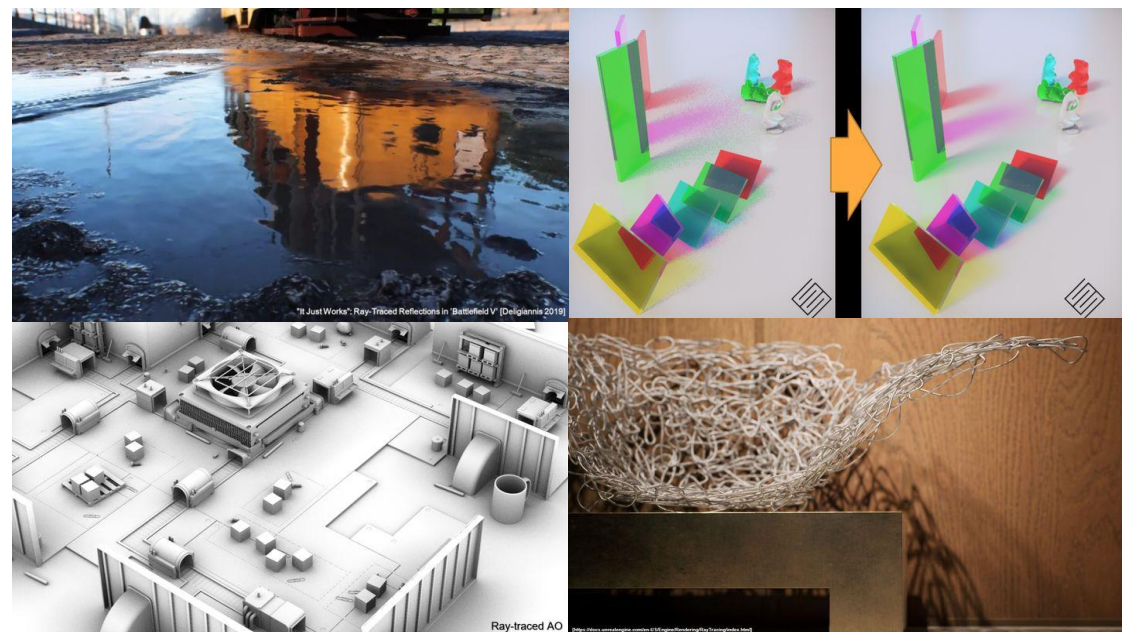
光线跟踪算法的现状

- 离线光线追踪：光线追踪和颜色计算离线进行，即使设置了很高的光线追踪采样率，很多的反射次数，是烘焙Lightmap的时间会变长，最终依旧可以渲染出很好效果。
- 实时光线追踪：就是随着摄像机视角的变动，后端需要实时发射追踪光线来重新计算光照信息，如果屏幕分辨率很高，计算量很大，对GPU的性能要求很高

光线跟踪算法的现状



混合渲染管线 (Hybrid Rendering Pipeline)
分利用光栅化, 计算着色器和光线追踪各自的优势, 对于管线的每一个渲染阶段, 在光栅化, 计算着色器和光线追踪中择优使用。



实时渲染(Real-time Rendering):
离线渲染技术应用到实时渲染
反射渲染、环境光遮蔽、阴影渲染、透明阴影渲染、多光源处理、粒子系统渲染.....

光线跟踪算法的现状

- 透明和半透明渲染实时渲染的发展近况:
 - ◆ 正确表示顺序无关的透明渲染 (Order-independent Transparency, OIT)
 - ◆ 可变的粗糙度, 折射和吸收 (Variable roughness, refractions and absorption)
 - ◆ 多IOR过渡 (Multiple index-of-refraction transitions)



光线跟踪算法的现状

■ 多光源处理的实时渲染:

- ◆ 基于加速结构的光源选择 (Acceleration structure-based selection)
- ◆ 基于重要性采样的光源选择 (Light Importance Sampling)

■ 全局光照的实时渲染:

- ◆ 基于面元 (Surfels)
- ◆ 基于网格 (Grid)
- ◆ 基于探针 (Probes)



光线投射算法

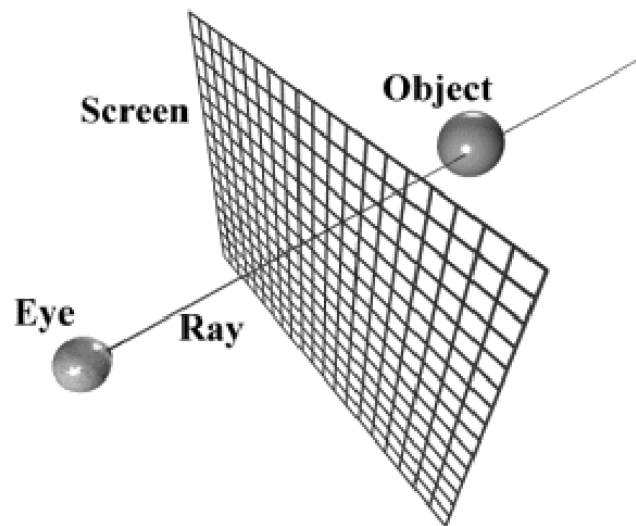
光线:

- 1.光以直线传播 (大部分)
- 2.光线交叉时不会相互干扰 (光子没有相互作用; 忽略驻波效应)
- 3.光线从光源传播到眼睛 (在路径反转作用下, 物理学性质是不变的)

光线投射算法

算法原理：

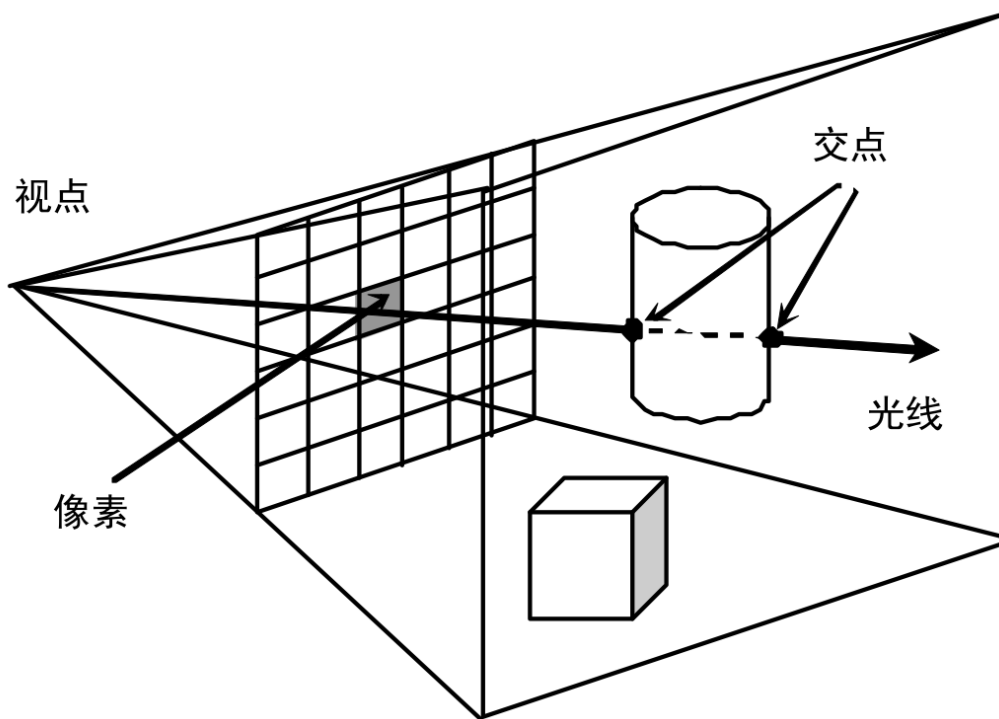
- 对屏幕上每一像素，执行下述3步操作：
 - 1: 从视点出发通过该像素中心向场景发出一条光线，并求出该条光线与场景中物体的全部交点
 - 2、将各交点沿光线方向排序，获得离视点最近交点
 - 3、依据局部光照明模型计算该交点处的光亮度，并将所得光亮度值赋给该像素
- 当所有屏幕像素都处理完毕时，即得到一幅真实感图形



光线投射

■ 步骤1:

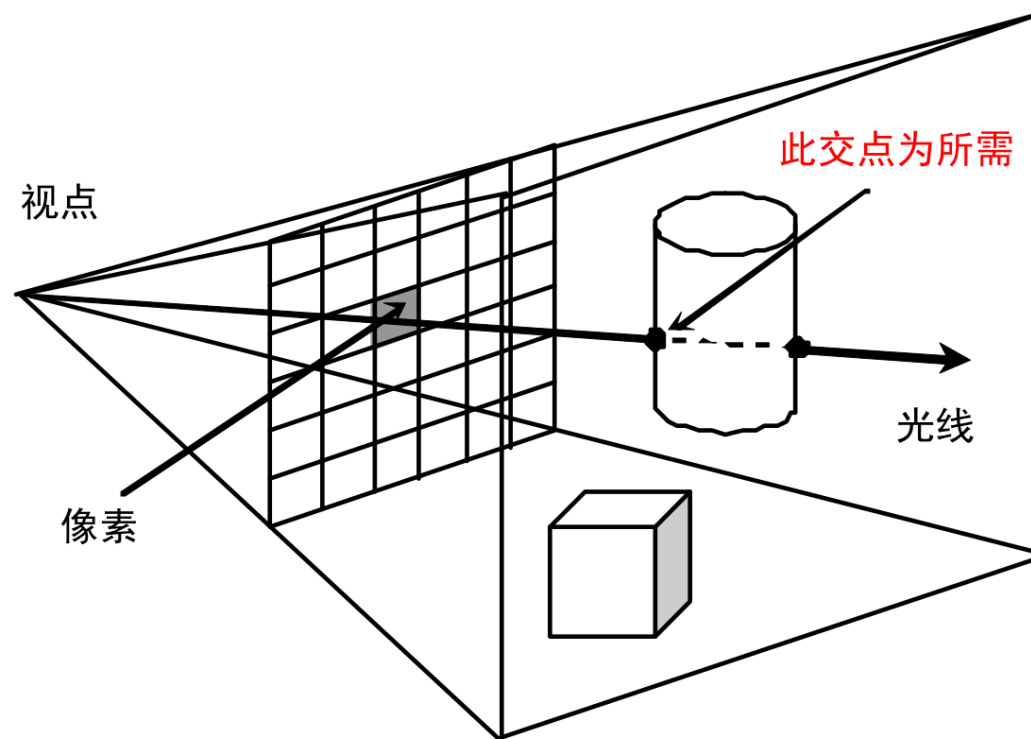
从视点出发通过屏幕上一像素中心向场景发出一条光线，并求出该条光线与场景中物体的全部交点



光线投射

■ 步骤2:

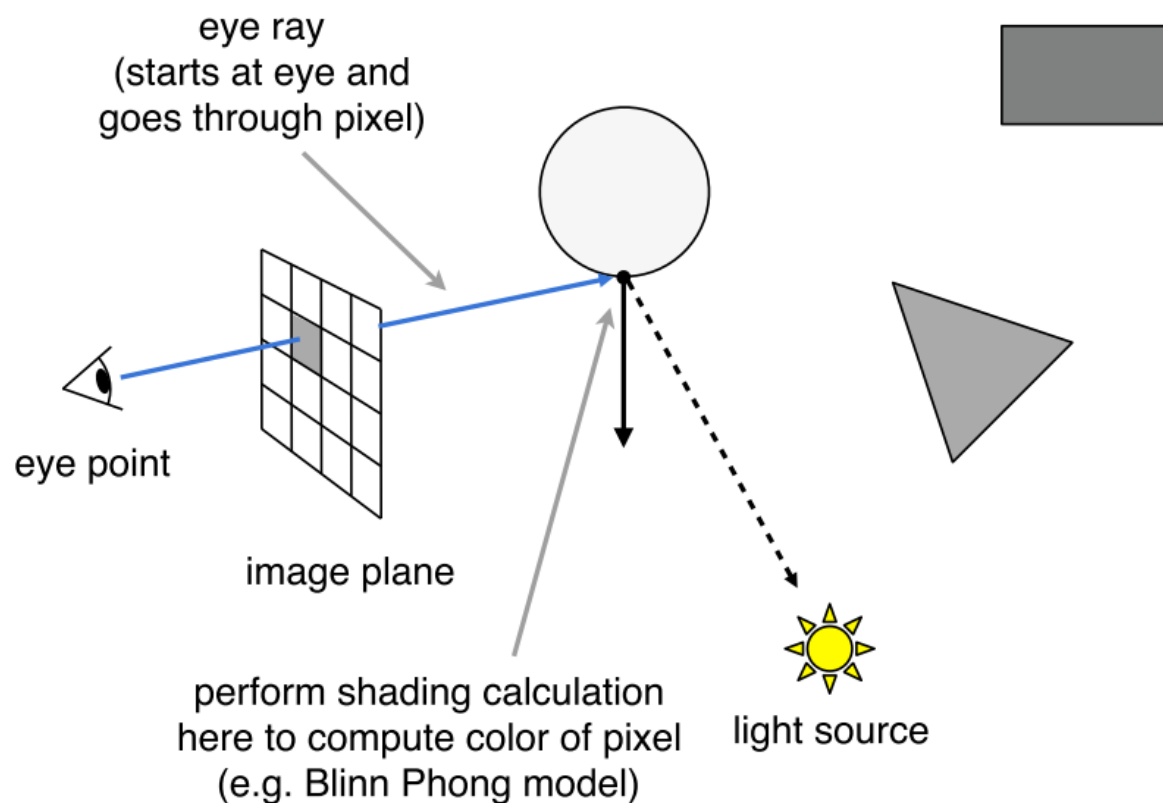
将各交点沿光线方向排序，获得离视点最近交点



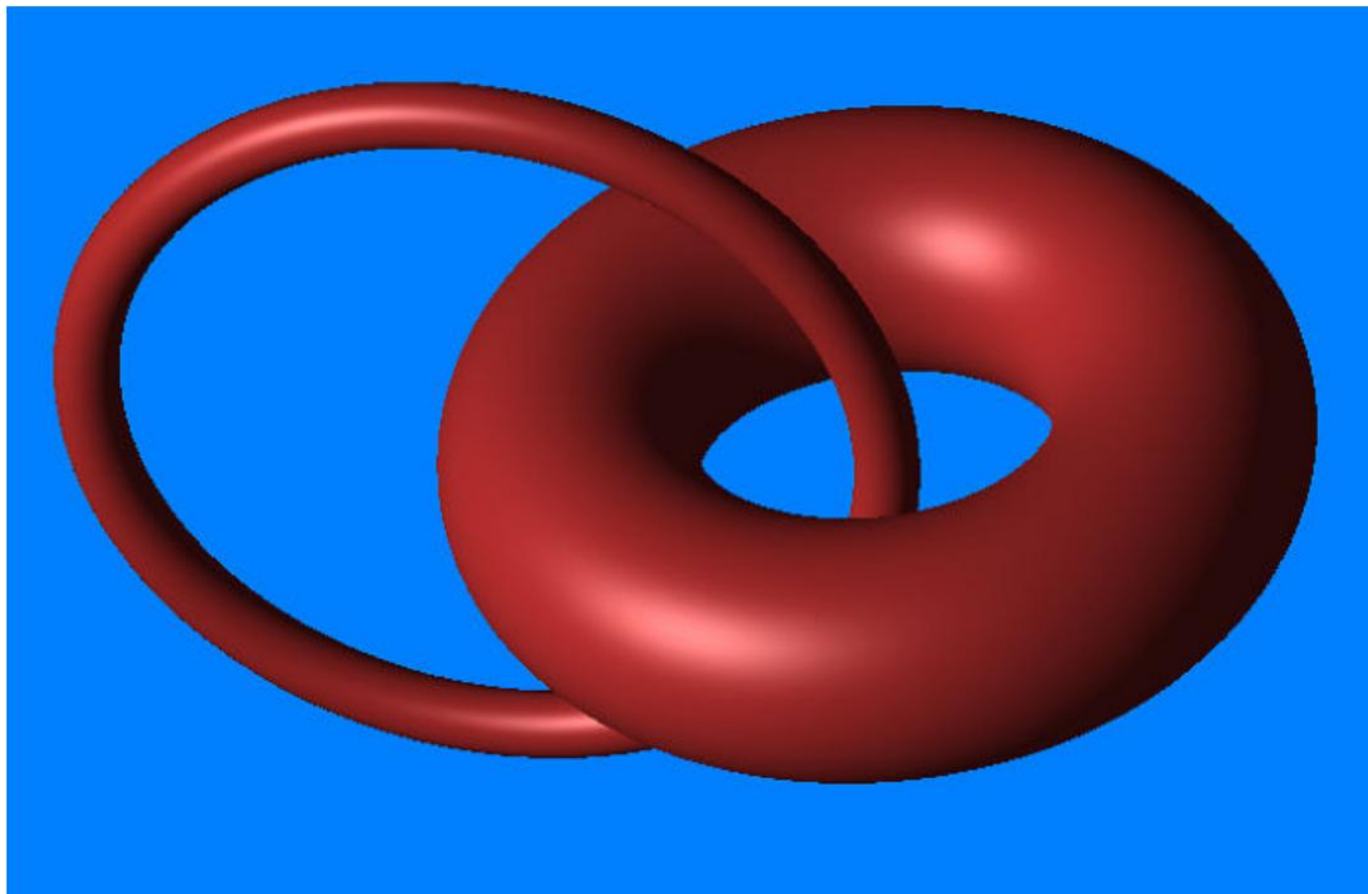
光线投射

■ 步骤3:

依据局部光照明模型计算该交点处的光亮度，并将所得光亮度值赋给该像素



光线投射

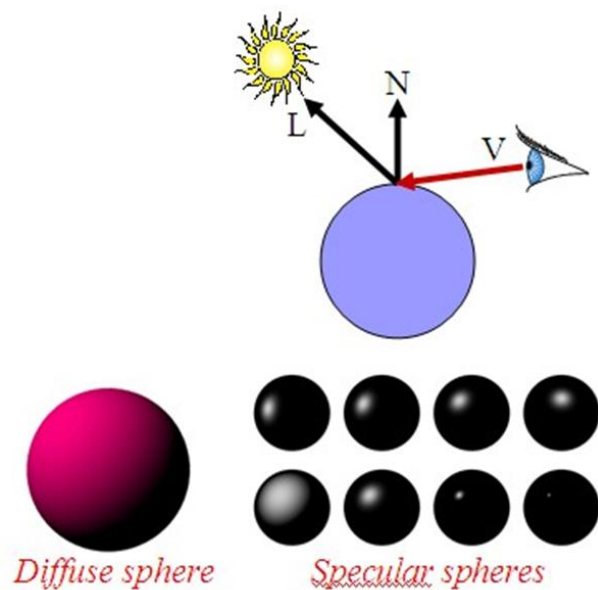


光线投射结果示例

光线投射算法

光线投射算法：

- 明暗效果仅仅由第一次相交的物体表面法向方向、材质、视点和光照方向、以及光照强度等因素共同决定。
- 光线投射并不考虑第二层以及更深层次的光线，因此不具有阴影、反射、折射等效果。



光线投射算法

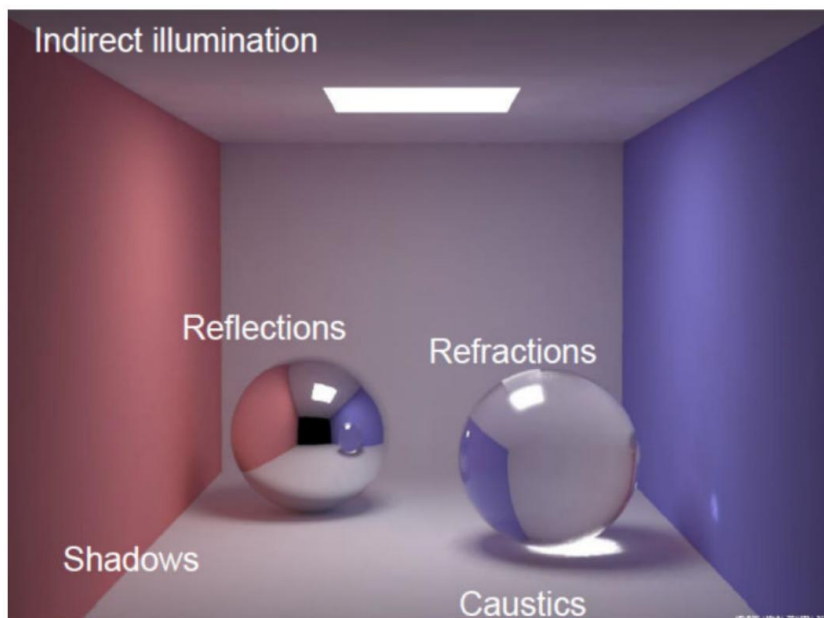
伪代码:

FOR each pixel

- Cast a ray and find the intersection point
- IF there is an intersection
 - color = ambient
 - FOR each light
 - color += shading from this light (depending on lightproperty
and material property)
 - return color
- ELSE
 - return background color

光线跟踪算法概述

- 光线求交 (Ray Intersection)、阴影 (Shadows)、透明和镜面反射 (Transparency and Specular Reflection)、纹理 (textures)
- 迄今为止最为成功的生成真实感图形算法之一：算法简单、生成的图形真实感强、计算量大
- 其前身是光线投射 (Ray Casting) 算法



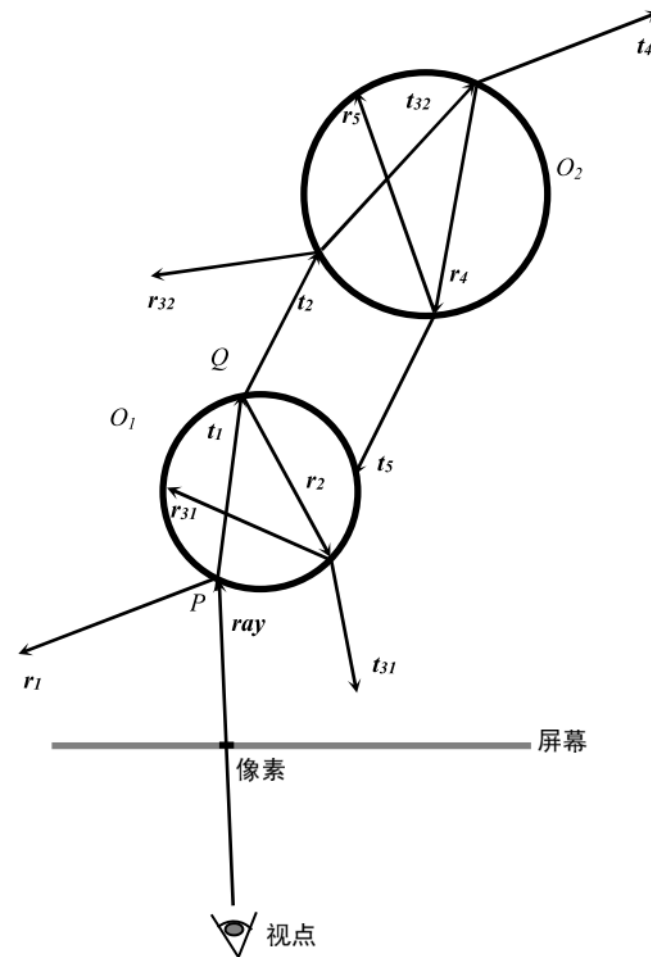
光线跟踪算法概述

■ 为什么我们能看见物体？

光 (Light) 可以理解为一组由光源发出、经物体反复弹射而最终进入视点的光线 (Rays)。最终将光线 (Rays) 射入人们的眼中，使我们看到物体。

■ 光线追踪：

逆向跟踪从光源发出的光经由物体之间的多次反射和折射后投射到物体表面，最终进入人眼的过程：



添加反射和折射效果

光线跟踪具有模拟物体表面镜面反射效果以及折射效果的能力：

- 首先，计算光线与场景中物体的最近的交点。
- 然后，计算光线在交点处被物体反射和折射所产生的新的光线的方向，新的方向由入射光方向、物体表面法向、及介质共同决定。
- 对新产生的光线 (反射光线和折射光线) 分别继续进行跟踪。

光线的反射

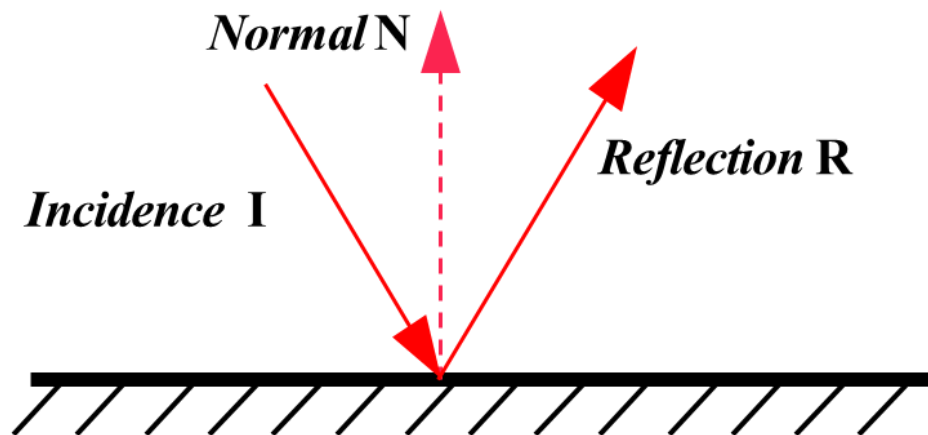
反射定律:

- 入射角 = 反射角
- 入射光线、反射光线、以及物体表面法向量位于同一个平面内。

反射光线的方向向量 R 可以按如下公式计算:

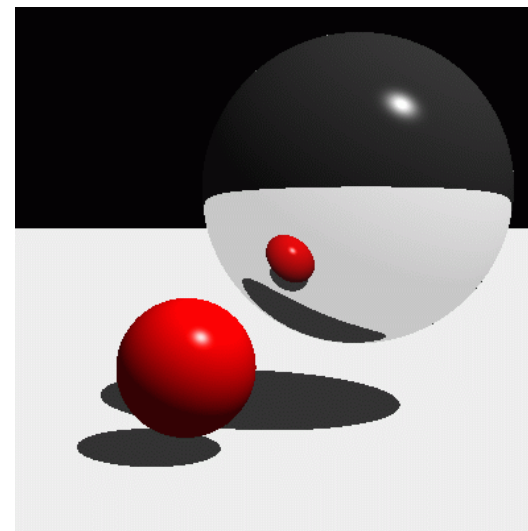
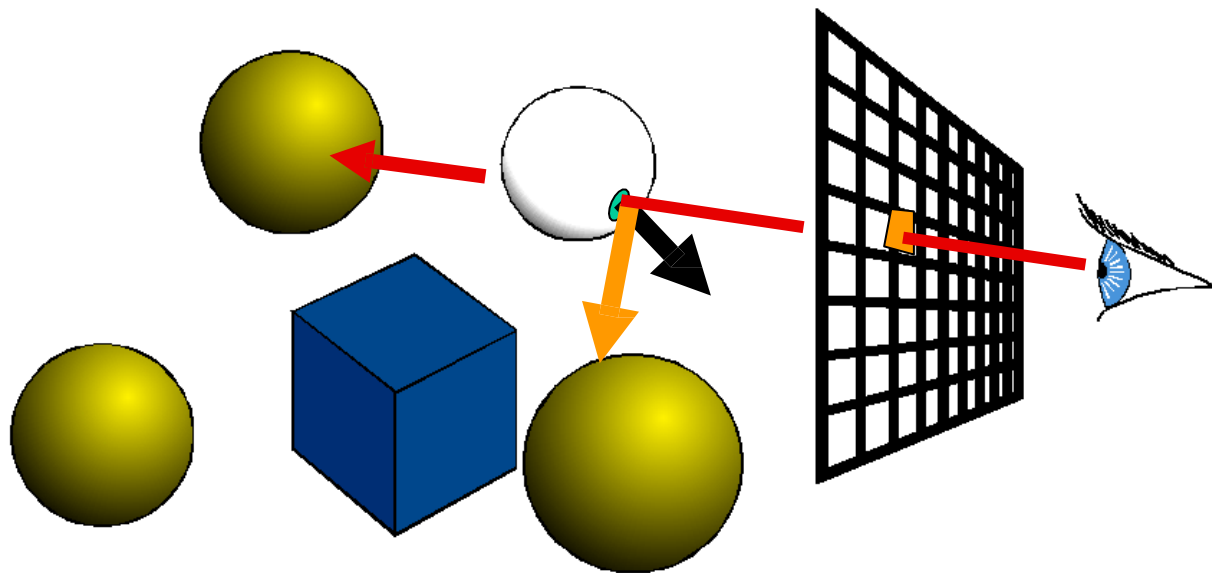
$$R = I - 2(I \cdot N)N$$

其中 I, N, R 均为单位向量



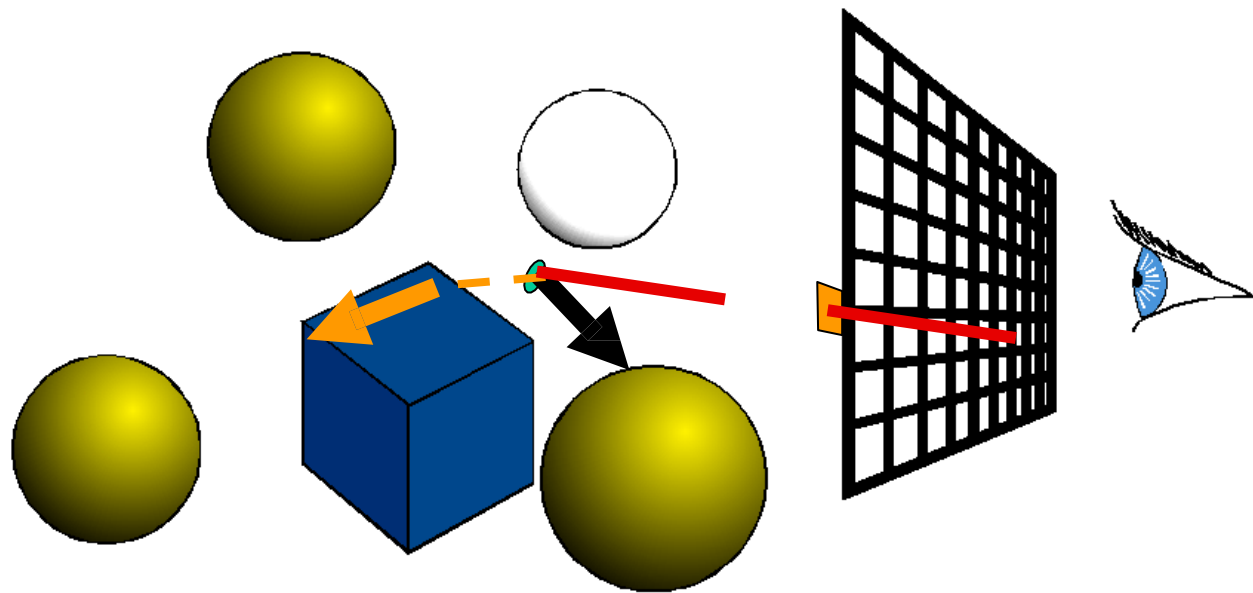
光线的反射

如图所示，反射光线的方向和视点方向，相对于法向方向呈对称关系：



光线的折射

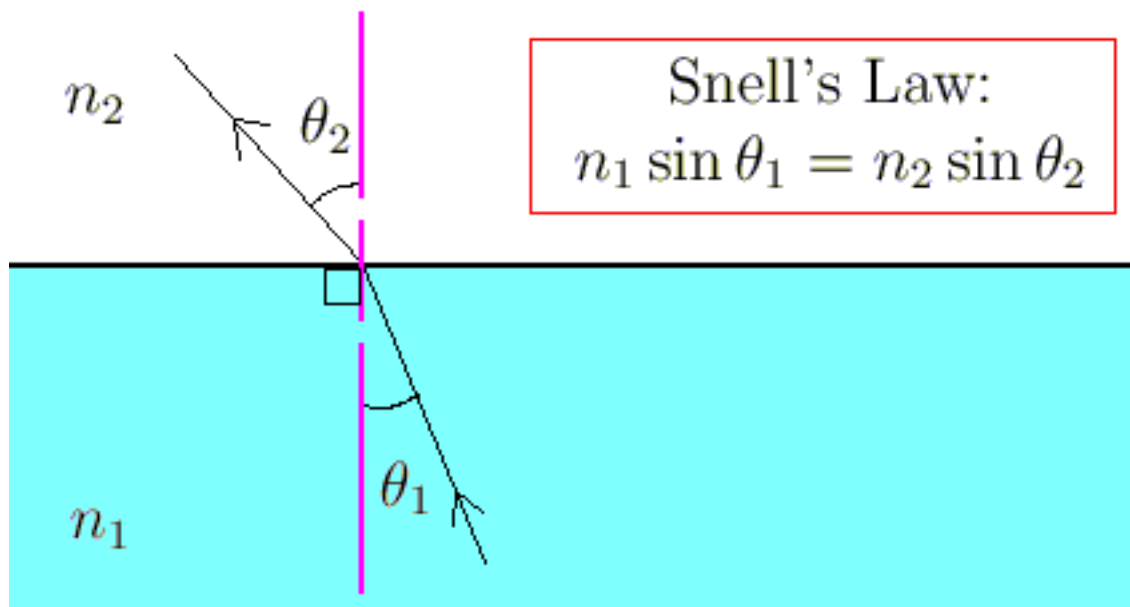
当光线由一种透明或半透明的介质进入另一种时，光的方向会由于介质密度的相对不同而发生偏折，也就是光的折射现象。



光线的折射

折射定律 (也叫 Snell 定律):

入射角和折射角的正弦值之比是一个取决于介质的常数, 这个常数被称为相对折射率。



光线的折射

由 Snell 定律:

$$\eta_i \sin \theta_i = \eta_T \sin \theta_T$$

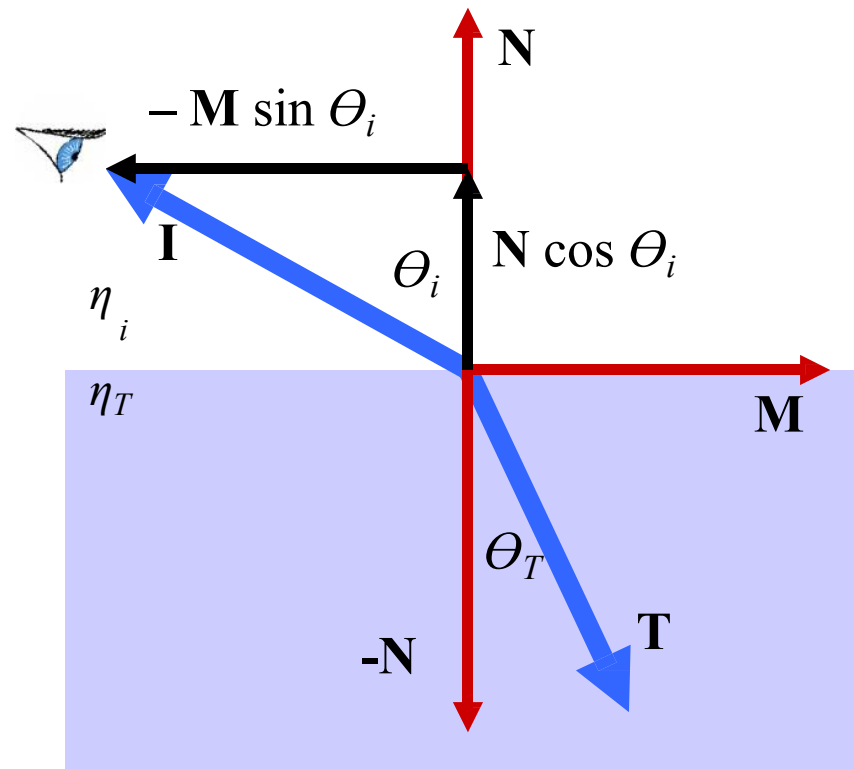
$$\eta_i^2 \sin^2 \theta_i = \eta_T^2 \sin^2 \theta_T$$

$$\eta_i^2 (1 - \cos^2 \theta_i) = \eta_T^2 (1 - \cos^2 \theta_T)$$

$$\cos \theta_T = \sqrt{1 - \frac{\eta_i^2 (1 - \cos^2 \theta_i)}{\eta_T^2}}$$

利用 $\cos \theta_T$, 可以将折射光的方向写成:

$$T = -\frac{\eta_i}{\eta_T} * I + \left(\frac{\eta_i}{\eta_T} (I \cdot N) - \cos \theta_T \right) * N$$

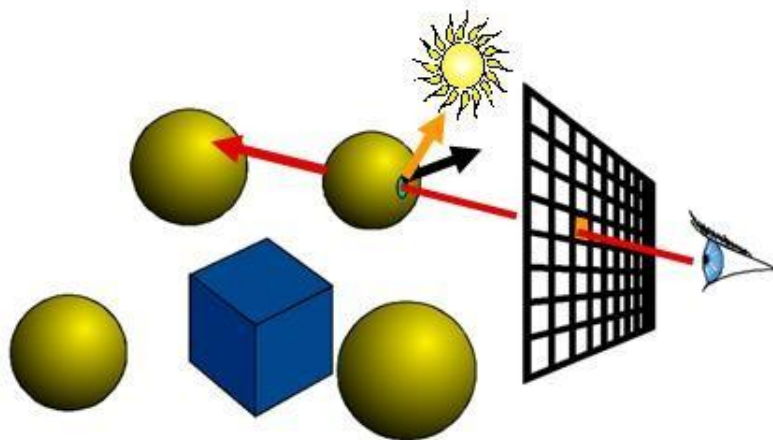


光线跟踪算法流程

将显示缓存区看成是由空间中的像素组成的矩形阵列，人眼透过这些像素看到场景中的物体

对于每个像素 P 计算其色彩值：

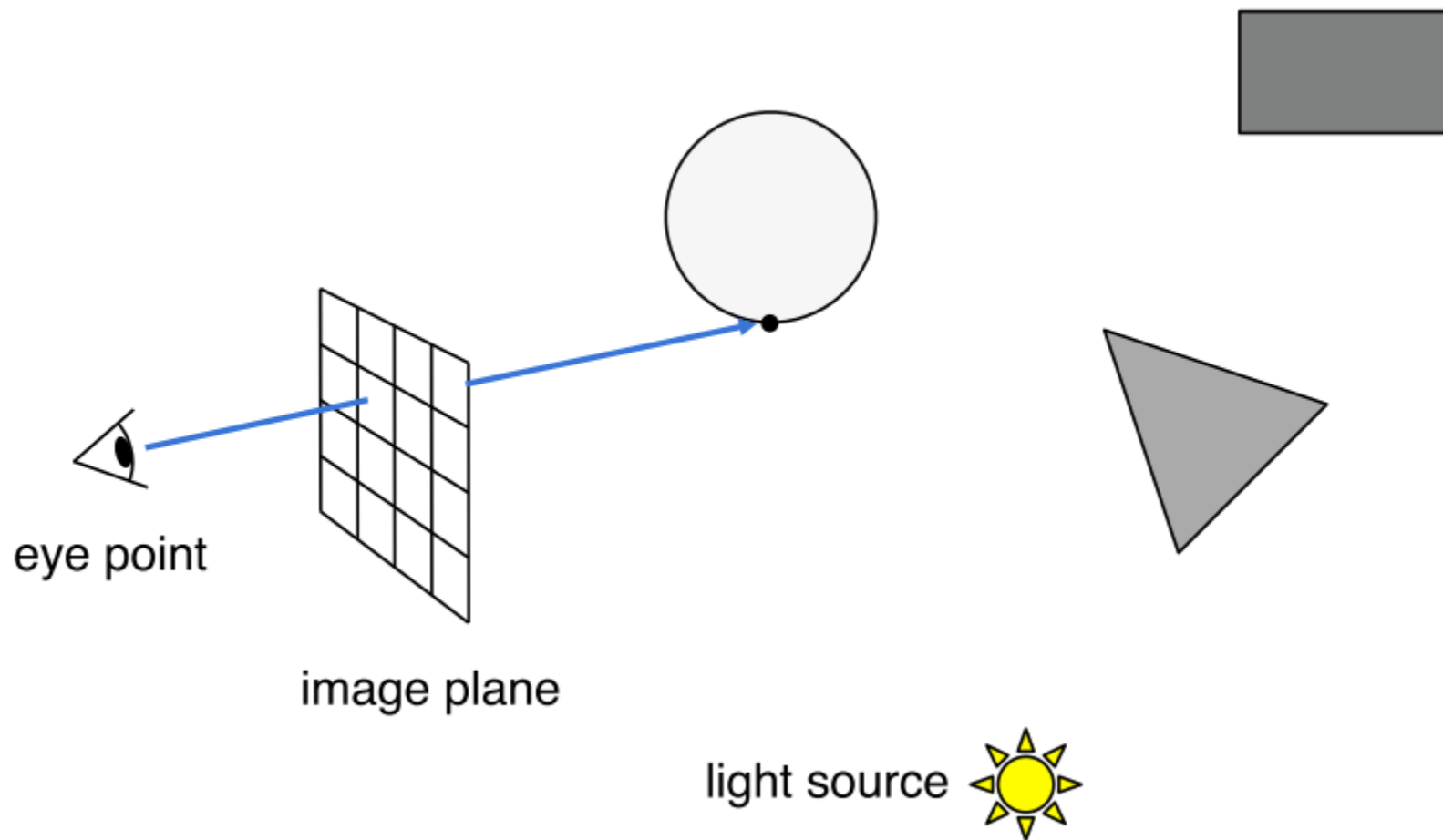
1. 计算由视点连接像素 P 中心的光线 (Ray) 延长后所碰到的第一个物体的交点。
2. 使用局部光照模型 (如 Phong模型) 计算交点处的颜色值。
3. 沿交点处的反射和折射方向对光线进行跟踪。



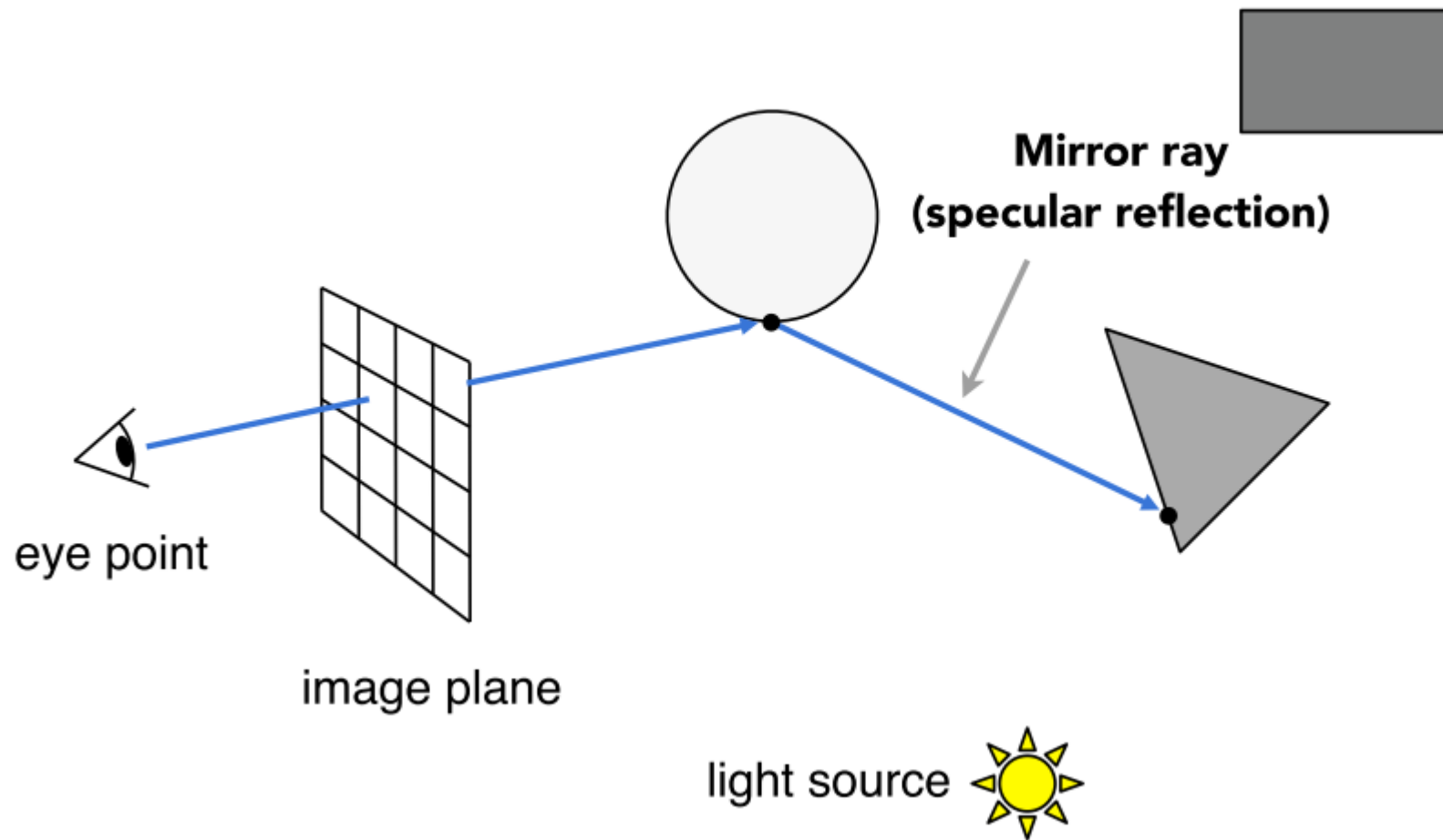
光线跟踪原理

- 跟踪从照相机中发出的黄线的话，每条黄线都可以产生出一系列secondary rays：一条反射光线，一条折射光线，并为每个光源产生一条阴影线。
- 这些光线产生后（除了阴影线外）都可以被视为普通的光线。
- 这意味着一条反射光线可以再被反射和折射，这种方法叫做“递归光线跟踪”。每条新产生的光线都增加了它先前光线聚集的地方的颜色，最终每条光线都对最开始由primary ray穿过的像素点的颜色做出了自己的贡献。

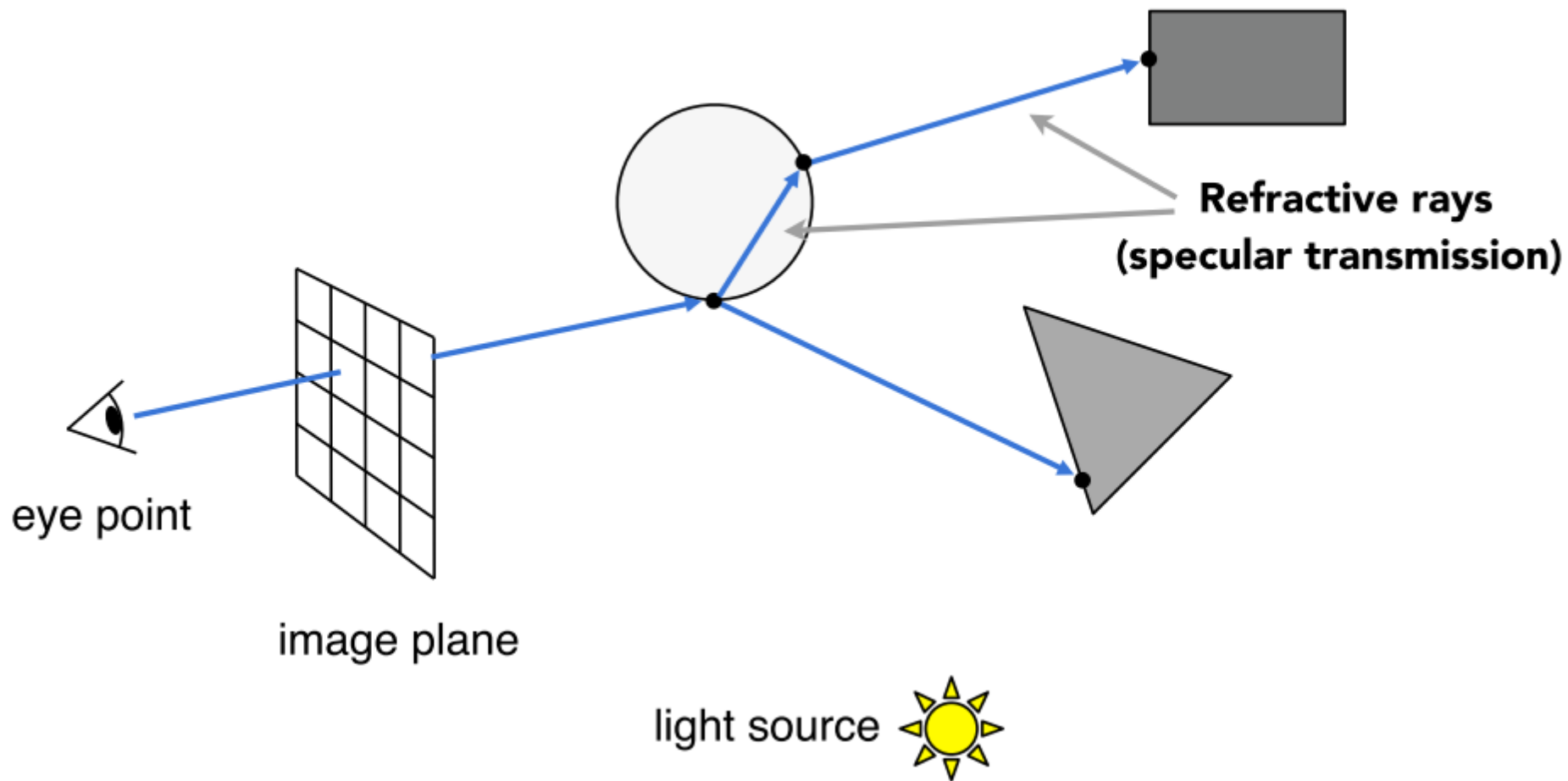
光线跟踪原理



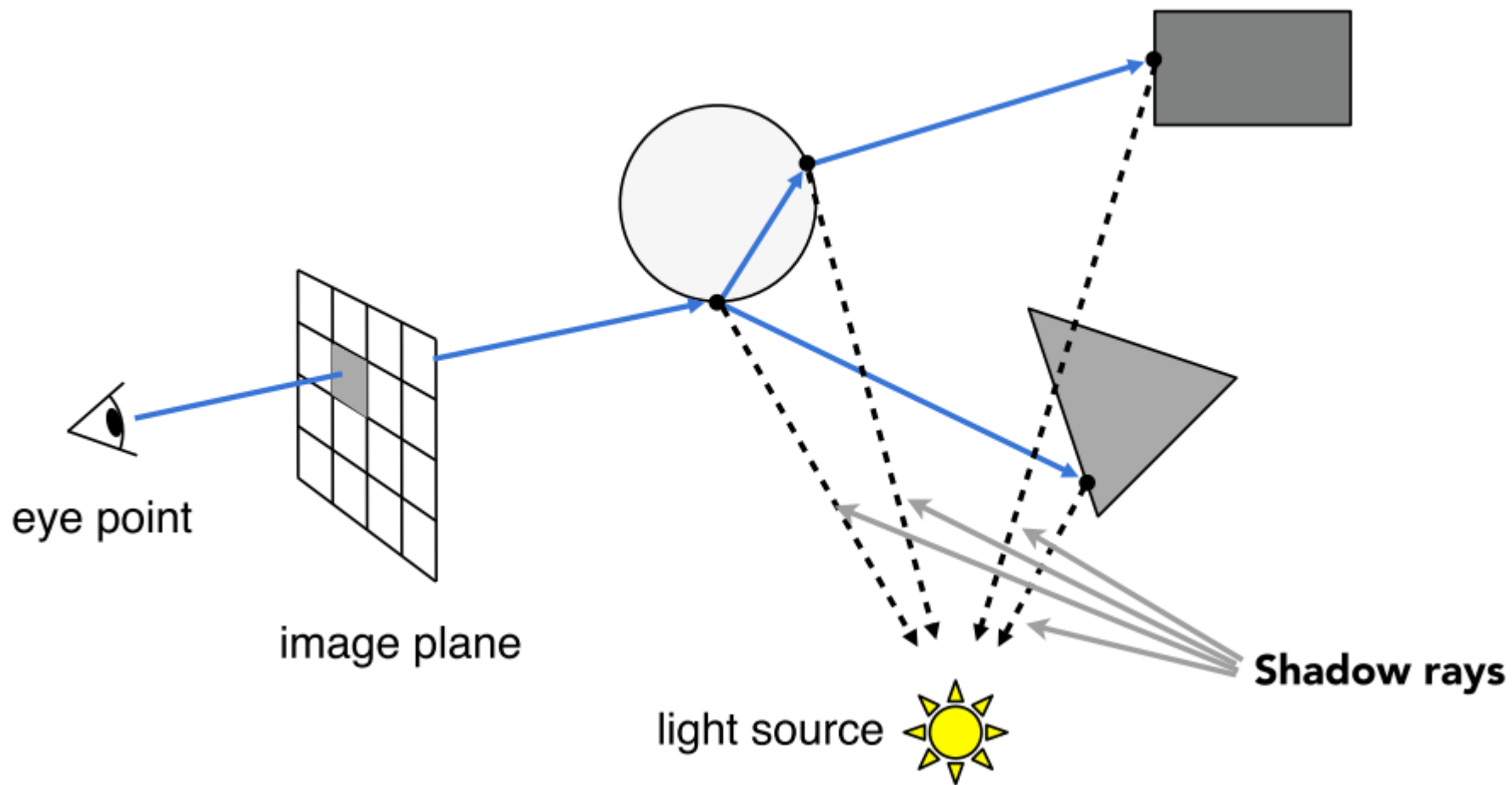
光线跟踪原理



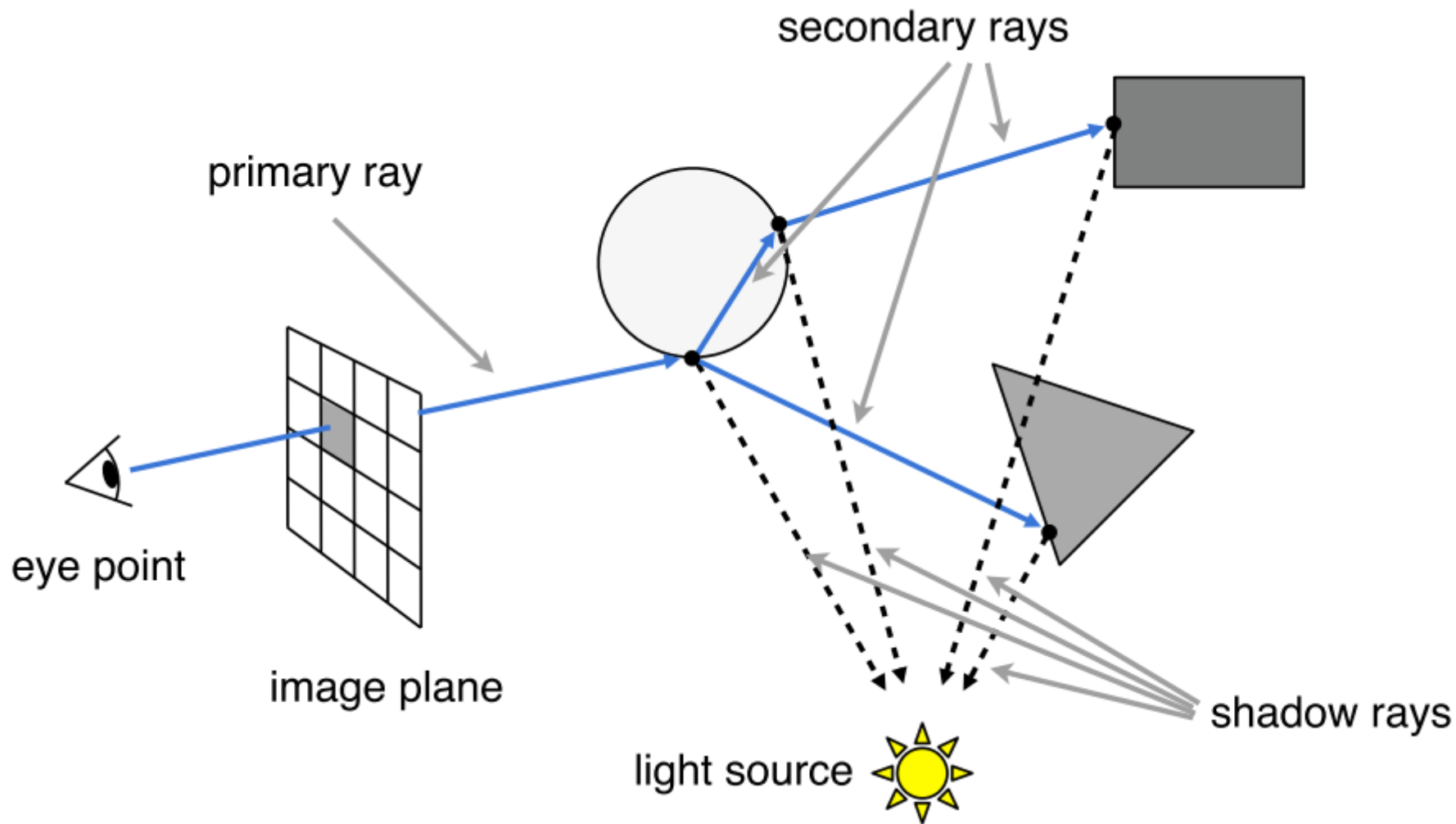
光线跟踪原理



光线跟踪原理

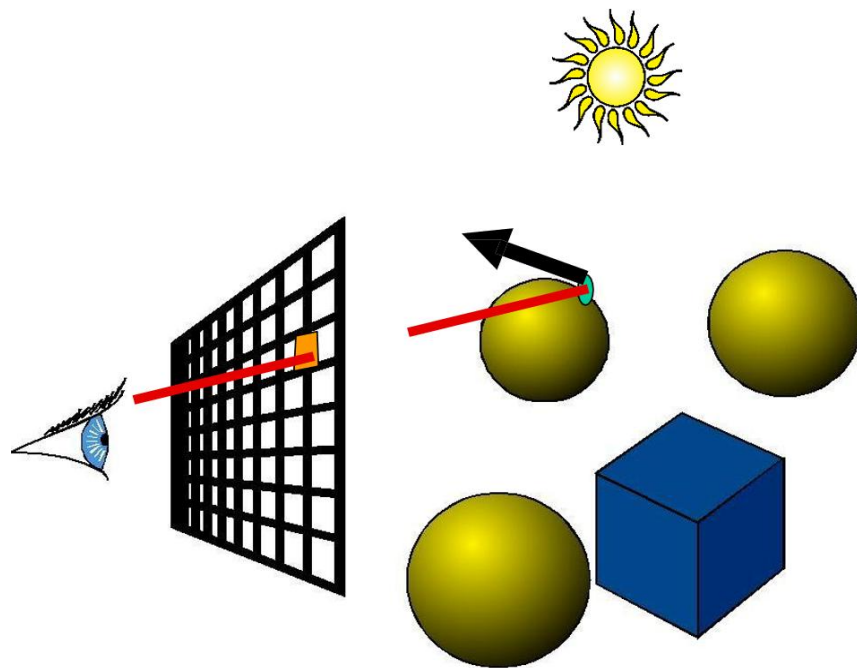


光线跟踪原理



光线跟踪算法流程

```
对图像中的每一个像素{  
    创建从视点通过该像素的光线  
    初始化 最近T 为无穷大, 最近物体为空  
    对场景中的每一个物体{  
        如果光线与物体相交{  
            如果交点处的 t 比 最近T 小{  
                设置 最近T 为交点的 t值  
                设置 最近物体 为该物体  
            }  
        }  
    }  
    //使用T, t来储存物体交点和视点的距离  
}  
//以上为对最近物体交点的计算
```



光线跟踪算法流程

如果 最近物体为空值{

用背景色填充该像素

}

否则{

对每个光源射出一条光线来检测是否处在阴影中

如果物体表面是反射面，生成反射光；递归

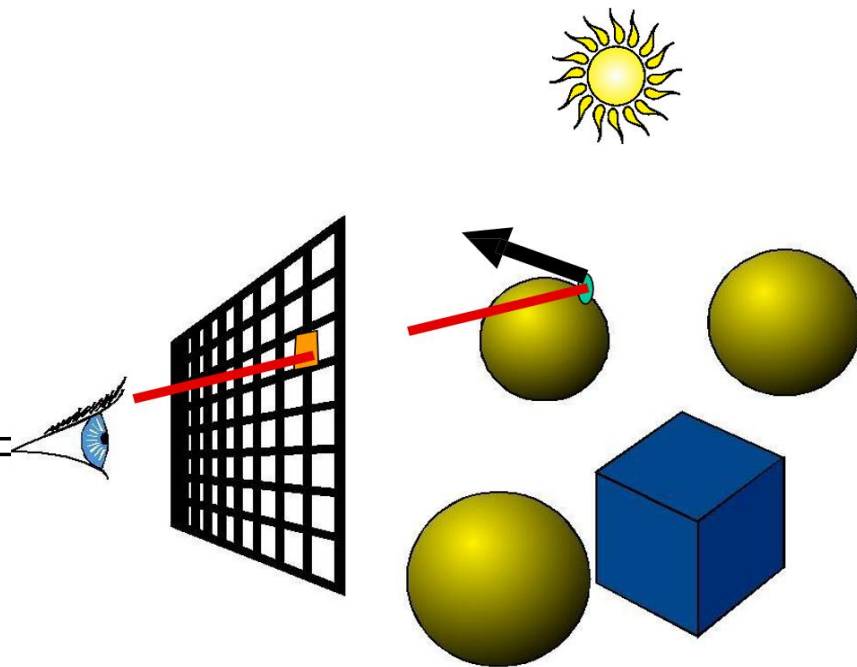
如果物体透明，生成折射光；递归

使用 最近物体 和 最近T 来计算着色函数

以着色函数的结果填充该像素

}

}



经典的光线跟踪算法

```
IntersectColor( vBeginPoint, vDirection) //输入参数, 射线的起点, 方向
{
    Determine IntersectPoint;
    Color = ambient color; //赋初值, 环境光
    for each light
        Color += local shading term;
    if( surface is reflective ) //如果物体可反射
        Color += reflect Coefficient * IntersectColor( IntersecPoint, Reflect Ray);
    if( surface is refractive ) // 如果物体可折射射
        Color += refract Coefficient * IntersectColor( IntersecPoint, Reflect Ray);
    return Color;
}
```

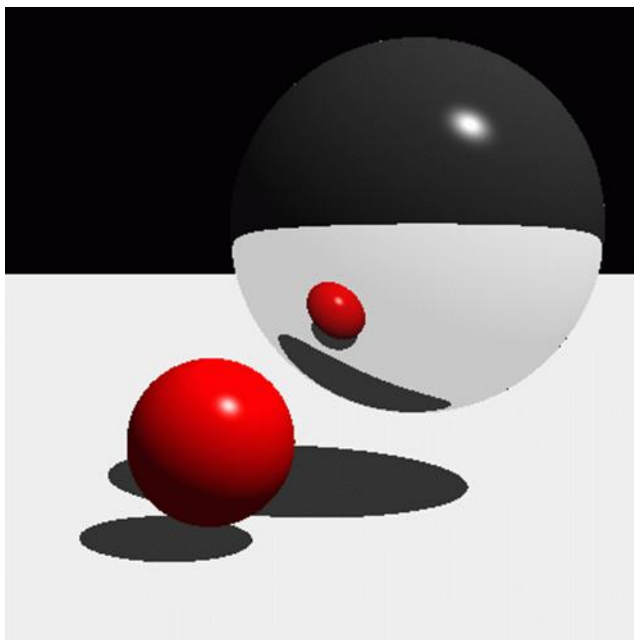
光线跟踪

■ 光线跟踪递归过程中止条件

1. 光线与环境中任何物体均不相交，或交于纯漫射面
2. 被跟踪光线返回的光亮度值对像素颜色的贡献很小
3. 已递归到给定深度

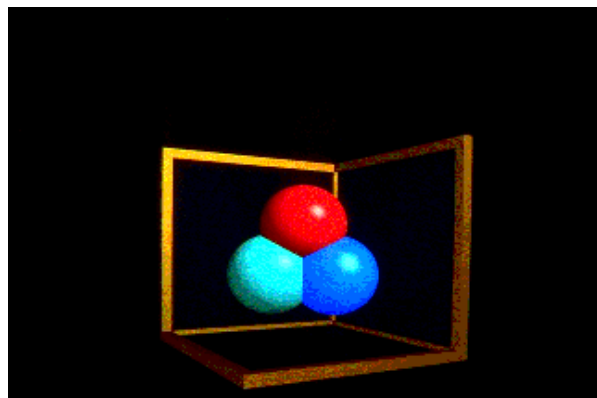
光线跟踪特征

- 通过光线跟踪，可以很容易地表现出例如阴影、反射、折射等引人入胜的视觉效果。
- 除了基本的几何形体，（例如球体、椎体、立方体等），光线跟踪容易适用于更复杂的物体表示方法，（例如多边形网格表示或者复合形体等）。

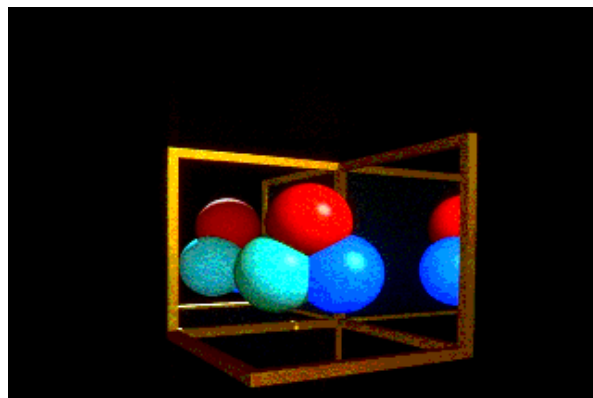


光线跟踪特征

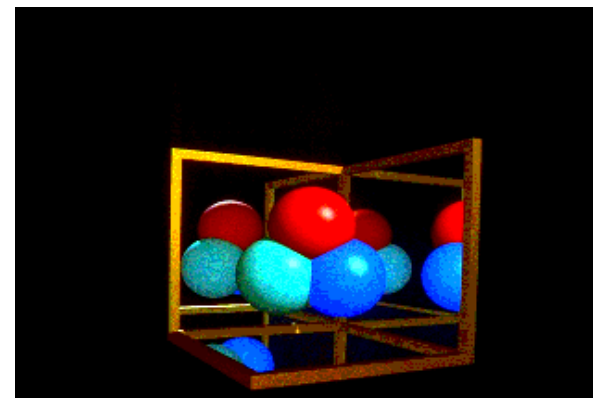
■ 光线跟踪效果示例：



0 层递归



1 层递归



2 层递归

光线求交

- 光线的表示
- 光线与平面 (Plane) 求交
- 光线与三角形 (Triangle) 求交
- 光线与球面 (Sphere) 求交
- 光线与多边形 (Polygon) 求交

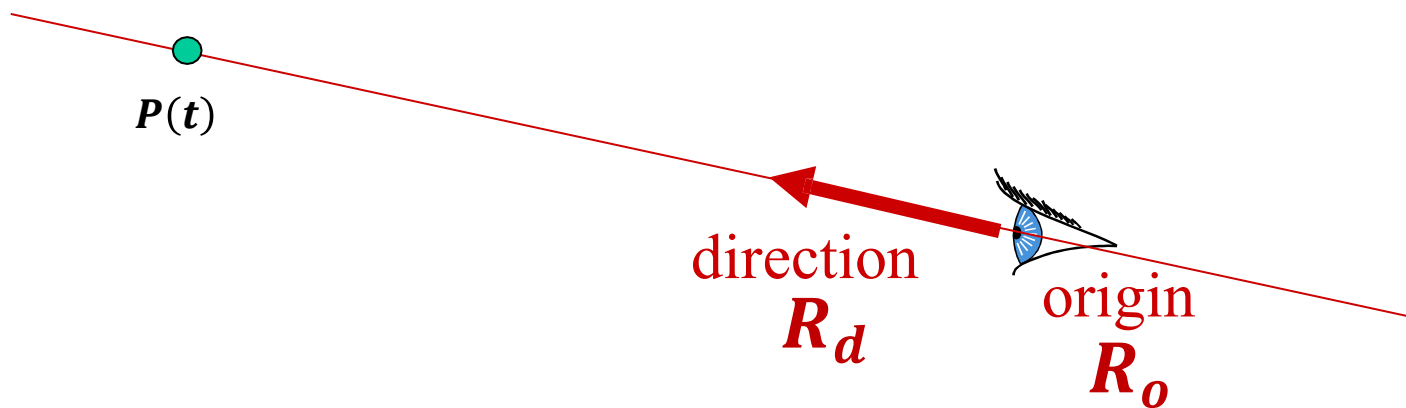
光线

光线 (射线) 的参数表示

$$P(t) = R_o + t * R_d$$

$R_o = (x_o, y_o, z_o)$ 是光线的源点; 向量 $R_d = (x_d, y_d, z_d)$ 代表光线的朝向, 通常来说 R_d 是单位向量

—参数 t 表示光线到达的位置: 在光线的正方向上, 参数 t 都是正数, $t > 0$



光线与平面求交

平面的表示:

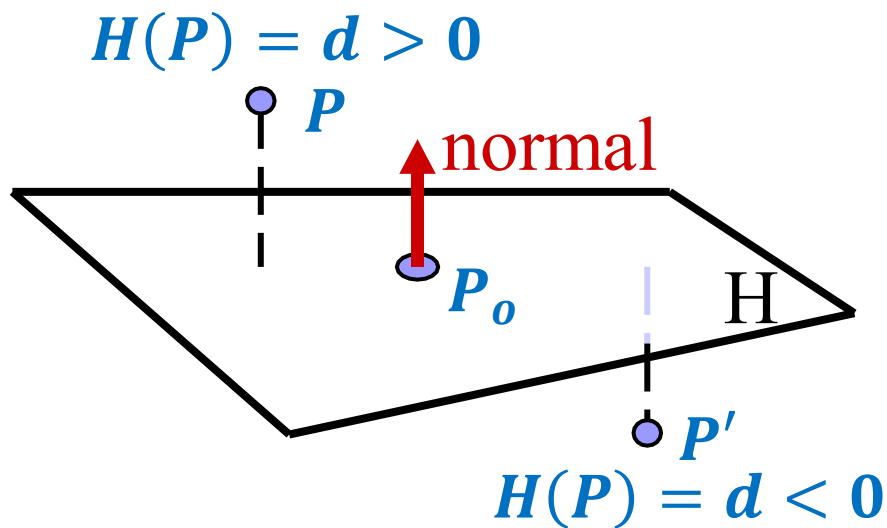
显示表示: $P_o = (x_o, y_o, z_o)$, $n = (A, B, C)$

隐式表示: $H(P) = Ax + By + Cz + D = 0$

$$H(P) = n \cdot P + D = 0$$

点到平面的距离:

当 n 是单位法向量时, P 到平面 H 的距离就是 $H(P)$ 。



光线与平面求交

- 给定满足如下的平面方程：

$$n \cdot P + D = 0$$

如何计算一条光线与该平面的交点？

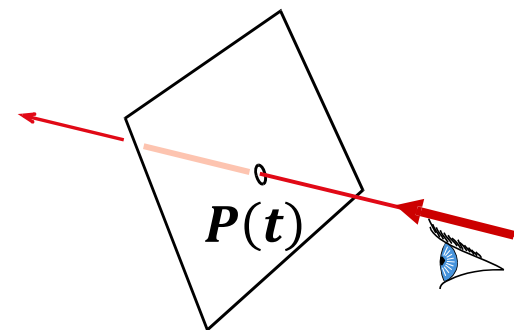
- 连列方程如下：

$$P(t) = R_o + t * R_d$$

$$n \cdot P(t) + D = 0$$

解得： $t = -(D + n \cdot R_o) / (n \cdot R_d)$

最后验算 $t > 0$



光线与平面求交

平面的表示:

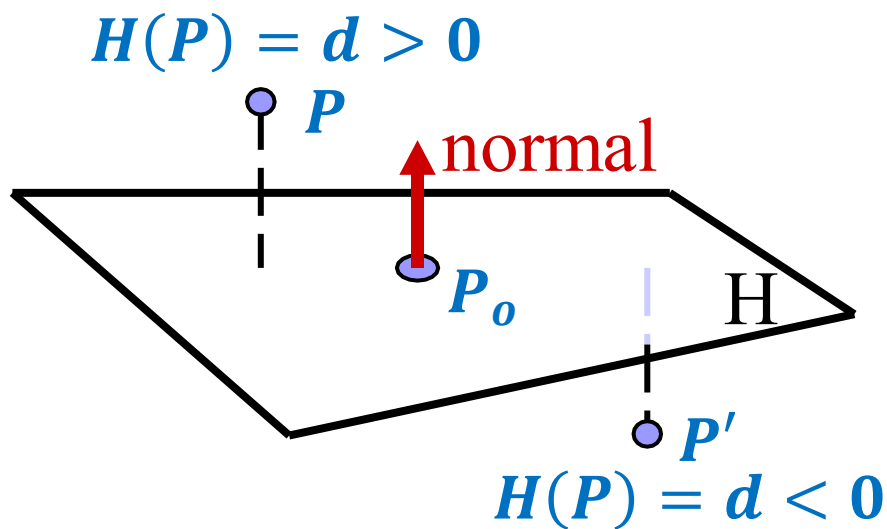
显示表示: $P_o = (x_o, y_o, z_o)$, $n = (A, B, C)$

隐式表示: $H(P) = Ax + By + Cz + D = 0$

$$H(P) = n \cdot P + D = 0$$

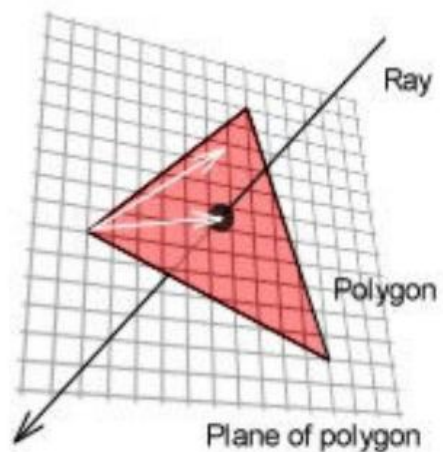
点到平面的距离:

当 n 是单位法向量时, P 到平面 H 的距离就是 $H(P)$ 。



光线与三角形求交

- 在实时的图形学中，基于三角形的几何表示 (三角面片) 十分常见：
每个三角形可以使用三个顶点的坐标来表示。
- 常用步骤：
 1. 判断光线是否与三角形所在平面求交
 2. 若交点存在，判别交点是否位于三角形内部



光线与三角形求交

■ 重心坐标:

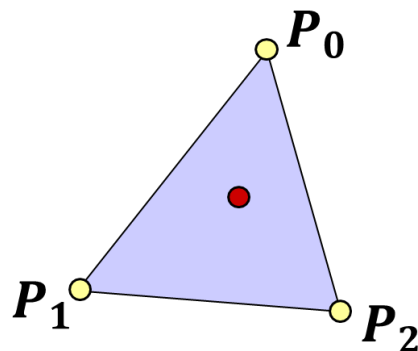
三角形 $P_0P_1P_2$ 内的一点 P 可以表示成:

$$p = \alpha p_0 + \beta p_1 + \gamma p_2$$

其中 (α, β, γ) 被称为重心坐标, 它们满足:

$$0 \leq \alpha, \beta, \gamma \leq 1, \alpha + \beta + \gamma = 1$$

重心坐标有许多其他应用, 例如纹理映射、法向插值、颜色插值, 等等。



光线与三角形求交

我们将 $\alpha + \beta + \gamma = 1$ 写成 $\alpha = 1 - \beta - \gamma$, 有:

$$p = (1 - \beta - \gamma)p_0 + \beta p_1 + \gamma p_2$$

可以将光线与三角形求交描述成如下方程:

$$R_0 + tR_d = (1 - \beta - \gamma)p_0 + \beta p_1 + \gamma p_2$$

即

$$(R_d \quad p_0 - p_1 \quad p_0 - p_2) \begin{pmatrix} t \\ \beta \\ \gamma \end{pmatrix} = p_0 - R_0$$

这说明交点的重心坐标以及其在直线上的 t 参数可以通过解一个线性方程组而得到。

光线与三角形求交

令: $E_1 = P_0 - P_1, E_2 = P_0 - P_2, S = P_0 - R_o$

根据 Cramer 法则, 我们可以写出:

$$\begin{bmatrix} t \\ \beta \\ \gamma \end{bmatrix} = \frac{1}{\det(R_d, E_1, E_2)} \begin{bmatrix} \det(S, E_1, E_2) \\ \det(R_d, S, E_2) \\ \det(R_d, E_1, S) \end{bmatrix}$$

最后需要检查 $t > 0$ 且 $0 \leq \beta, \gamma \leq 1, \beta + \gamma \leq 1$ 以保证交点位于三角形的内部。

光线与球面求交

数学求解:

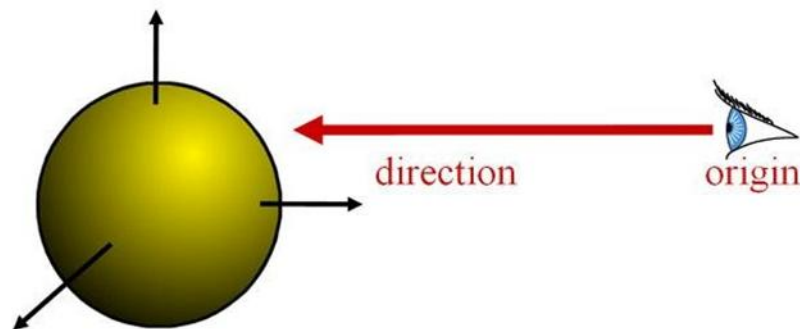
首先给出球面的数学描述:

给定中心点 (球心) p_c , 半径 r , 可以写出球面的隐式方程

$$f(P) = \|P - P_c\| - r = 0$$

求光线与球面的交点, 相当于要求解如下的方程:

$$\|P - P_c\| - r = 0$$



光线与球面求交

将之前的方程进行化简：

$$\|P(t) - P_c\| - r = 0$$

$$\|R_o + tR_d - P_c\| = r$$

$$(R_o + tR_d - P_c) \cdot (R_o + tR_d - P_c) = r^2$$

$$t^2(R_d \cdot R_d) + 2t(R_d \cdot (R_o - P_c)) + (R_o - P_c) \cdot (R_o - P_c) - r^2 = 0$$

由于 R_d 是单位向量：

$$t^2 + 2t(R_d \cdot (R_o - P_c)) + (R_o - P_c) \cdot (R_o - P_c) - r^2 = 0$$

记为： $t^2 + 2tb + c = 0$

解得： $t = -b \pm \sqrt{b^2 - c}$

光线与球面求交

之前的代数方法存在可能改进和加速的地方，例如，注意到我们实际上并不需要计算被挡住的第二个交点。

下面的几何方法则具有如下优点：

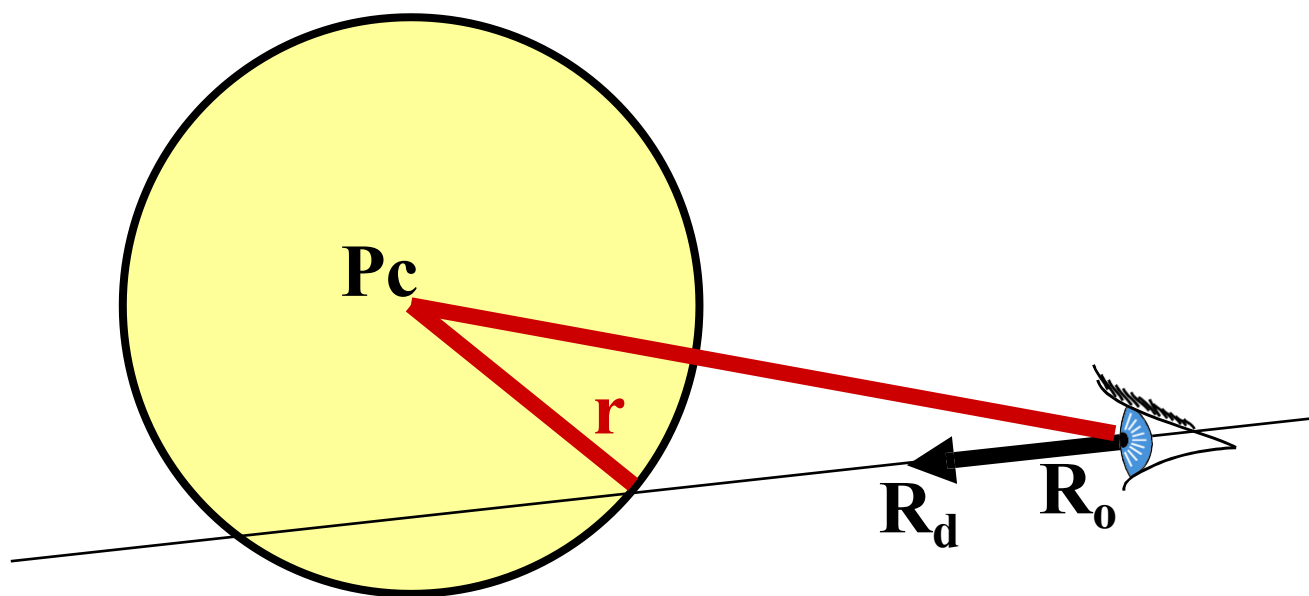
- 能快速判断光线与球面是否相交。
- 能快速判断光源是否在球体内部。
- 能快速判断光线的方向是朝向是远离球面。

光线与球面求交——几何法

几何方法：

计算光线起点到球心中心的向量 $\vec{l}$

$$l = P_c - R_0$$

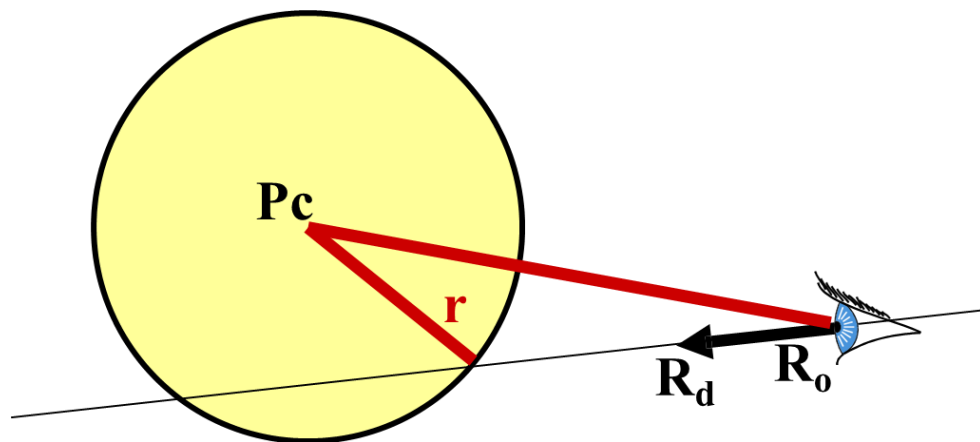


光线与球面求交——几何法

几何方法：

判断光线起点是否在球的内部

位于球的内部：	$l^2 < r^2$
位于球的外部：	$l^2 > r^2$
位于球上：	$l^2 = r^2$



光线与球面求交——几何法

几何方法：

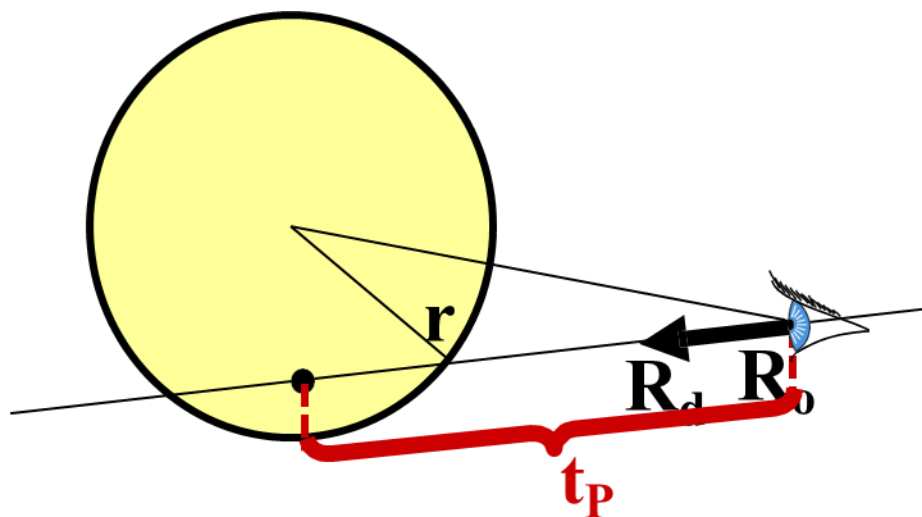
找到距离球心最近的点：

$$t_p = l \cdot R_d$$

If 起点位于球外 & $t_p < 0 \rightarrow$ 不相交

球位于光线起点的后方

光线与球不相交



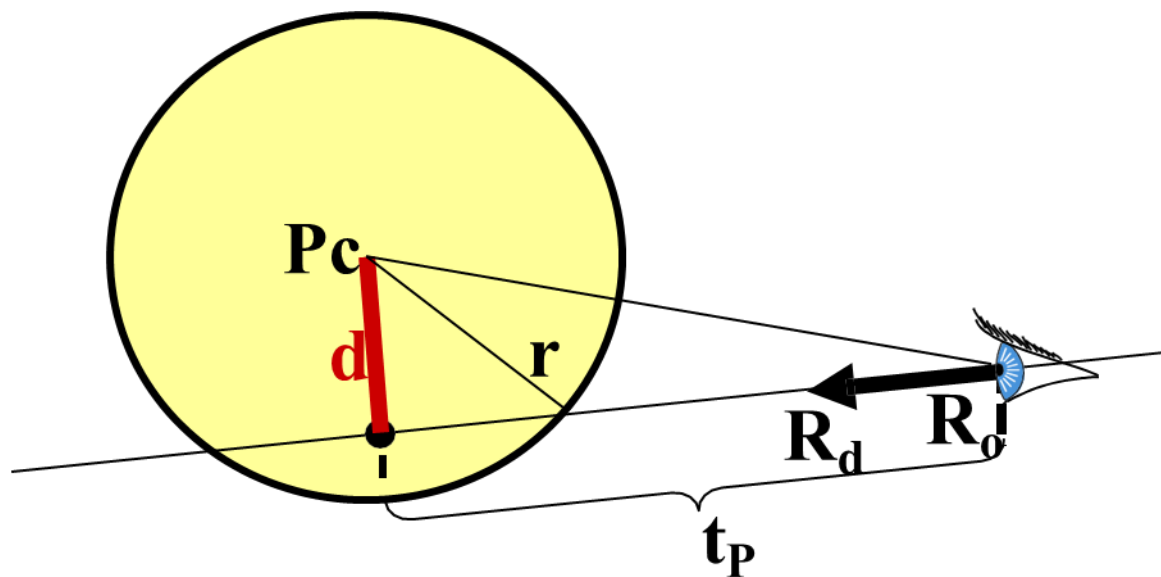
光线与球面求交——几何法

几何方法：

找到距离球心最近的点与球心的距离 d 的平方：

$$d^2 = l^2 - t^2$$

If $d > r \rightarrow$ 不相交



光线与球面求交——几何法

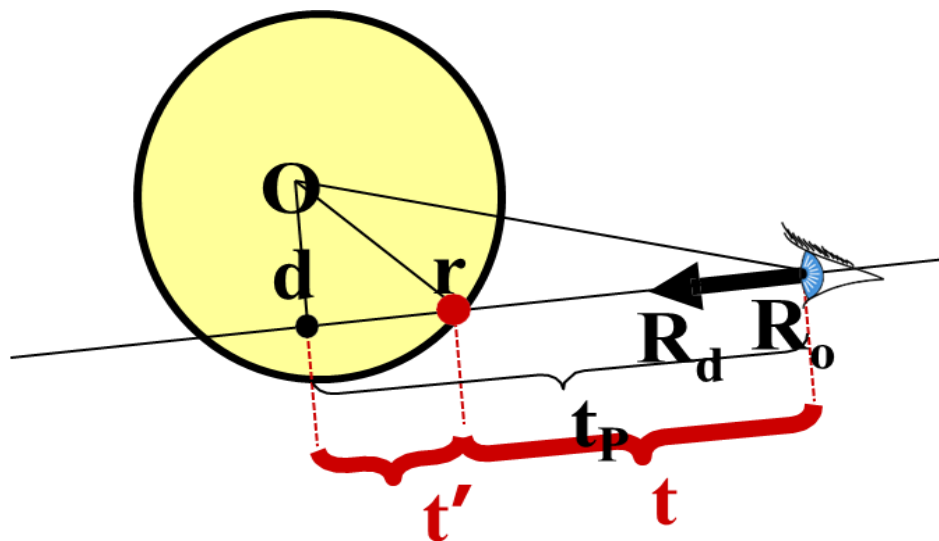
几何方法：

求最近点与相交点的距离：

$$t'^2 = r^2 - d^2$$

解为： If 光线起点位于球外部 $\rightarrow t = t_p - t'$

If 光线起点位于球内部 $\rightarrow t = t_p + t'$



光线与多边形求交

尽管三角形是最为常用的基本渲染单元，但是光线与多边形的求交也是经常需要用到的。

一个 n 多边形可以表示成 n 个顶点的有序列表： $\{v_0, v_1, \dots, v_{n-1}\}$ ，且该多边形的 n 条边分别为 $v_0v_1, v_1v_2, \dots, v_{n-2}v_{n-1}, v_{n-1}v_0$ 。其中 $v_0, v_1, \dots, v_{n-1}$ 位于同一平面，该平面可以表示为：

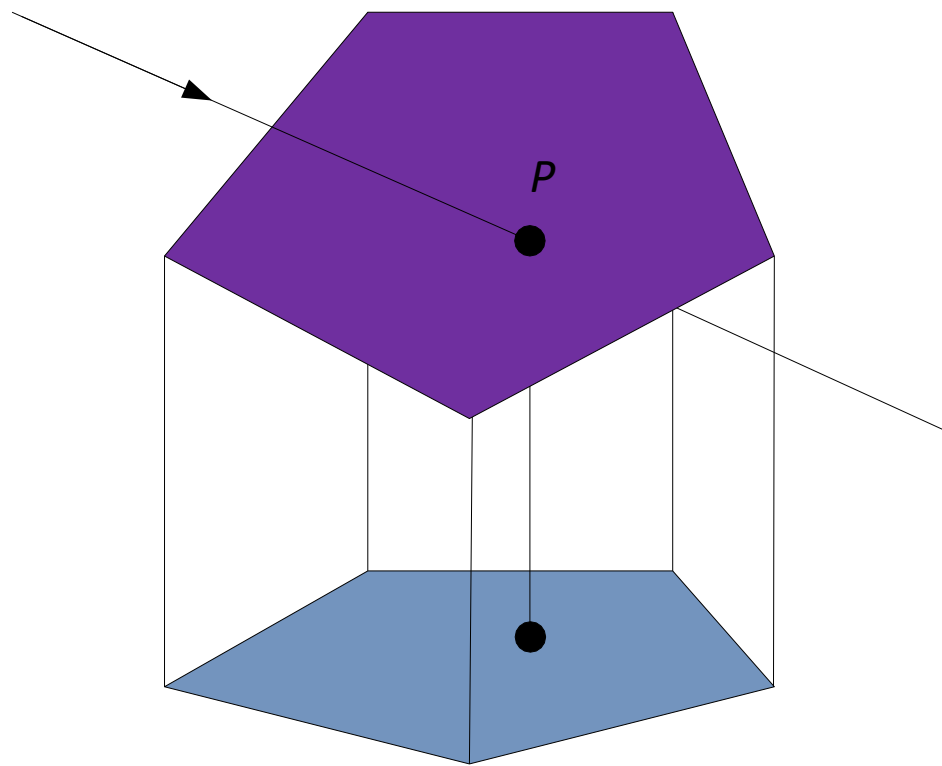
$$\pi_p: n_p \cdot x + d_p = 0$$

光线与多边形求交

我们首先计算光线与多边形所在平面的交点。

如果交点存在，我们需要判断这个交点是否位于多边形的内部。

为了进行这一判断，我们将交点以及多边形的所有顶点投影到 XY-, YZ-, ZX-平面中一个 (如图所示)，以方便后续处理。



光线与多边形求交

在二维情况下，判断一个点是否在一个多边形的内部，可以使用交点检测算法 (Crossing Test)。

交点检测算法基于如下 Jordan 曲线定理：平面上一个点位于一个多边形的内部，当且仅当由该点出发的任何一条射线都与多边形的边界有奇数个交点。交点检测算法也叫做奇偶检测算法。

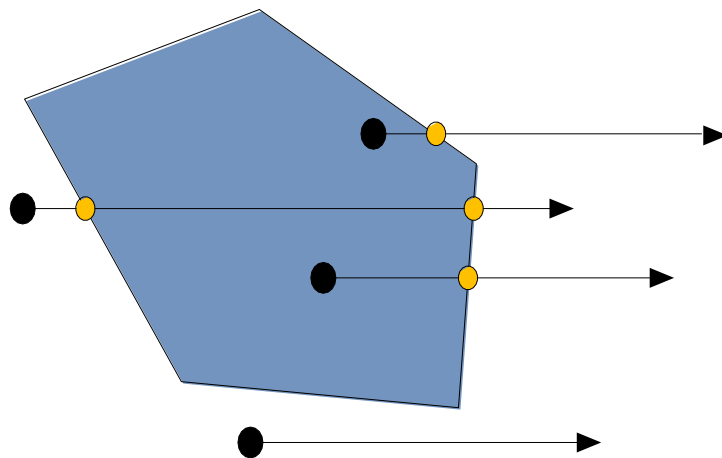
光线与多边形求交

交点检测算法如图所示：

多边形内的两个黑点分别与多边形边界相交1次。

多边形外的两个黑点分别与多边形边界相交0次和2次。

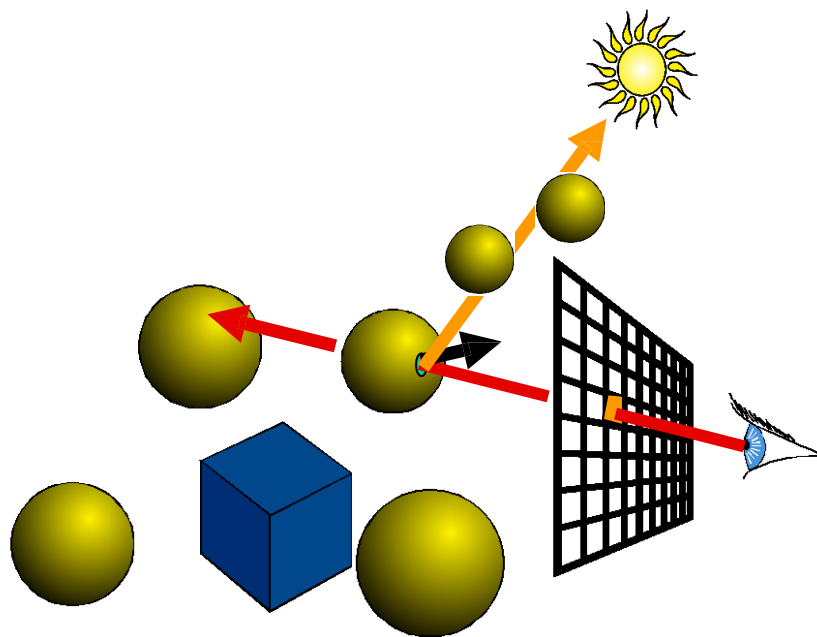
交点检测算法是所有不进行预处理的方法中最快的。



添加阴影 (Shadows)

如图所示，由交点向光源方向投射一道光线，如果这道光线与场景中物体有交点，则该交点属于阴影区域。

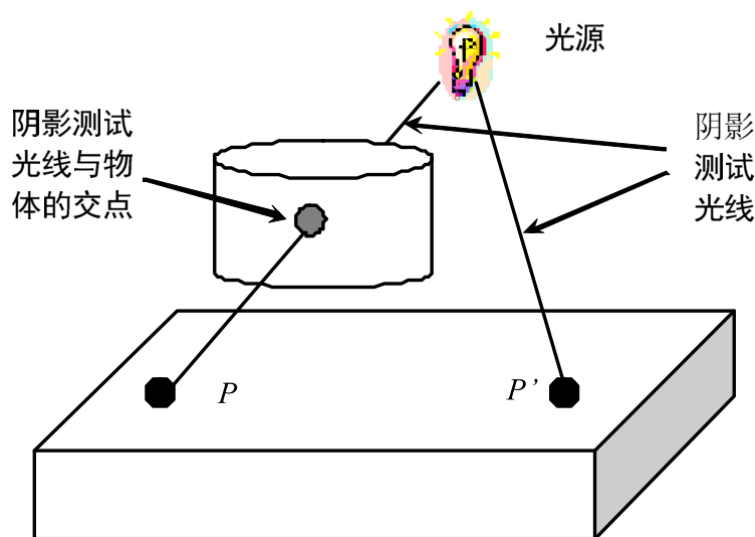
计算阴影区域时，我们只关心是否与物体相交，而不关注哪个是最近交点。



阴影计算 (Shadows)

从P出发向光源L发射一条阴影测试光线R:

- 若R在到达L的途中与场景中的物体不相交，则点P受光源L直接照射
- 反之，点P被位于它与光源L之间某一物体所遮挡
- 若遮挡物为不透明体，则点P位于光源阴影之中



阴影计算 (Shadows)

包含阴影计算的Phong模型:

$$I(P) = K_a I_a + \sum_{i=1}^m (f_i(P) I_i (k_{d_i}(N, L) + k_s(N, H)^n))$$

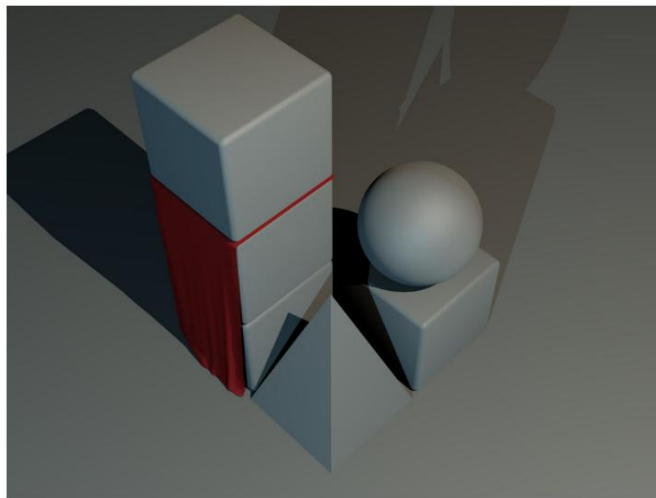
$$f_i(P) = \begin{cases} 0 & \text{若 } P \text{ 未受光源 } i \text{ 直接照射} \\ 1 & \text{若 } P \text{ 受光源 } i \text{ 直接照射} \end{cases}$$

阴影计算 (Shadows)

包含阴影计算的Phong模型:

$$I(P) = K_a I_a + \sum_{i=1}^m (f_i(P) I_i (k_{d_i}(N, L) + k_s(N, H)^n))$$

$$f_i(P) = \begin{cases} 0 & \text{若 } P \text{ 未受光源 } i \text{ 直接照射} \\ 1 & \text{若 } P \text{ 受光源 } i \text{ 直接照射} \end{cases}$$



光线跟踪阴影效果

走样

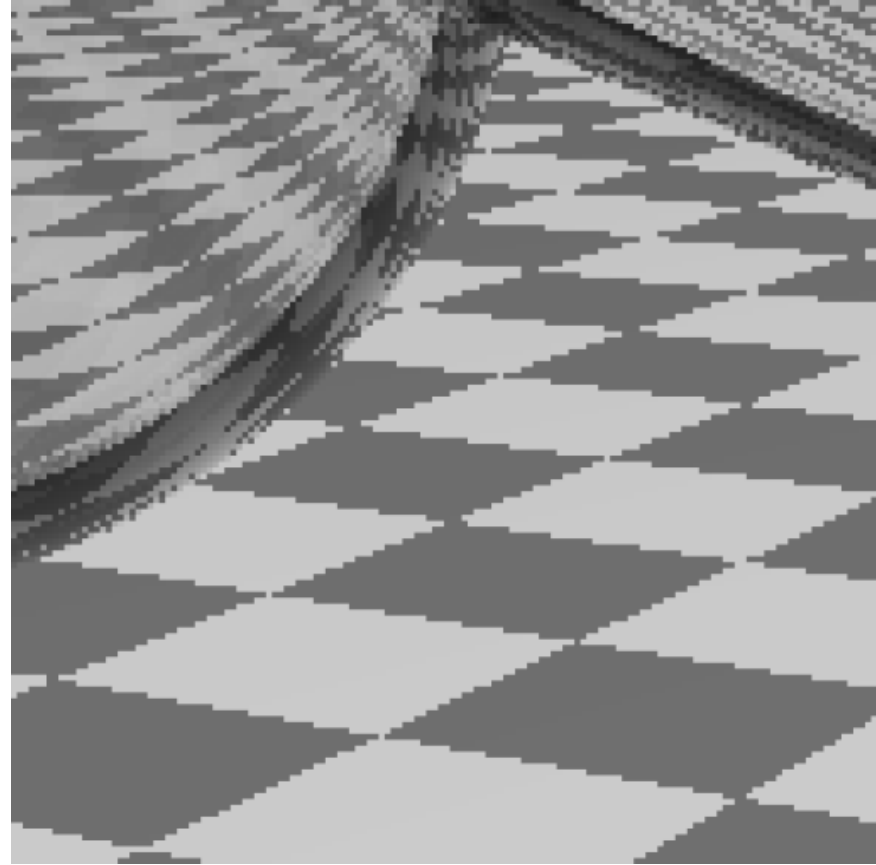
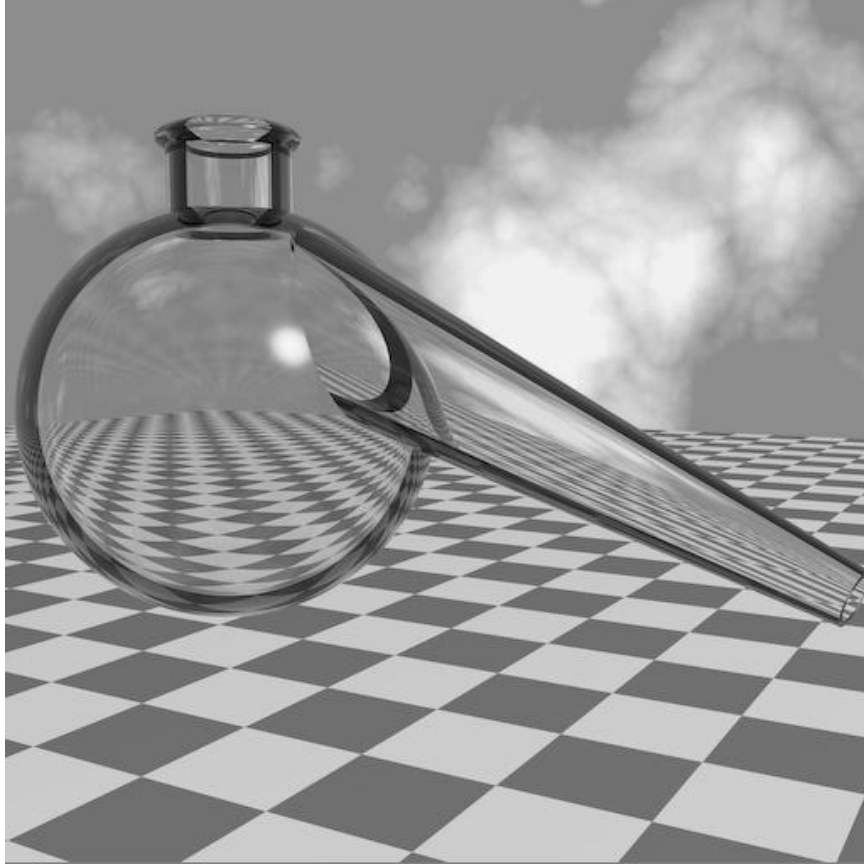
走样：用离散的量（像素）表示连续的量（图形）而引起的失真

- 数学上的点、直线是无宽度、面积的
- 像素是有面积的

光栅图形的走样现象：

- 阶梯状边界
- 图形细节失真
- 狭小图形丢失
- 动画序列中时隐时现，产生闪烁

走样



走样现象

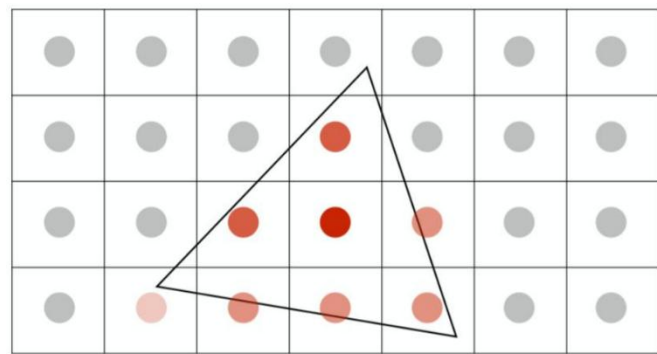
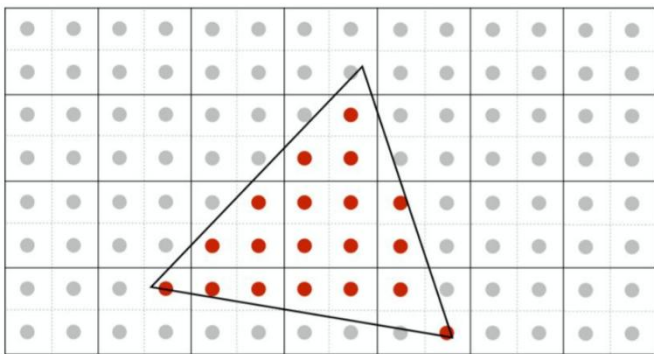
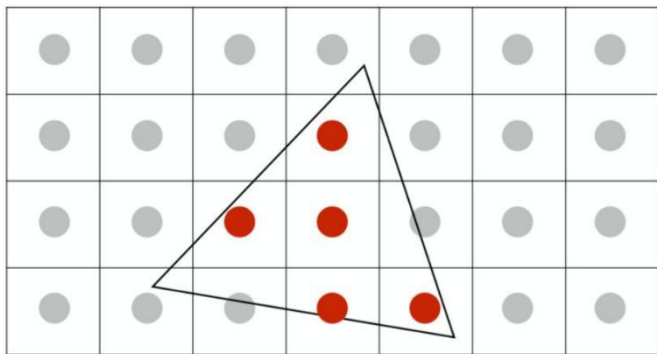
反走样

超采样（超级采样抗锯齿）

- 把当前分辨率成倍提高，然后再把画缩放到当前的显示器上
- 有限离散像素点逼近结果不好，用更多的采样点去逼近得到更好的效果

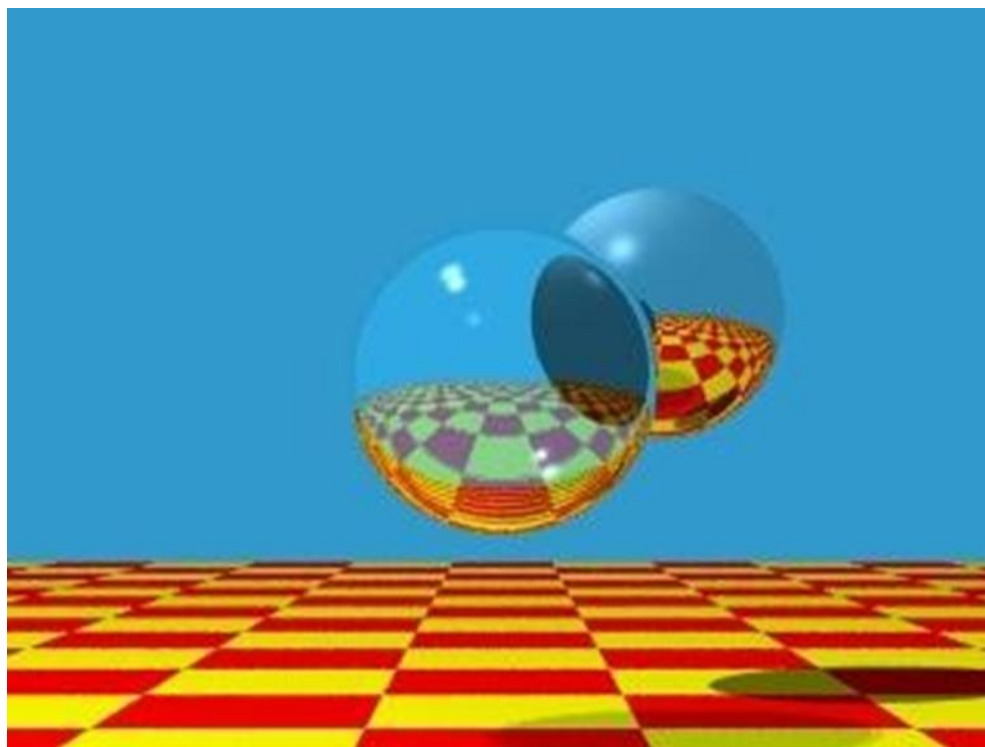
自适应超采样：

- 只在当像素在物体边沿时进行超级采样
- 减少缓存、储存带宽的消耗



添加纹理

在不同的表面添加多样化的纹理，可以使得计算机绘制的结果更加逼真。
右图的地面上添加了棋盘格形式的纹理。



添加纹理

二维纹理

– 以矩形表面为例：

为矩形的四个顶点指定二维纹理坐标。

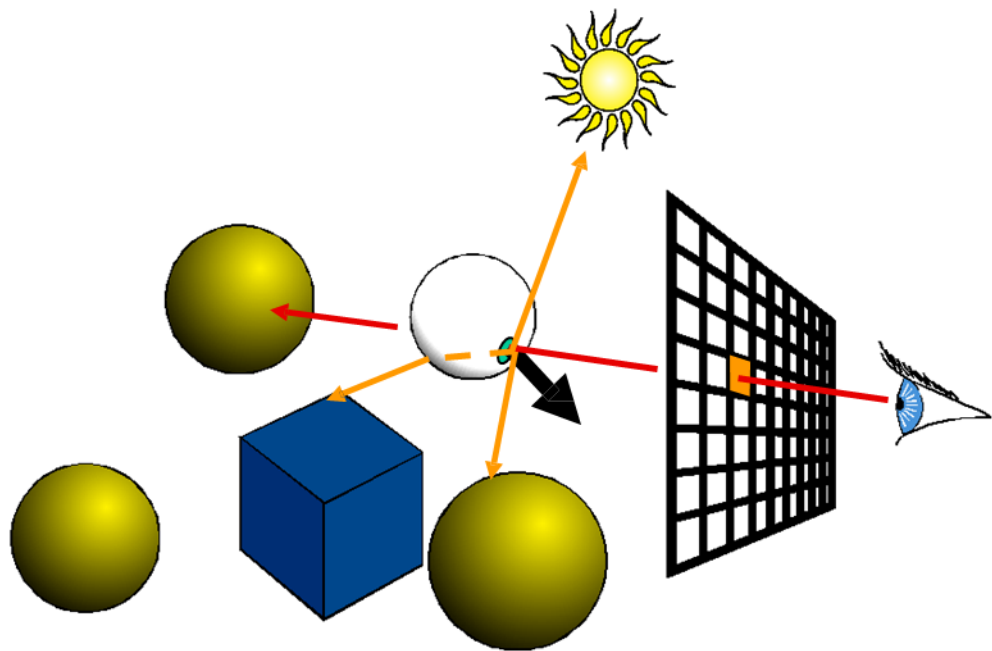
计算矩形内部与光线的交点的二维纹理坐标(双线性插值)。

使用该纹理坐标在纹理图上进行查找，根据查找结果赋予交点相应的颜色值。

光线跟踪符合物理原理吗？

真实的光子由光源出发，经场景物体弹射最终进入观察者的眼睛，而不是向光线跟踪一样。

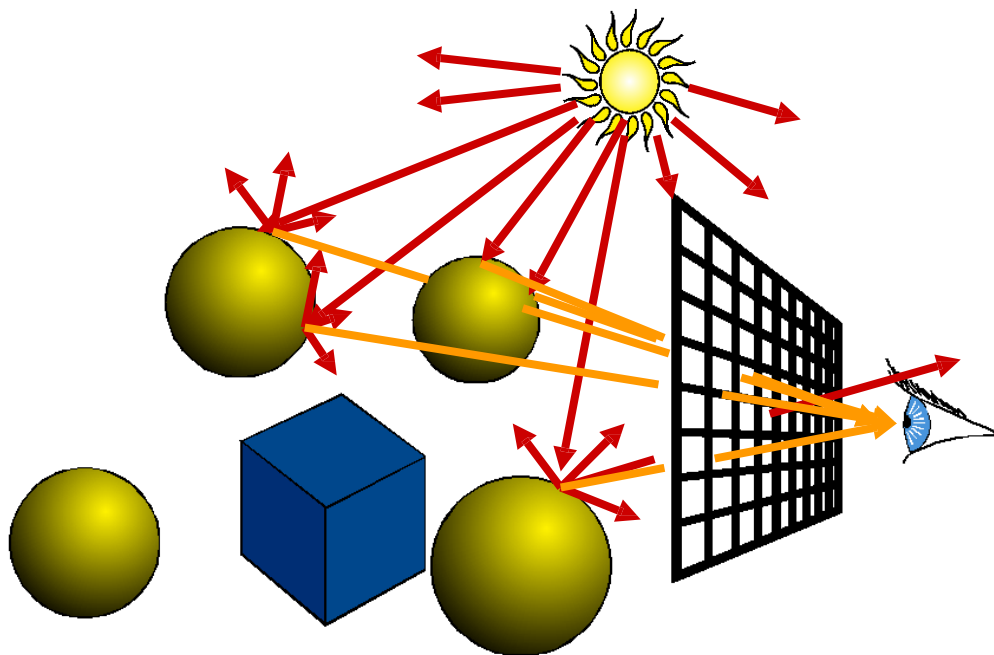
光线跟踪算法作的是反向 (Backward) 的跟踪，对视点处的光子进行跟踪直到追溯到光源。



前向 (Forward) 光线跟踪

由光源出发对光线 (光子轨迹) 进行跟踪，经过弹射最终能够抵达视点的光线是极其少的。

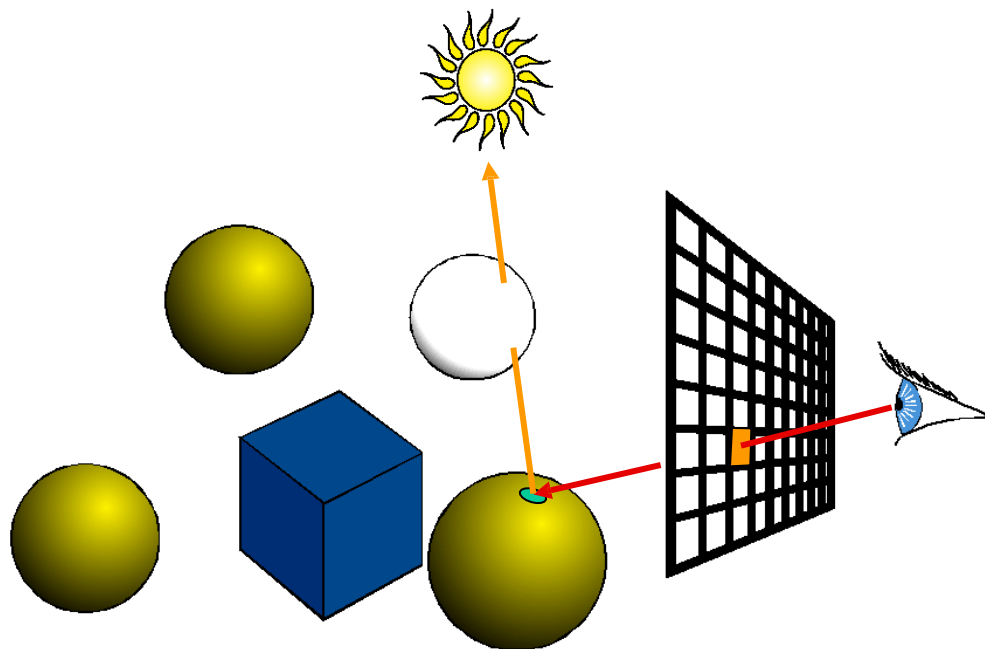
如何改进？ 每次弹射都发出一条指向视点的光线，但仍然低效。



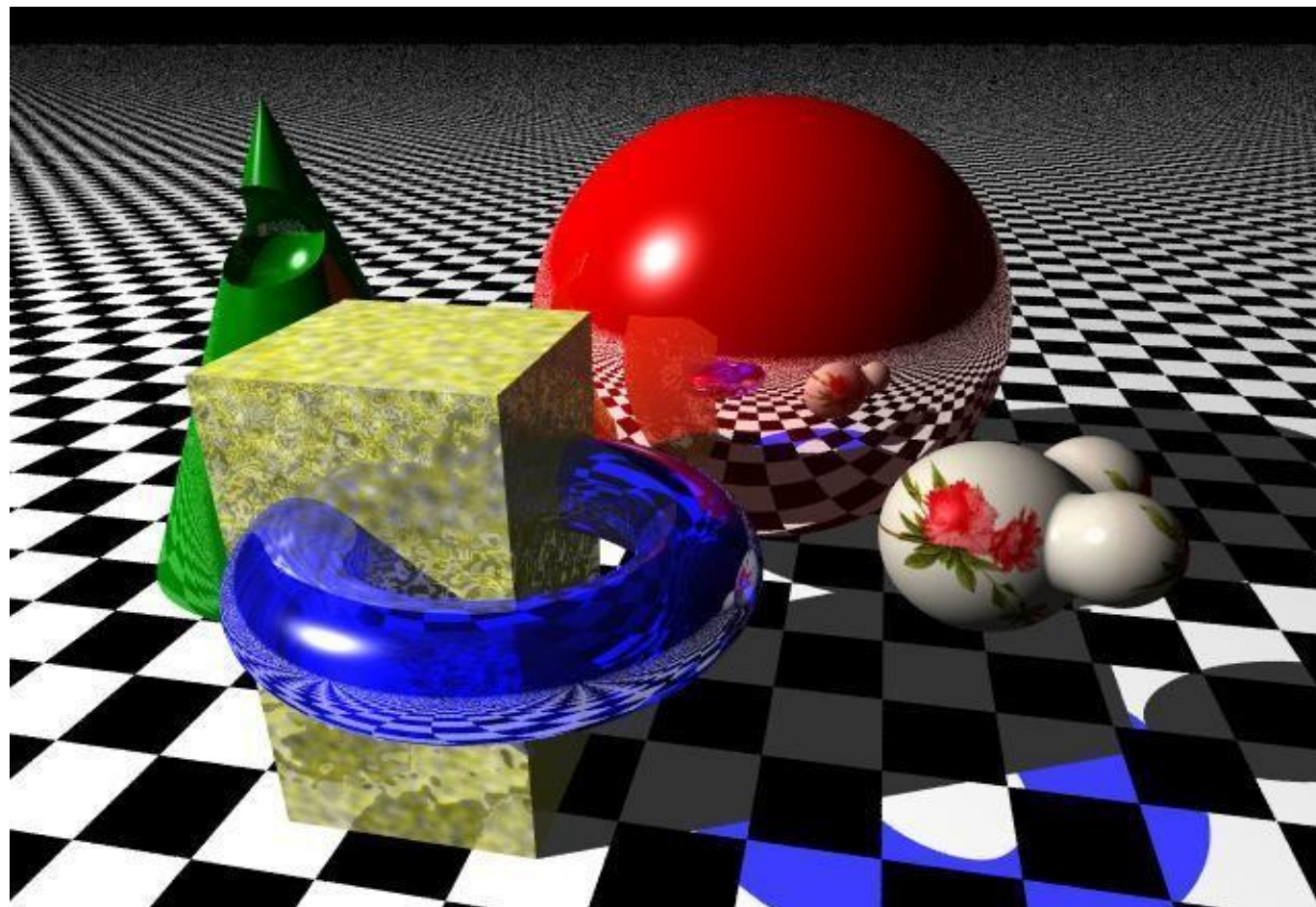
光线跟踪符合物理原理吗？

光线跟踪其实是使用许多巧妙的 Trick 来达到类似真实物理场景的效果。

例如，在计算半透明物体的阴影时：直接将光的颜色乘以遮挡物体的透明因子以达到阴影效果，而忽略光的折射。



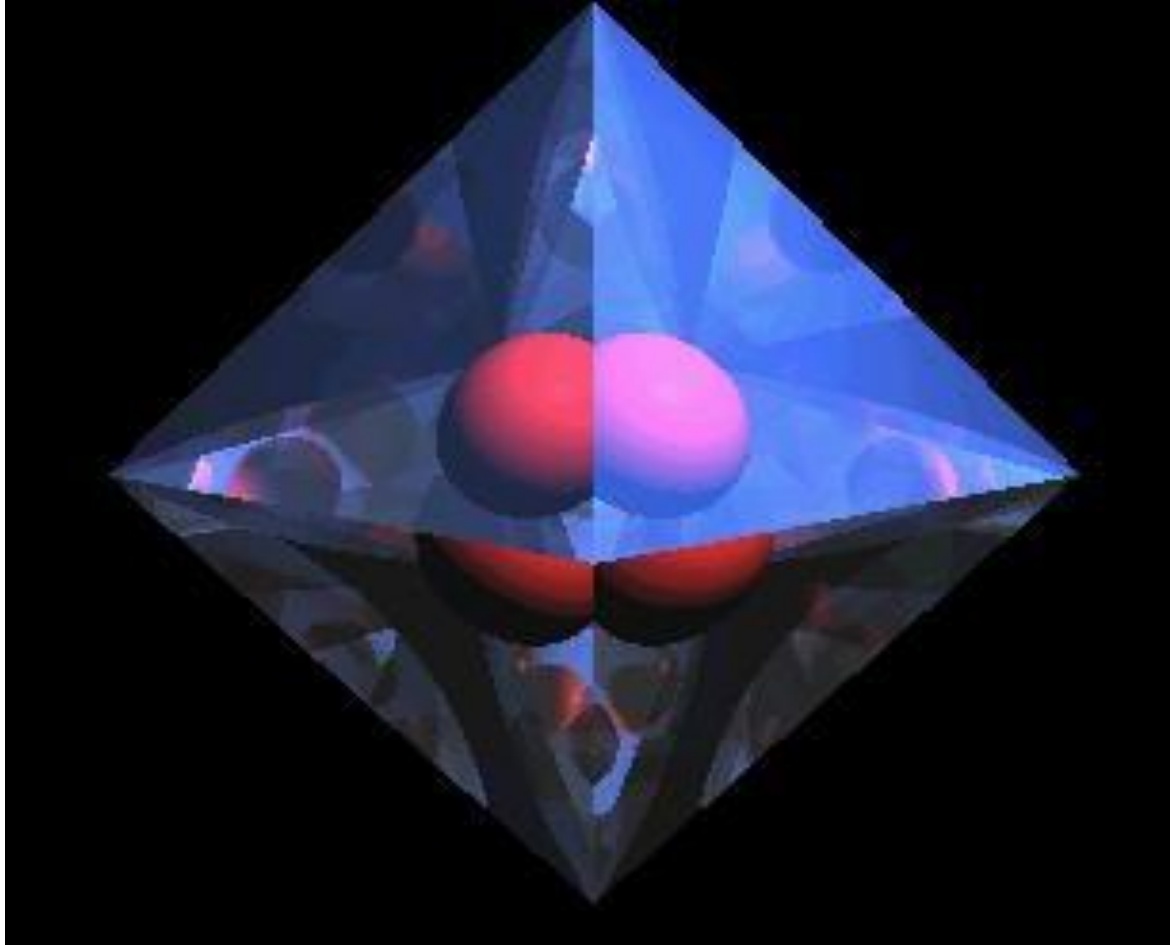
光线跟踪-效果演示



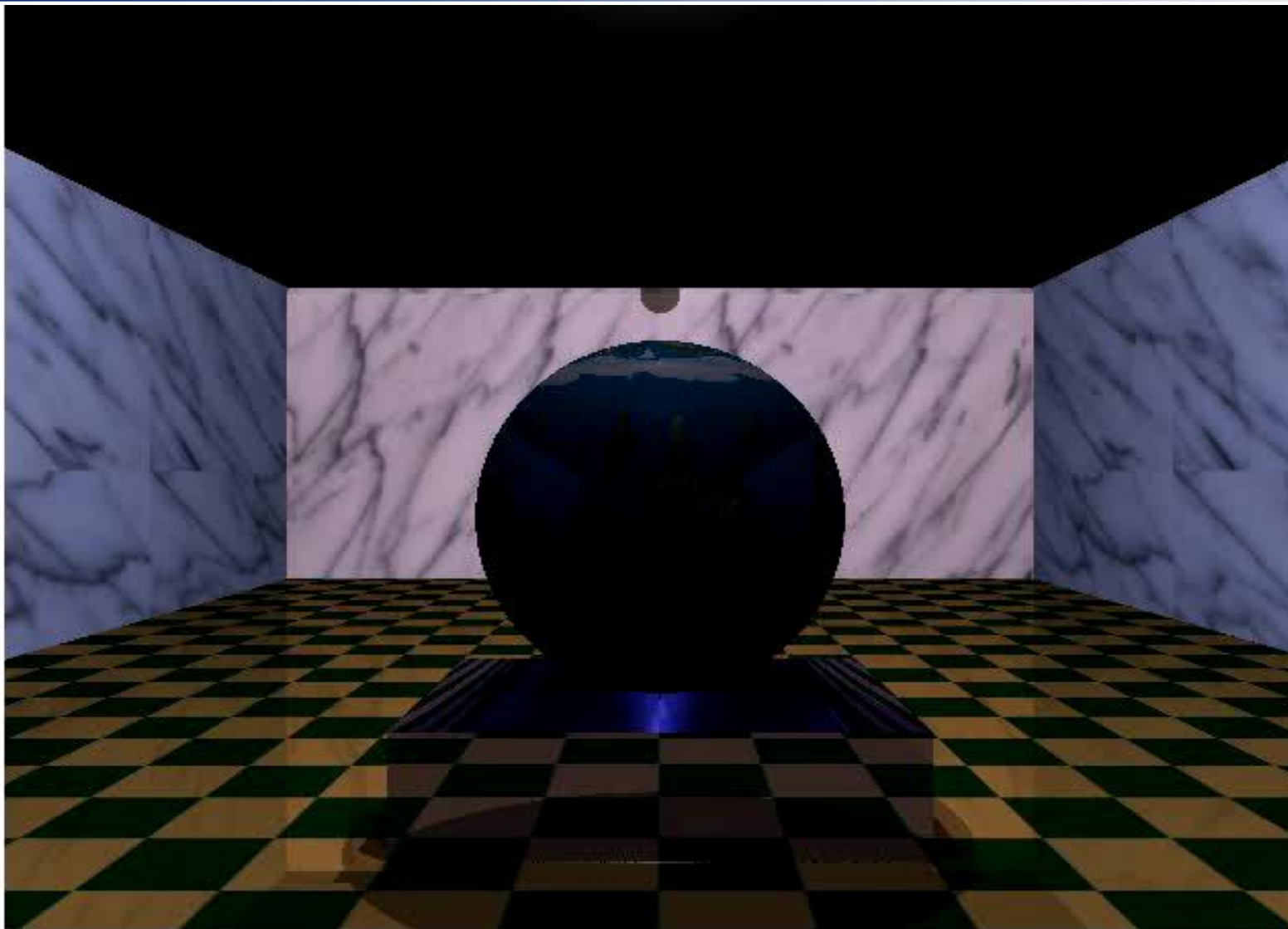
光线跟踪-效果演示



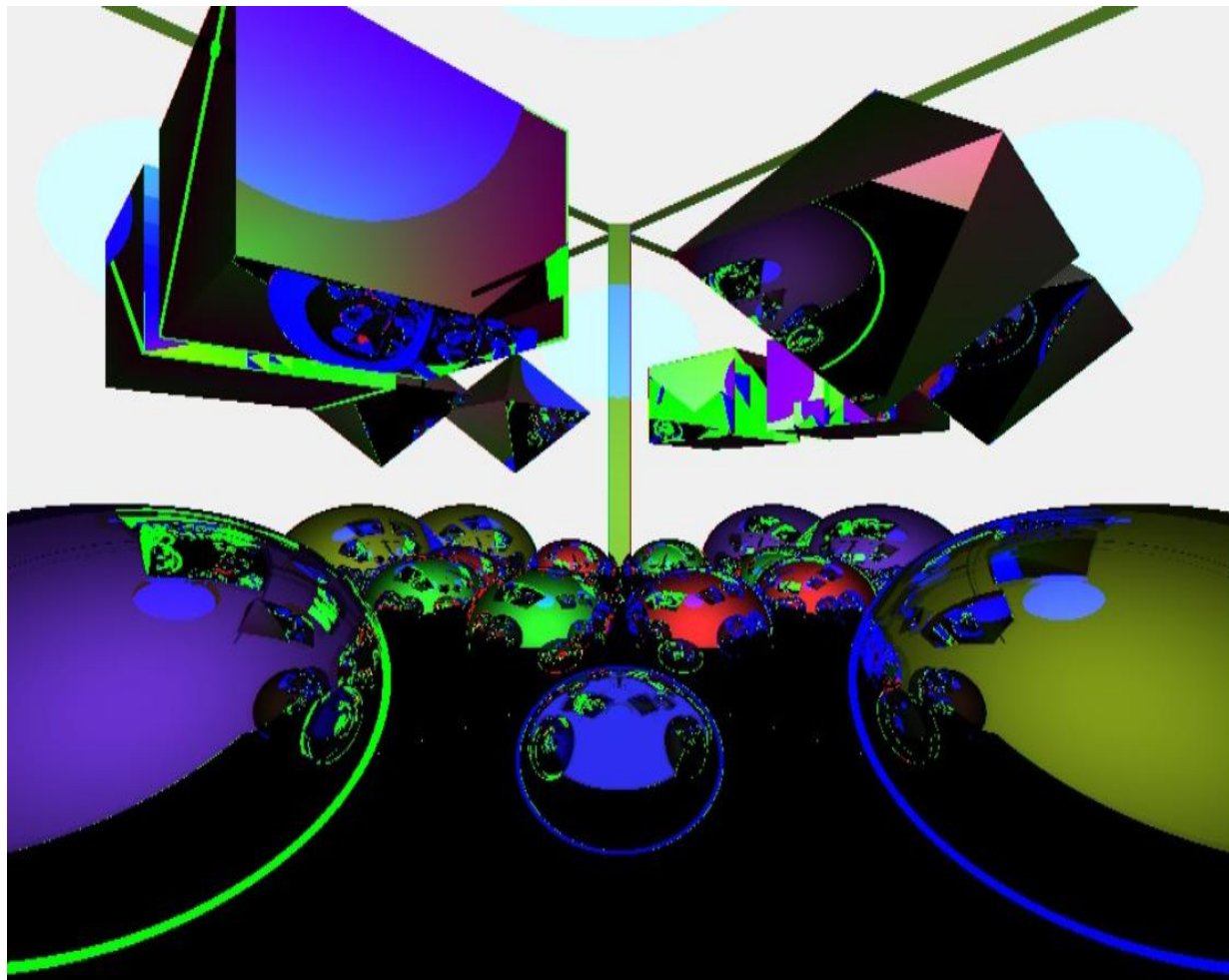
光线跟踪-效果演示



光线跟踪-效果演示

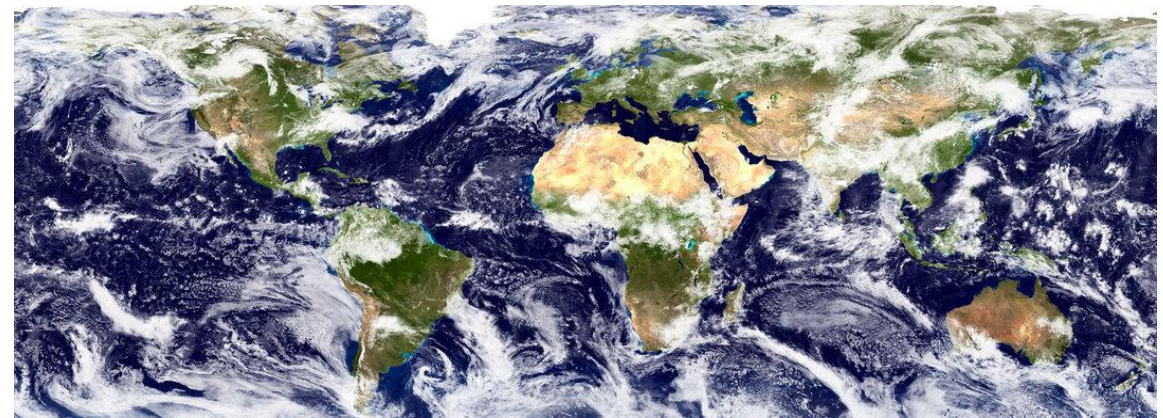
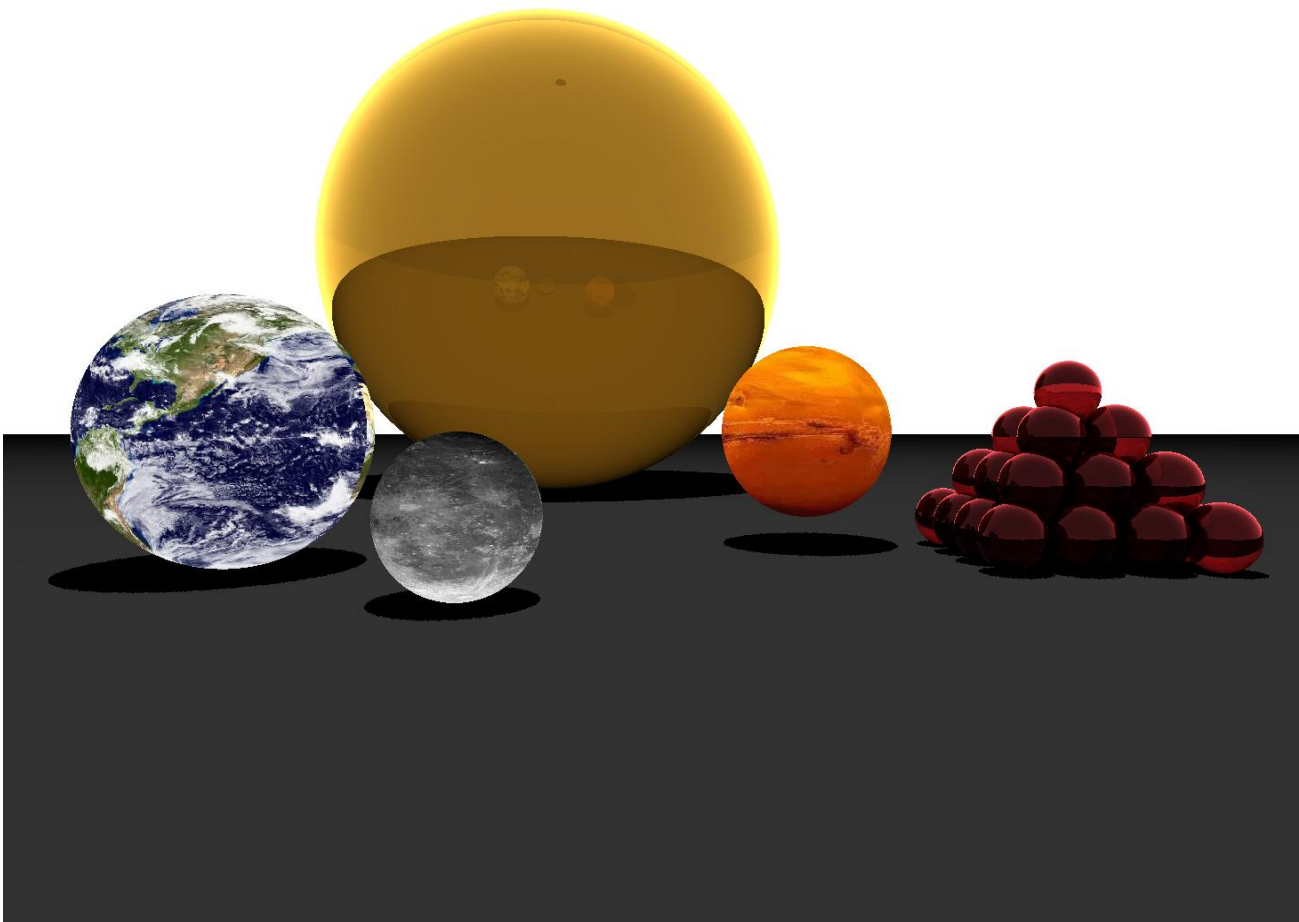


效果展示

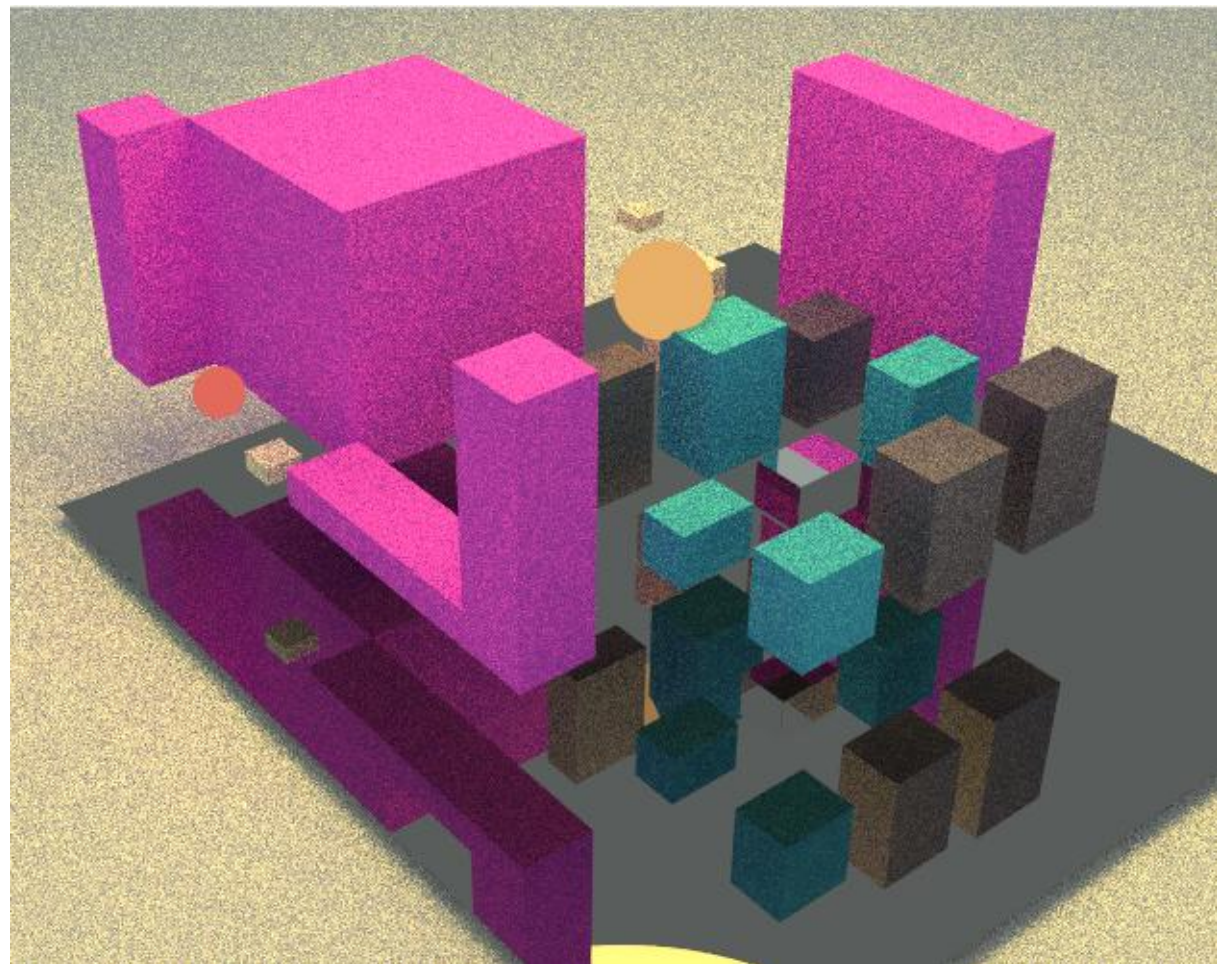
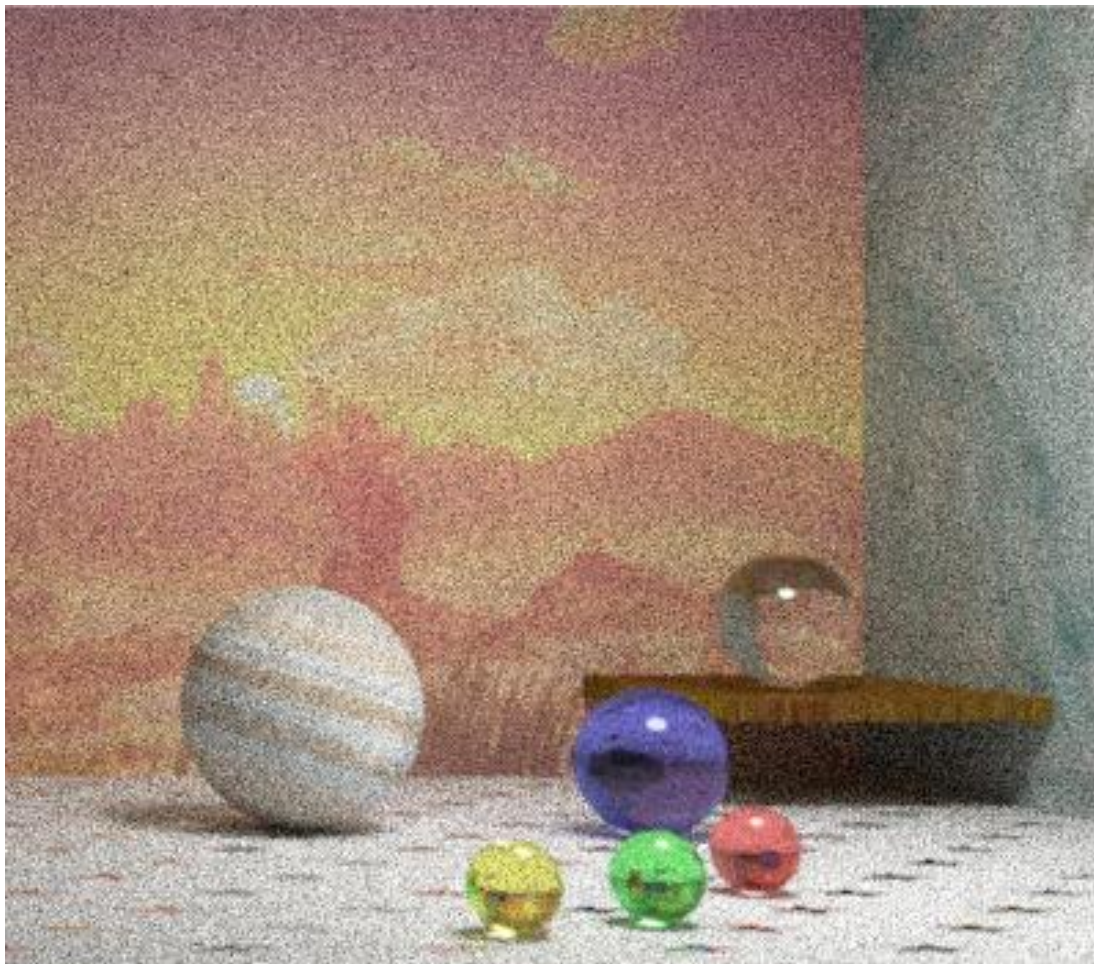




效果展示



效果展示



光线跟踪加速的原因

时间复杂度很高

- 大量的光线：

每一个像素要引出一条光线，并进行递归。

- 大量的物体或片元：

早期的光线追踪算法，光线和物体的可见性计算和求交测试需要遍历所有的物体或所有的面片。对 N 个物体，其时间复杂度为 $O(N)$ 。

- 昂贵的求交测试：

对每条光线，都需要遍历所有物体进行相交测试。

光线跟踪加速的思路

- 加速光线跟踪：

1. 降低每条光线跟踪的代价

相交测试的代价 $\times$ 物相交测试的次数

减少相交测试的次数

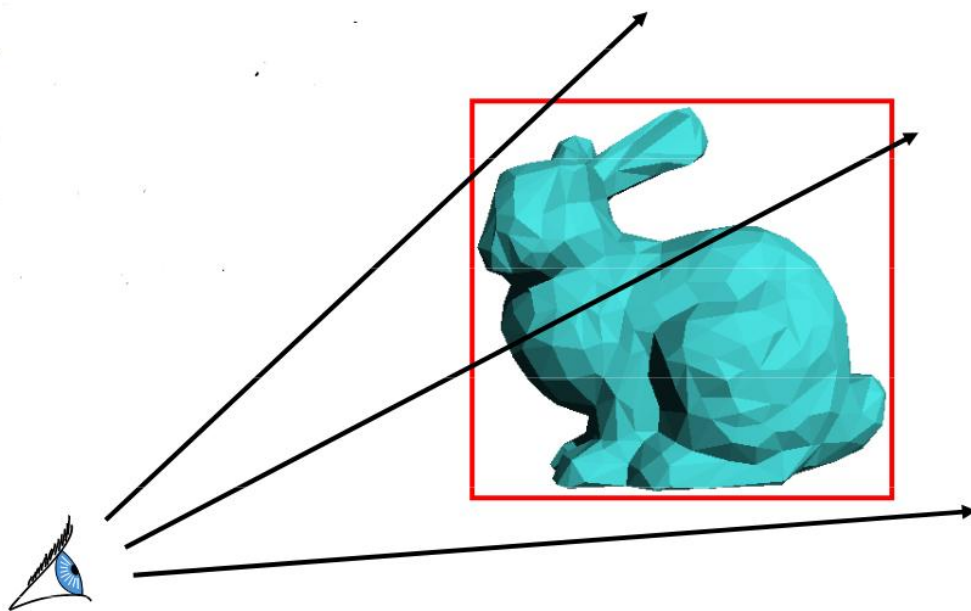
2. 减少光线数量

- 分治的技巧：

和大多数搜索问题类似，光线物体的求交可以用分治的技巧，减少光线物体的求交测试，将时间复杂度降为亚线性。常用的策略及数据结构有：包围盒技术、均匀格点法、四叉树、八叉树、空间二分树、有向包围盒、三维DDA等。

包围盒技术

- 将每一个物体（或片元）和它的包围区域（Bounding Volumes）联系起来。如果光线没有和包围区域相交，那便不会和物体（或片元）相交；如果光线和包围区域相交，再继续判断光线是否和物体（或片元）相交以及计算相交的位置。



包围盒技术

光线和包围空间的相交测试更快：

避免了光线和物体不必要的相交测试

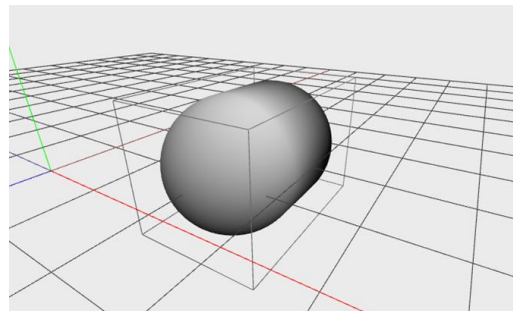
- 物体被完全包裹于立方体中
- 如果光线没有和包围区域相交，那便不会和物体（或片元）相交
- 如果光线和包围区域相交，再继续判断光线是否和物体（或片元）相交以及计算相交的位置



包围盒技术

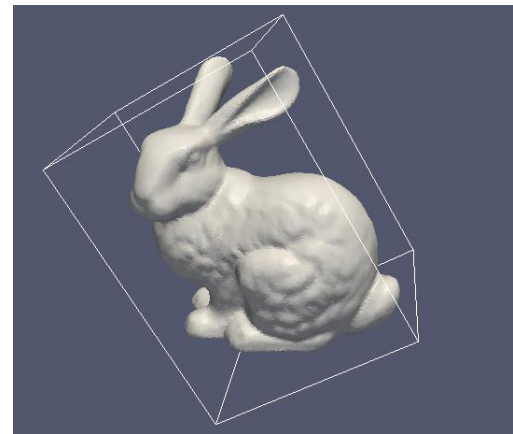
■ 轴向包围盒：

- 其包围盒面法向和坐标系的轴平行
- 轴向包围盒构建简单，但不能随物体旋转



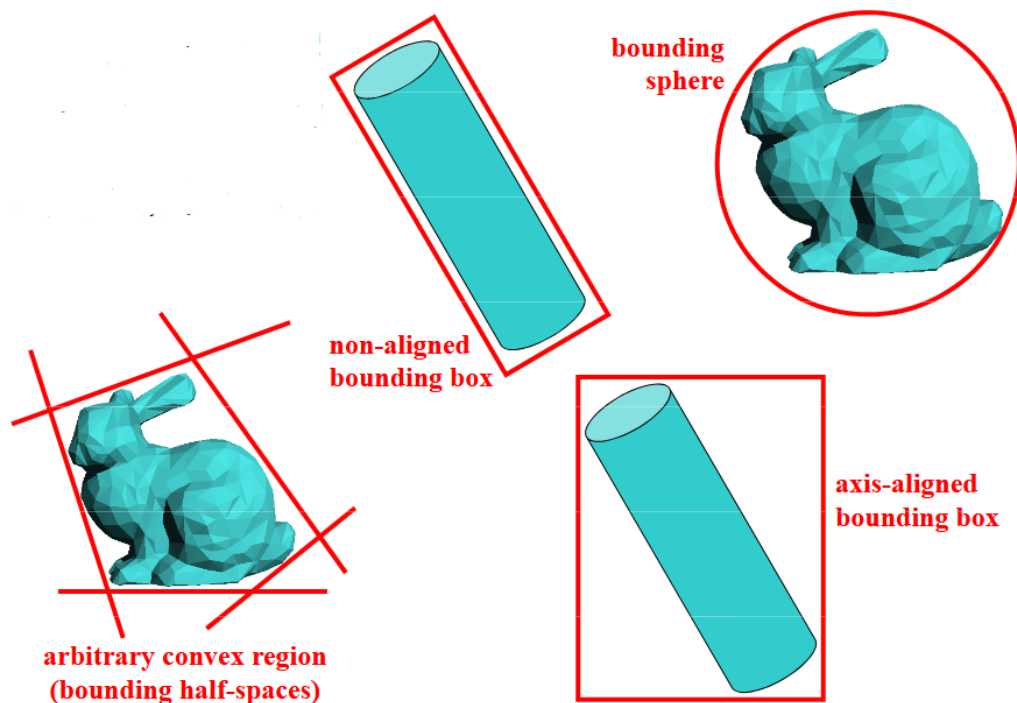
■ 方向包围盒：

- 方向包围盒是也是六面体，但不要求面方向和轴平行
- 方向包围盒可根据物体表面的顶点通过主成分分析（PCA）获得特征向量，即包围盒的主轴，进而构建出包围盒。



包围盒技术

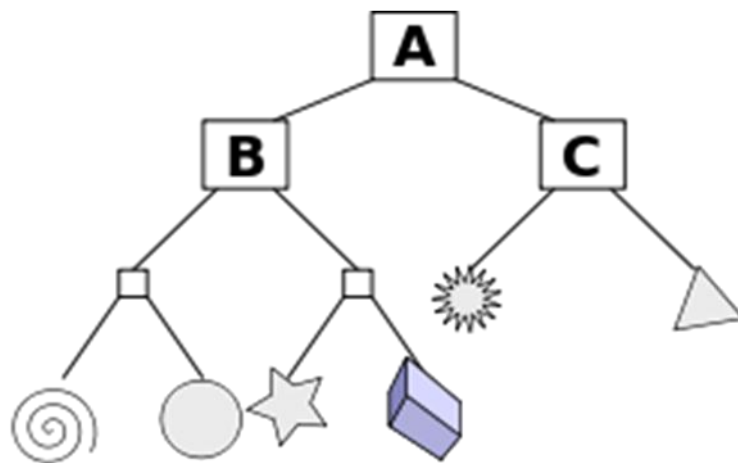
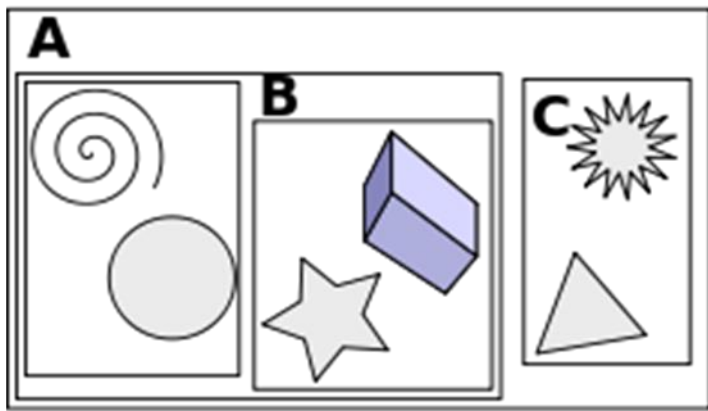
- 具体实现中，包围区域可以是（对齐或不对齐的）包围盒，包围球，包围凸多面体等；
- 合适的包围区域可以紧贴物体，更大程度地减少和物体不必要的相交测试。



层次包围盒

■ 层次包围盒：

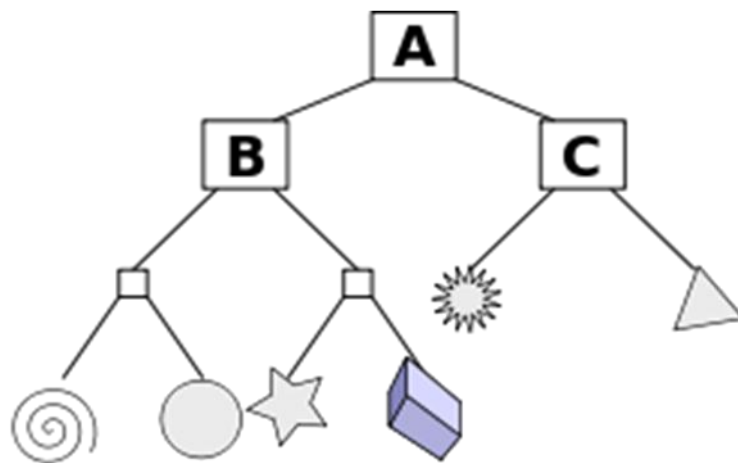
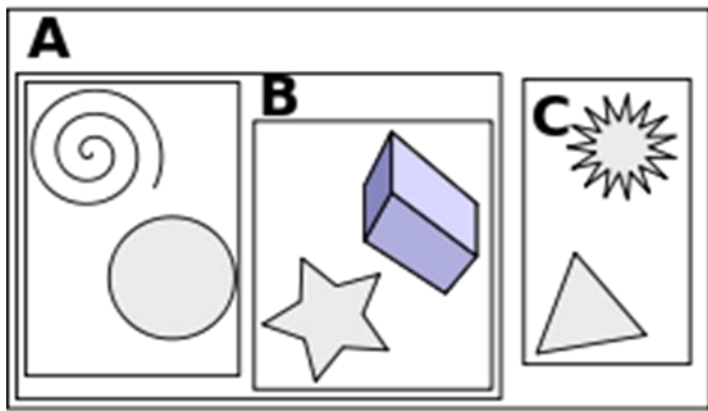
- 大量物体意味着大量的包围盒光线和包围盒的相交检测要求遍历所有包围盒，时间复杂度较大。
- 引入二叉树的结构来索引包围盒——层次包围盒



层次包围盒

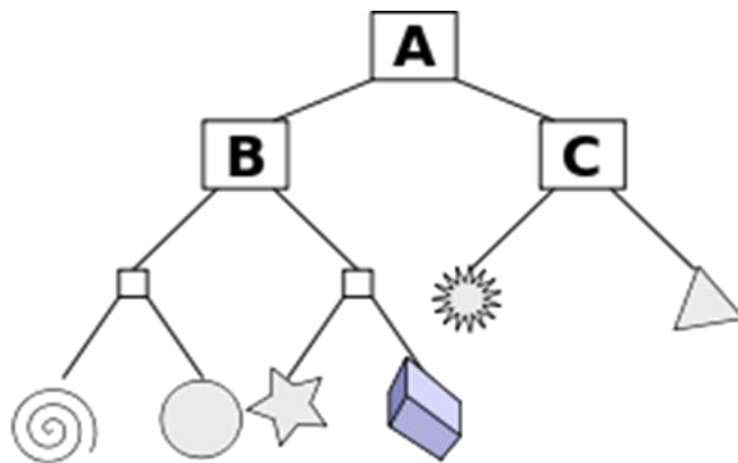
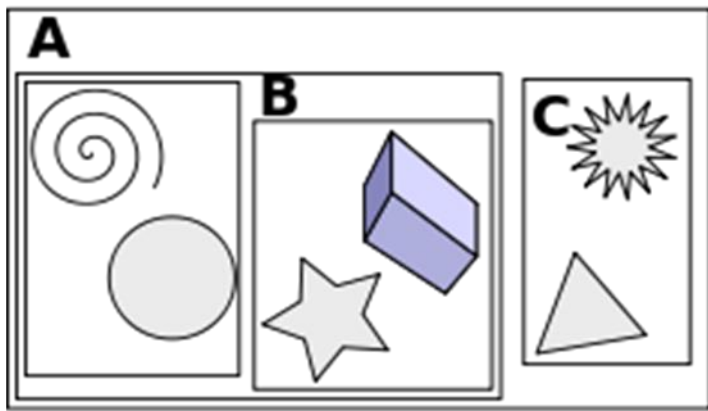
■ 数据结构:

- 层次包围盒是二叉树的数据结构。叶节点表示物体或片元，内节点表示包围盒。每个父节点包围盒包含两个子节点的包围盒。根节点包围盒则包含了所有物体。



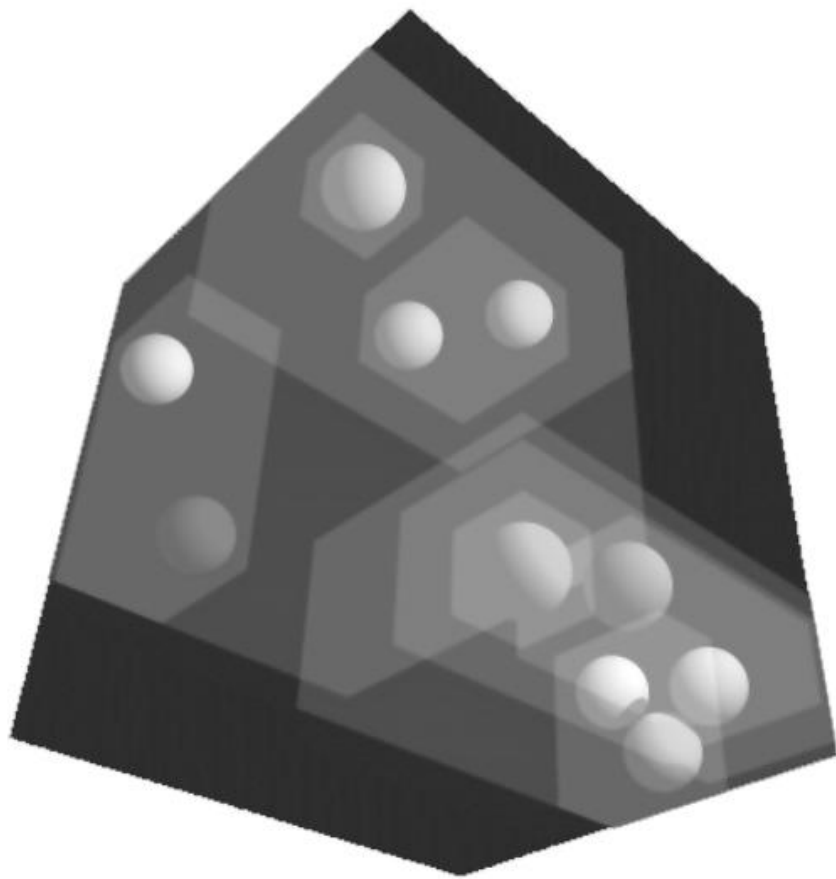
层次包围盒

- 相交测试：需要对层次包围盒树执行搜索算法，从根节点开始递归进行相交检测：
 1. 如果光线和当前节点包围盒相交，就继续和子节点做相交检测；
 2. 如果光线和当前节点不相交，返回 false（不相交）；
 3. 如果光线和物体相交，返回 true；



层次包围盒

层次包围盒示例：



层次包围盒

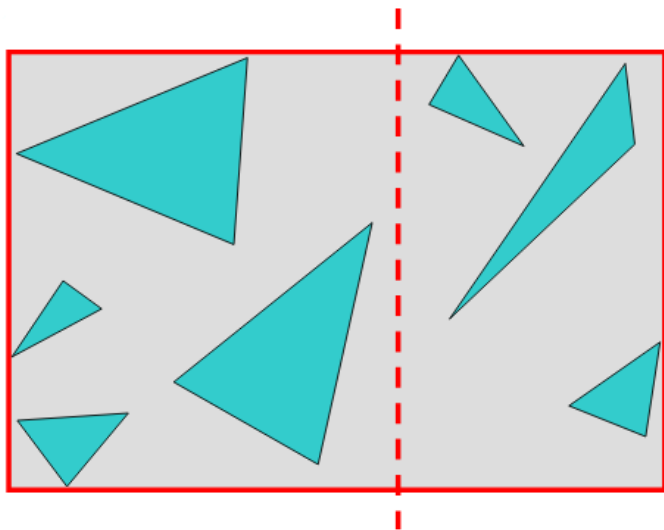
将一组物体分为两组，分割点在哪里？

1. 从中点分割；
2. 排序，两组物体按个数平均分；
3. 直接使用建模时候的层次化结构.

层次包围盒

- 层次包围盒的构建：

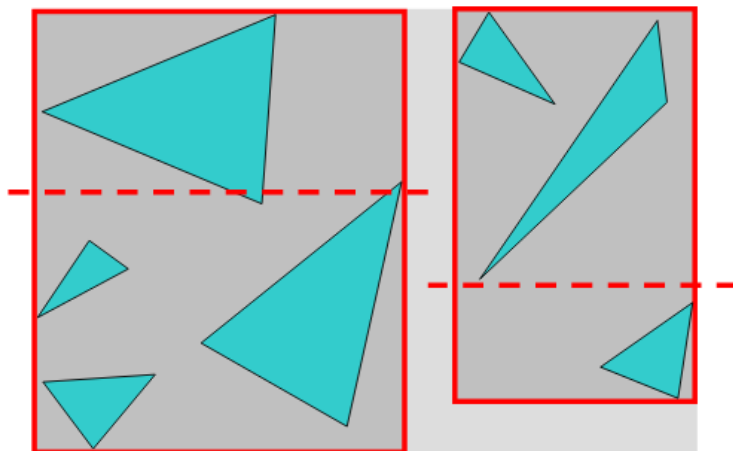
1. 求得所有物体的包围盒；
2. 将其中物体分为两组，分别求出包围盒；
3. 递归进行第二步，直到递归深度或组中只有一个物体；



层次包围盒

- 层次包围盒的构建：

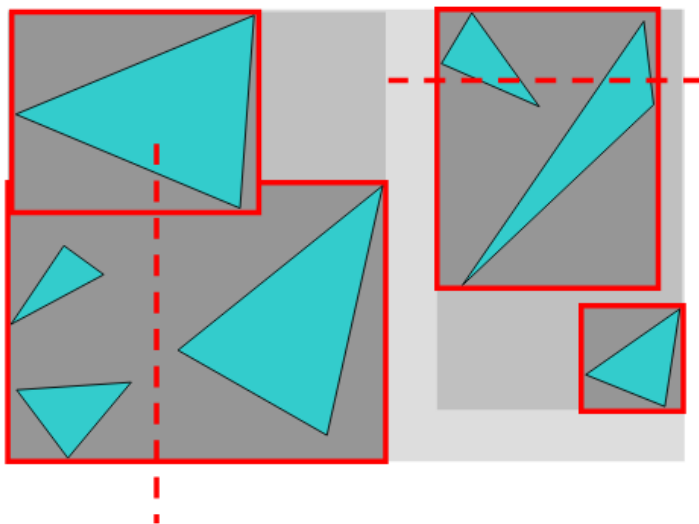
1. 求得所有物体的包围盒；
2. 将其中物体分为两组，分别求出包围盒；
3. 递归进行第二步，直到递归深度或组中只有一个物体；



层次包围盒

- 层次包围盒的构建：

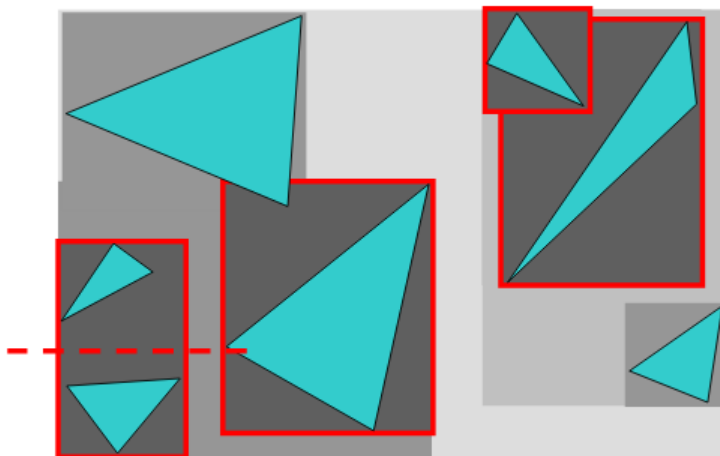
1. 求得所有物体的包围盒；
2. 将其中物体分为两组，分别求出包围盒；
3. 递归进行第二步，直到递归深度或组中只有一个物体；



层次包围盒

- 层次包围盒的构建：

1. 求得所有物体的包围盒；
2. 将其中物体分为两组，分别求出包围盒；
3. 递归进行第二步，直到递归深度或组中只有一个物体；

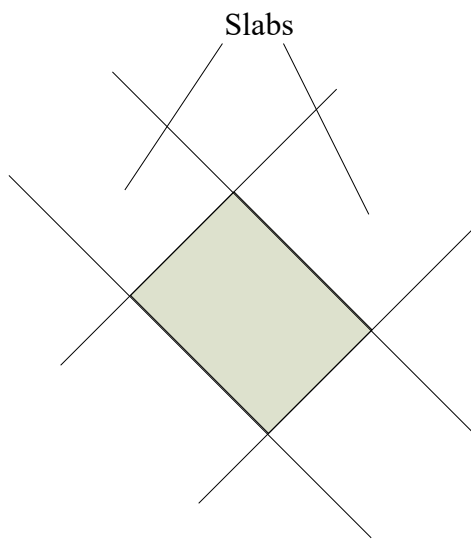


包围盒求交

- 光线与长方体的求交算法：

基于 Slab 的快速长方体求交算法：一个 Slab 是指两个平行的平面，将两个平行平面放在一组是为了计算更快。每个长方体是由三个 Slab (三组相对的面) 构成的。

以2D为示例，3D与之相同



包围盒求交

对每个 Slab, 计算其两个平面与光线的交点, 并将这两个交点按照其参数 t 的大小排序, 记为:

$$t_i^{min} \text{ 和 } t_i^{max} \quad (i = 0, 1, 2)$$

然后, 我们计算:

$$t_i^{min} = \max(t_0^{min}, t_1^{min}, t_2^{min})$$

$$t_i^{max} = \min(t_0^{max}, t_1^{max}, t_2^{max})$$

并测试: $t^{min} < t^{max}$

如果 $t^{min} < t^{max}$, 那么光线与长方体相交, 且 t^{min} 是射入长方体的交点; t^{max} 是射出长方体时的交点。否则, 光线与长方体不相交。

包围盒求交

如右图中的两条射线 (二维情况):

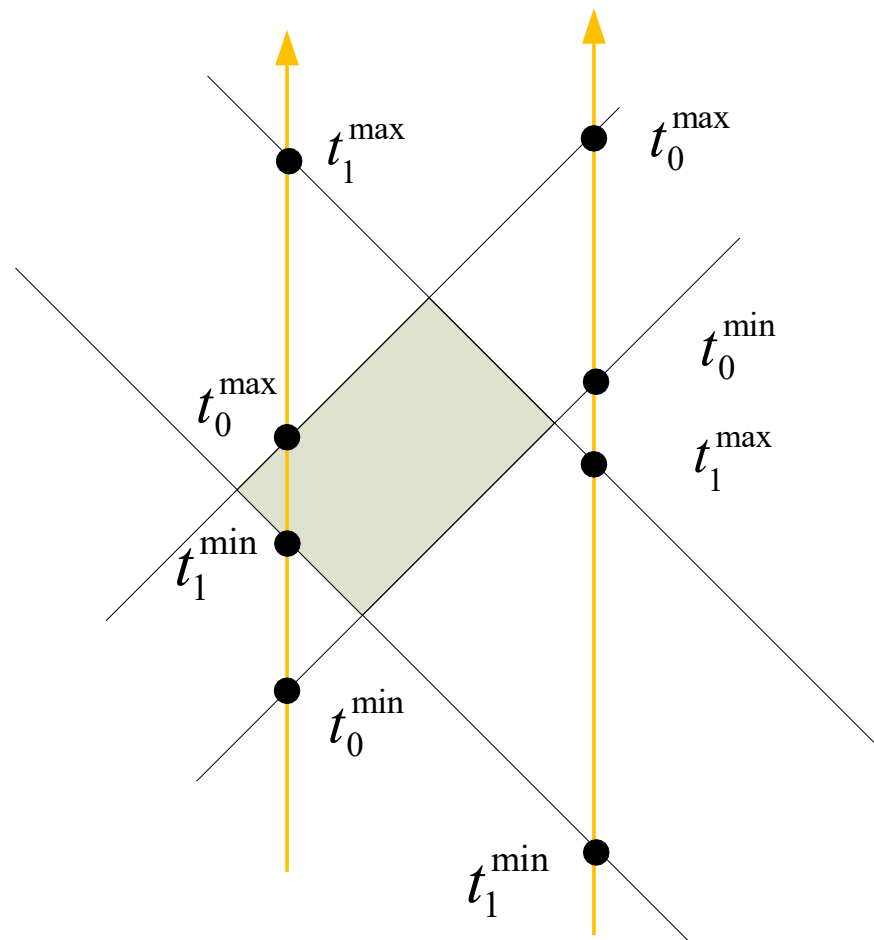
左边的射线与矩形相交, 满足:

$$t^{\min} < t^{\max}$$

右边的射线与矩形 不相交, 不满足:

$$t^{\min} < t^{\max}$$

思想上与 Liang-Barsky 裁剪算法类似。



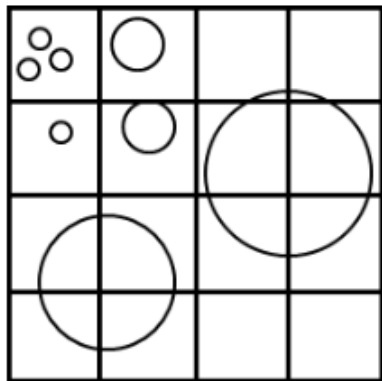
空间分割技术

■ 原理：

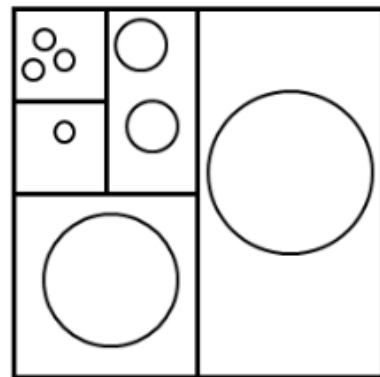
- 将景物空间分割成一个个小的空间单元
- 被跟踪的光线仅与它所穿过空间单元中所含物体表面进行求交测试
- 用相邻空间单元的空间连贯性，使一条光线快速跨越空单元，迅速到达非空单元，求得光线与景物的第一个交点

空间分割技术

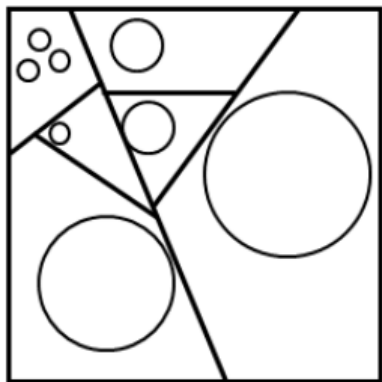
- 典型方法：



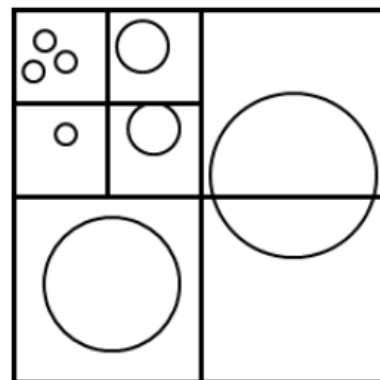
均匀网格



Kd树



BSP树

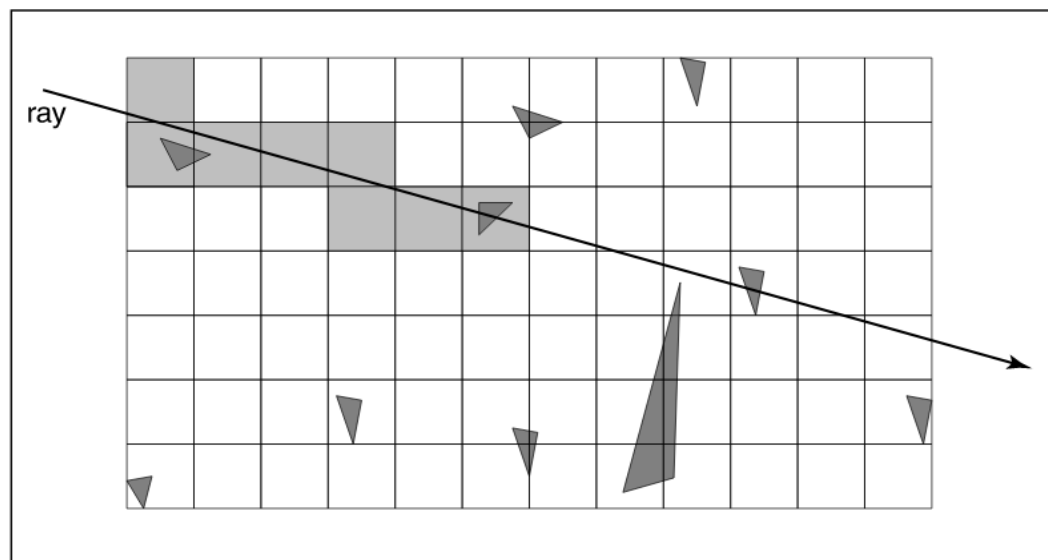
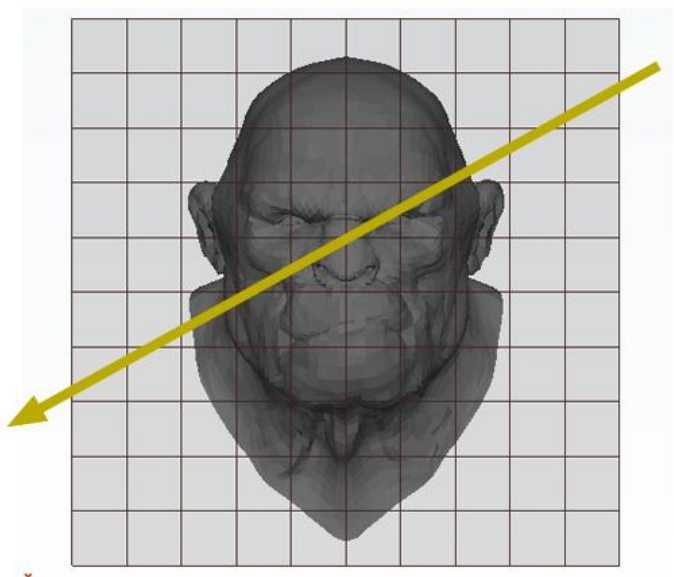


四叉树(2D)/八叉树(3D)

均匀格点法

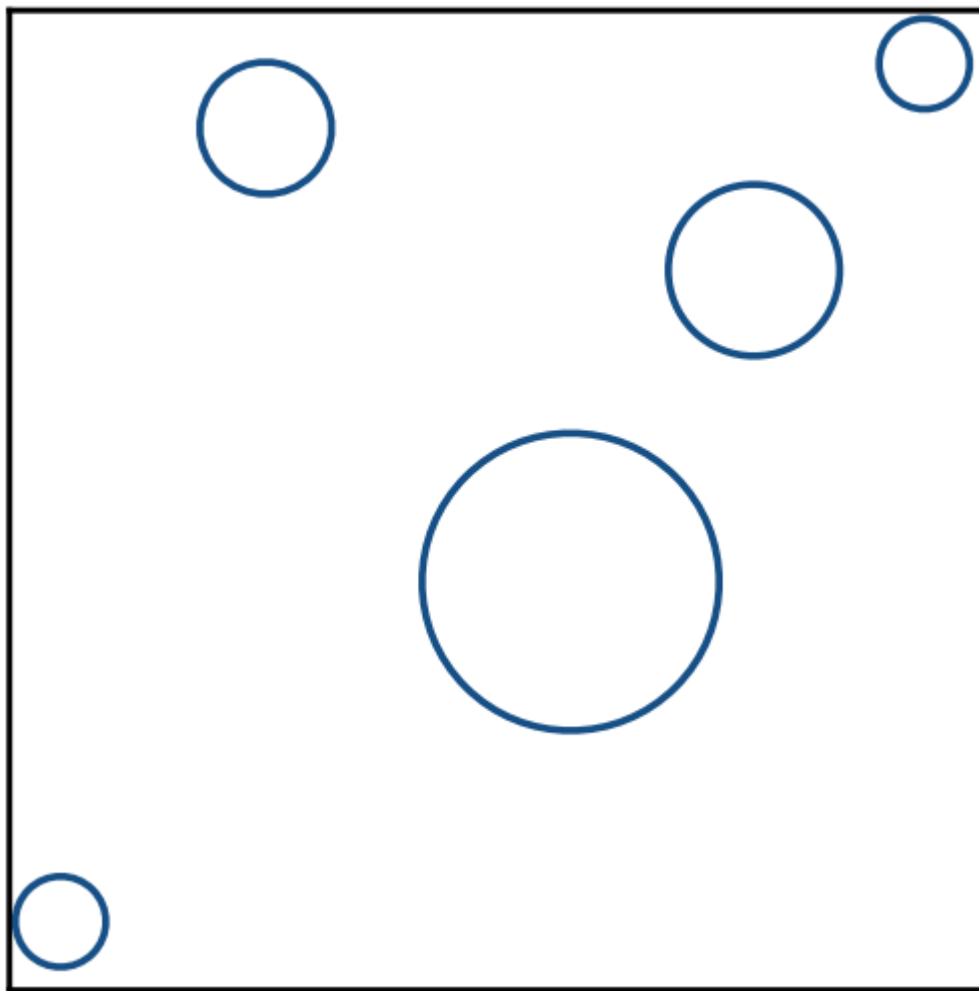
■ 均匀格点法:

- 均匀格点法将空间分为均匀的单元。光线通过单元向前追踪，直到和其中一个单元中的对象相交。



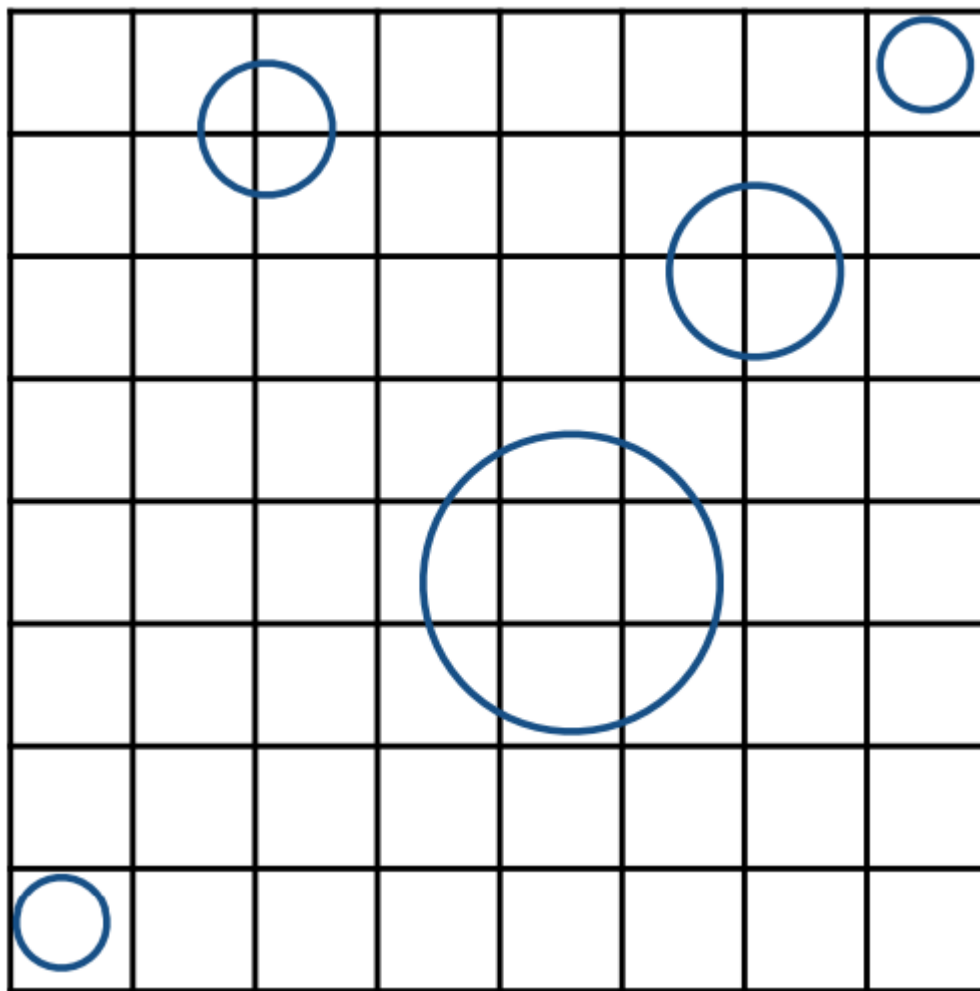
均匀格点法

1. 找到物体的包围盒



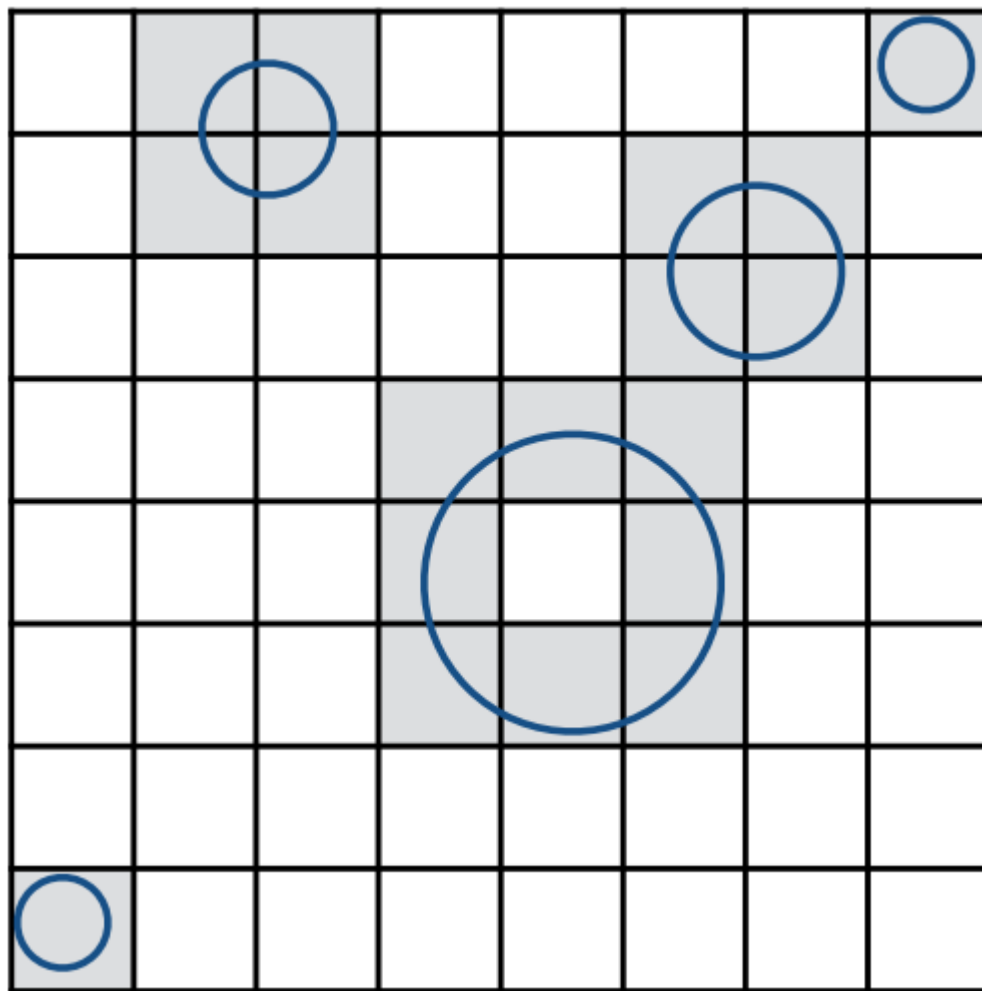
均匀格点法

1. 找到物体的包围盒
2. 划分网格



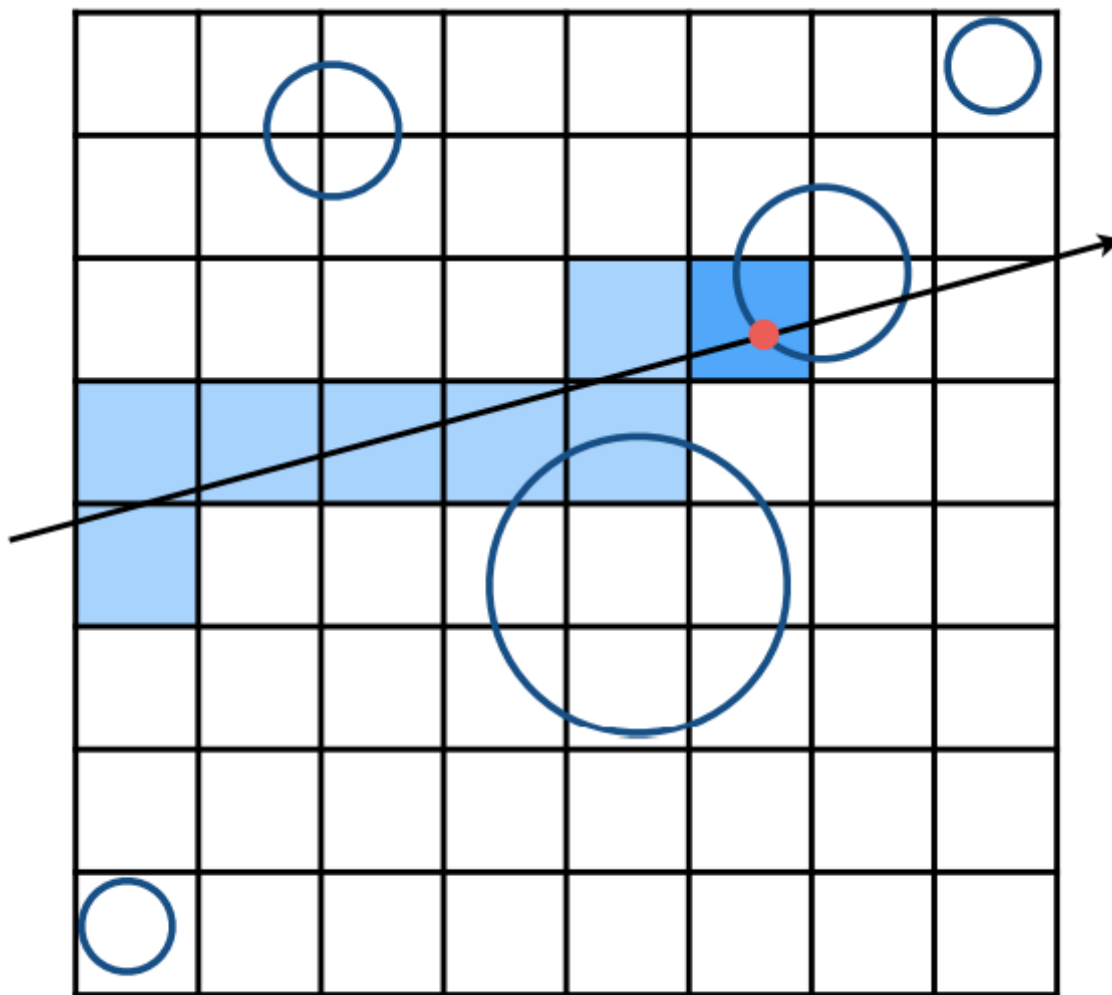
均匀格点法

1. 找到物体的包围盒
2. 划分网格
3. 对与物体重叠的单元格，
储存所有重叠的物体



均匀格点法

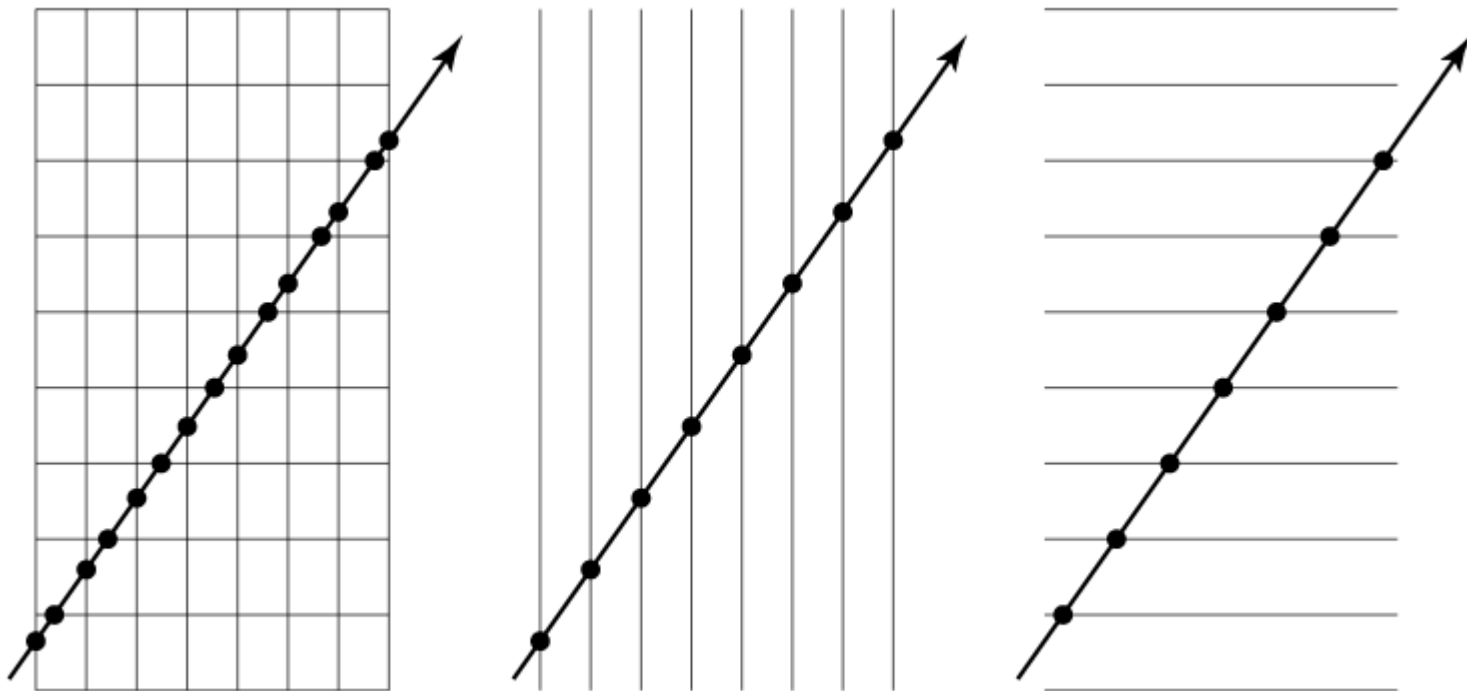
- 按照光线传播路径遍历经过的网格(3D DDA)
- 对于通过的所有的网格：
 计算光线与网格中所有物体的交点



均匀格点法

光线步进的计算

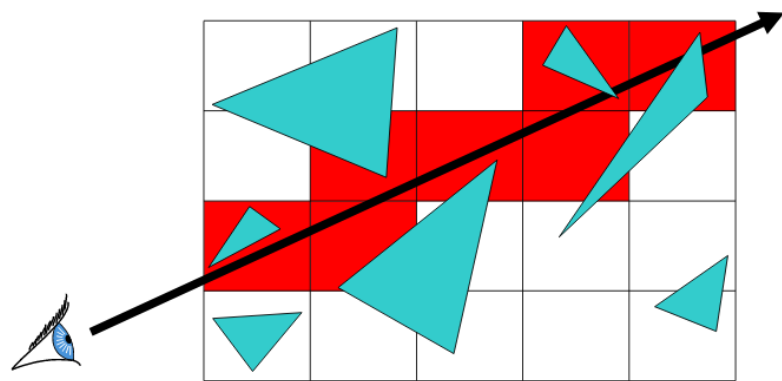
二维情形中，光线和水平线和竖直线的交点、分别在各自方向上是等间距的，容易计算交点坐标及所在单元。



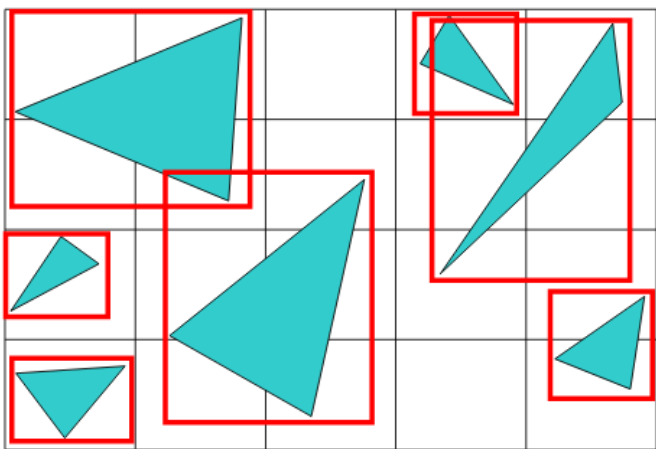
均匀格点法

■ 算法原理：

- 空间被分为均匀方格上的单元（体素）
- 每个单元存储了与其相交的所有物体或片元
- 光线通过单元向前追踪，在每个单元中和一个单元中的对象做相交检测；
- 检测到单元内相交点即停止

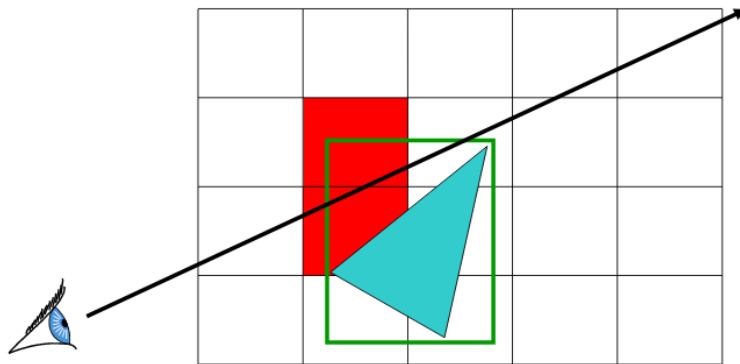


均匀格点法



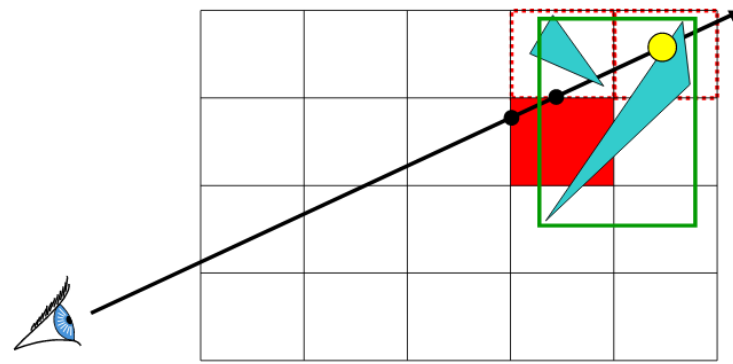
避免重复记录

避免物体或片元覆盖或交叠了多个单元，造成重复记录，可在单元中使用指针记录物体。



避免重复检测

标记检测过的物体或片元，避免接下来的cell里再次检测。

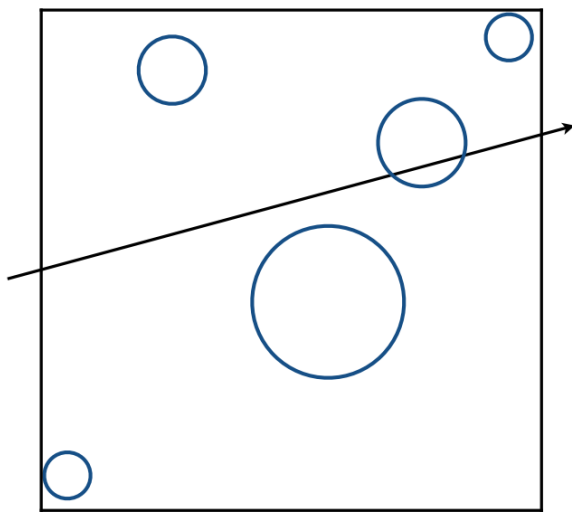


如果相交点不在当前的单元内，不能直接返回

需要继续向前追踪，可能存在更近的相交物体。

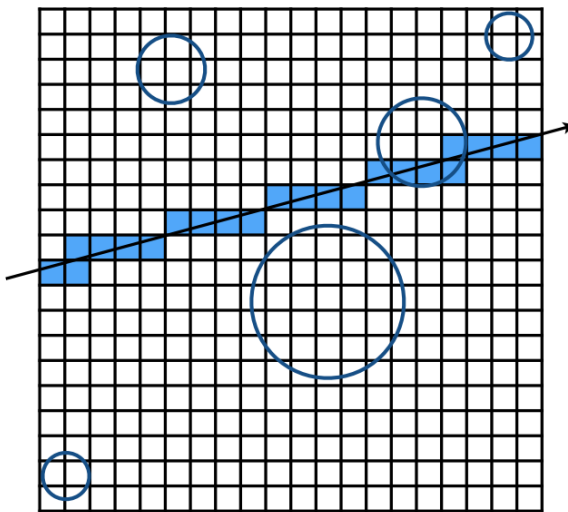
均匀格点法

格点分辨率:



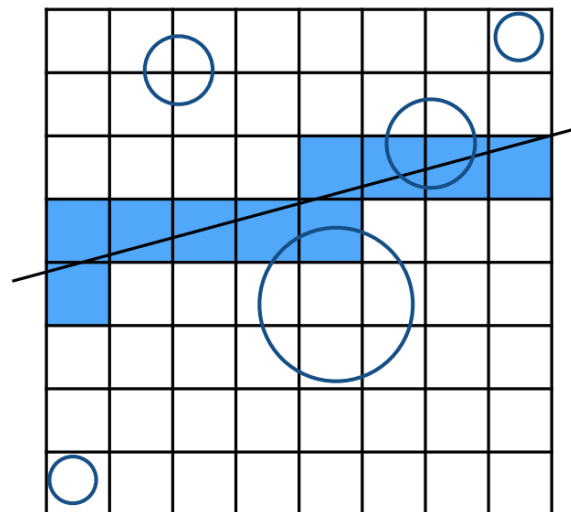
一个单元格

没有加速效果



单元格过多

网格遍历而导致
效率低下



单元格适中

较好的加速效果

均匀格点法

- 优点:

1. 容易构造
2. 容易进行光线跟踪

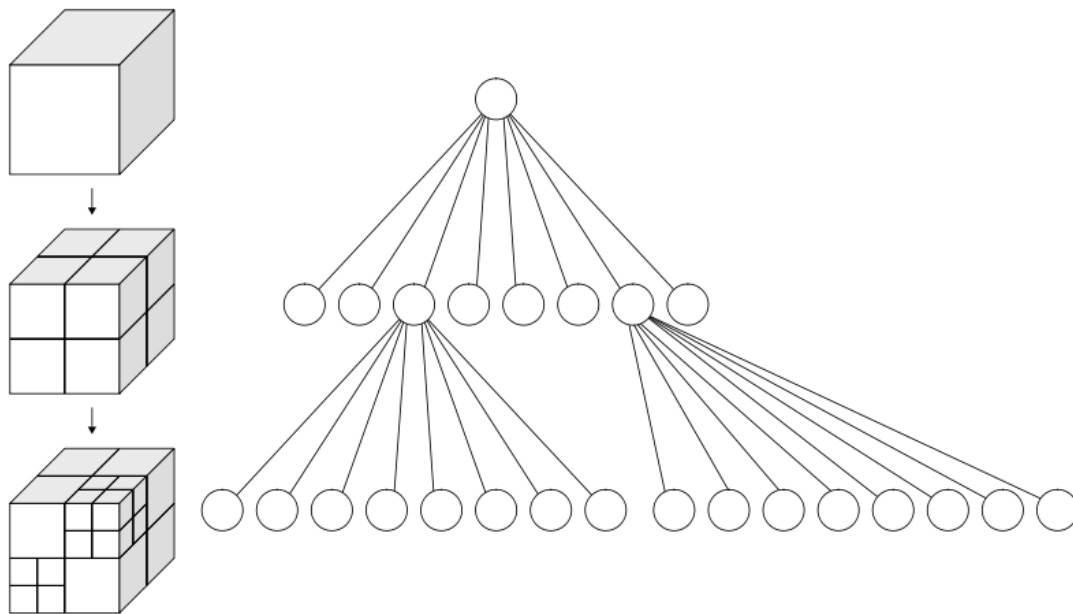
- 缺点，应对以下情形开销较大:

1. 物体片元在空间中整体是稀疏分布的
2. 物体片元可能在空间局部聚集

- 非均匀空间细分: Kd树 (k-dimensional tree) , 八叉树 (oct-tree) 、二叉空间分割树 (bsp-tree) 等

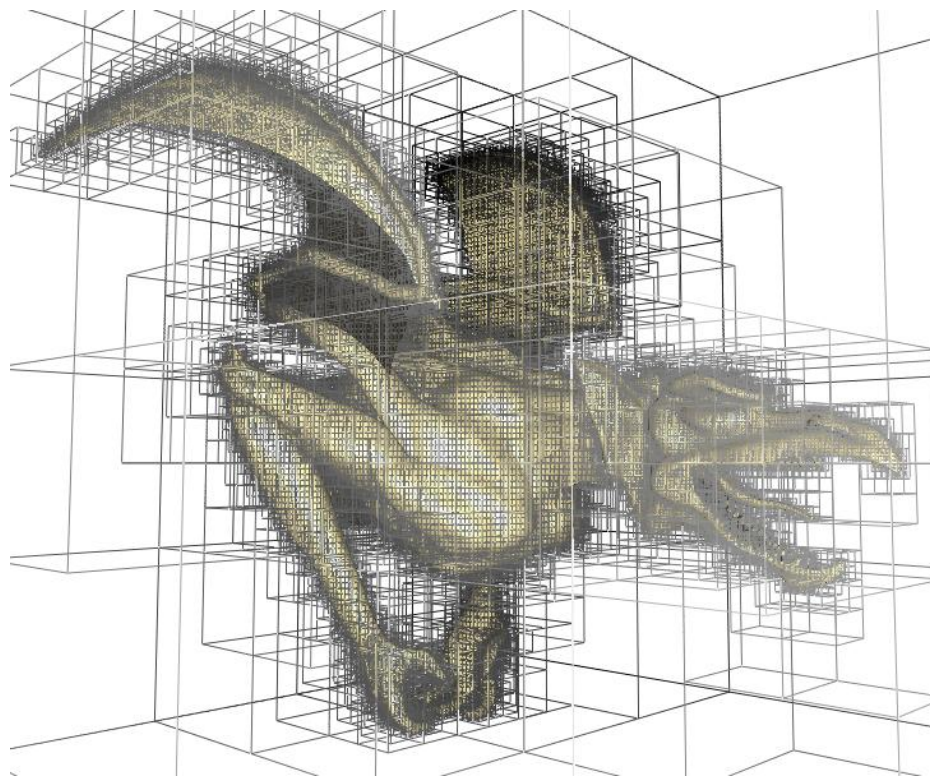
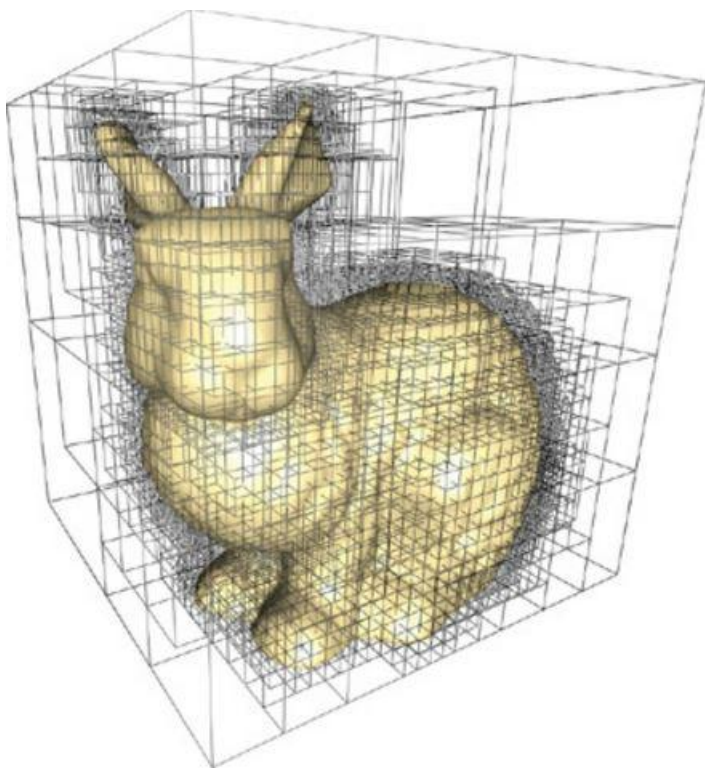
八叉树

- 八叉树内一个父节点有8个儿子节点，对应一个立方体区域、分割为8个等大的小立方体区域；
- 叶子节点是物体或片元；
- 对细节复杂的区域可以自适应细分；



八叉树

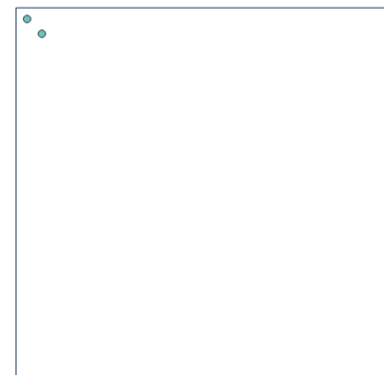
对三角网格模型构建的八叉树:



八叉树

■ 八叉树 (Oct-tree) 的问题:

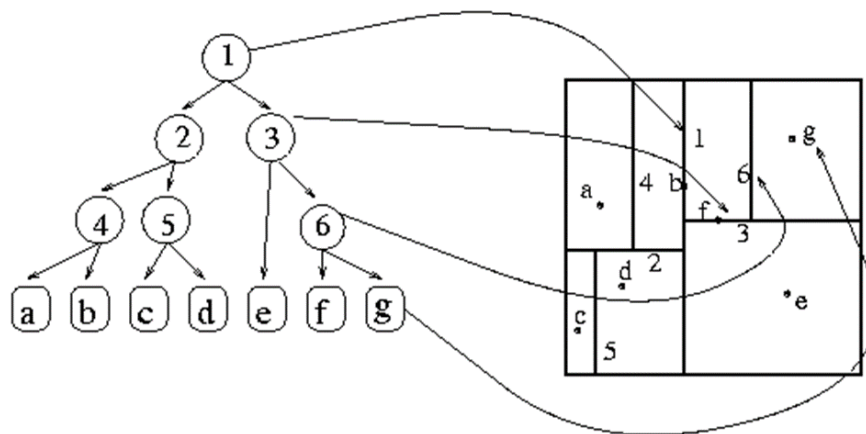
子立方体只能被均匀的分割出来。遇到物体或片元在空间中不是均匀分布的情形，构建出的八叉树会不平衡，如右图的情形。



A bad octree case

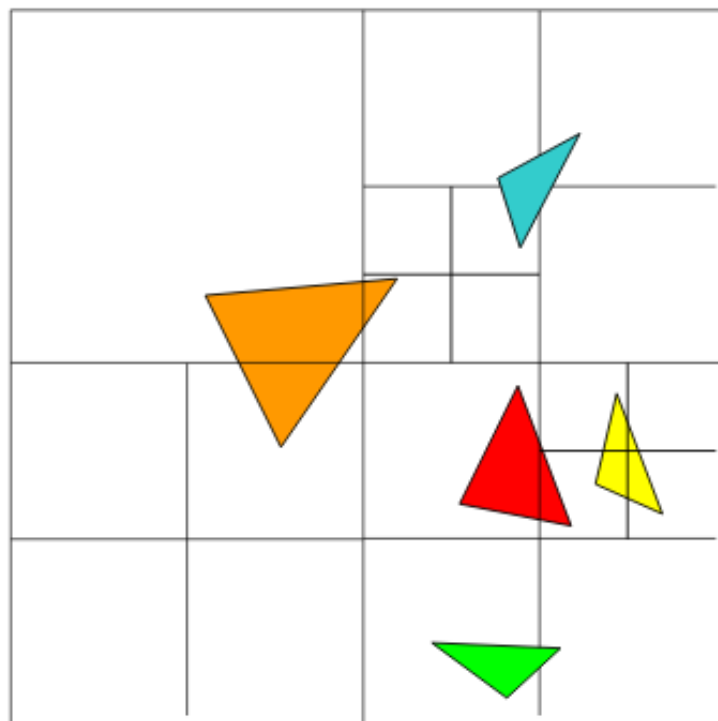
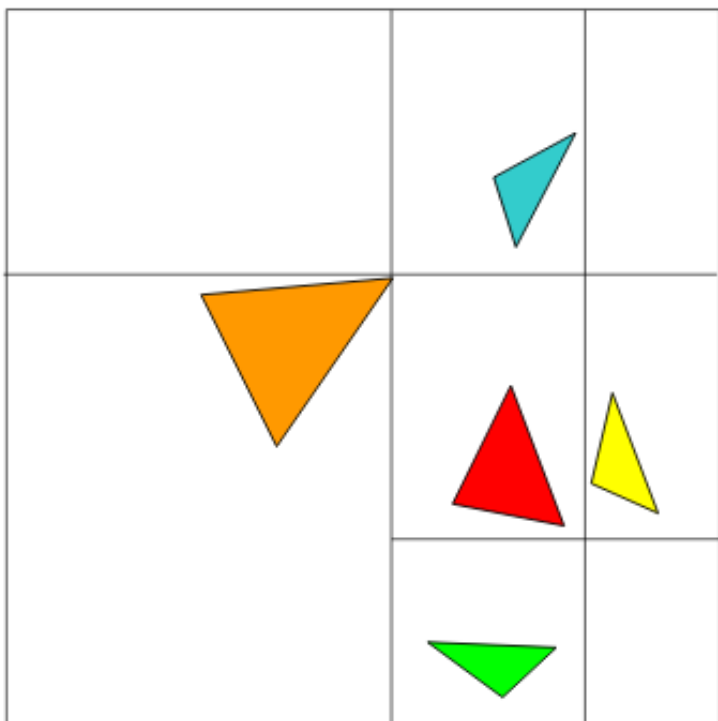
■ 解决方案:

允许分割平面的位置可变，不局限于立方体的平分面。这就引入了k-dimensional tree.



八叉树-KD树

Kd树 v.s. 八叉树



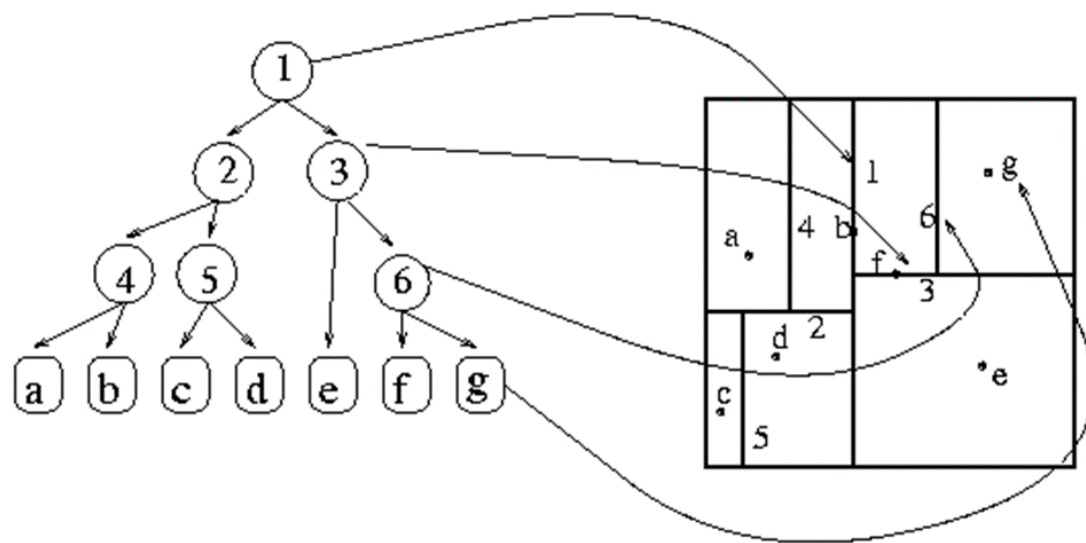
KD树

■ 中间结点储存:

- 划分坐标轴: x轴、y轴、z轴
- 划分位置: 划分平面在轴上的坐标
- 子结点

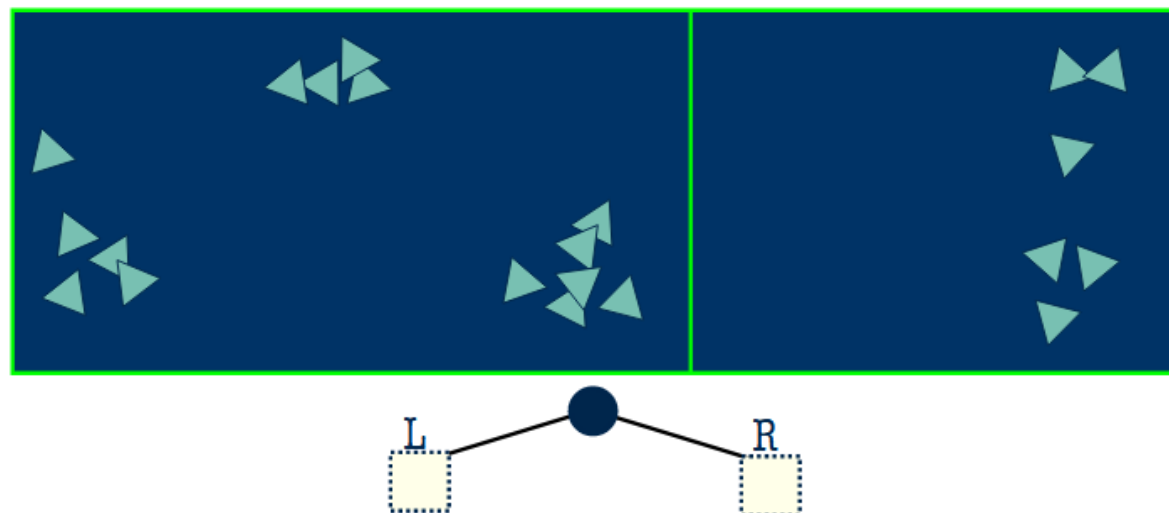
■ 叶子结点储存:

- 物体信息列表
- 包围盒



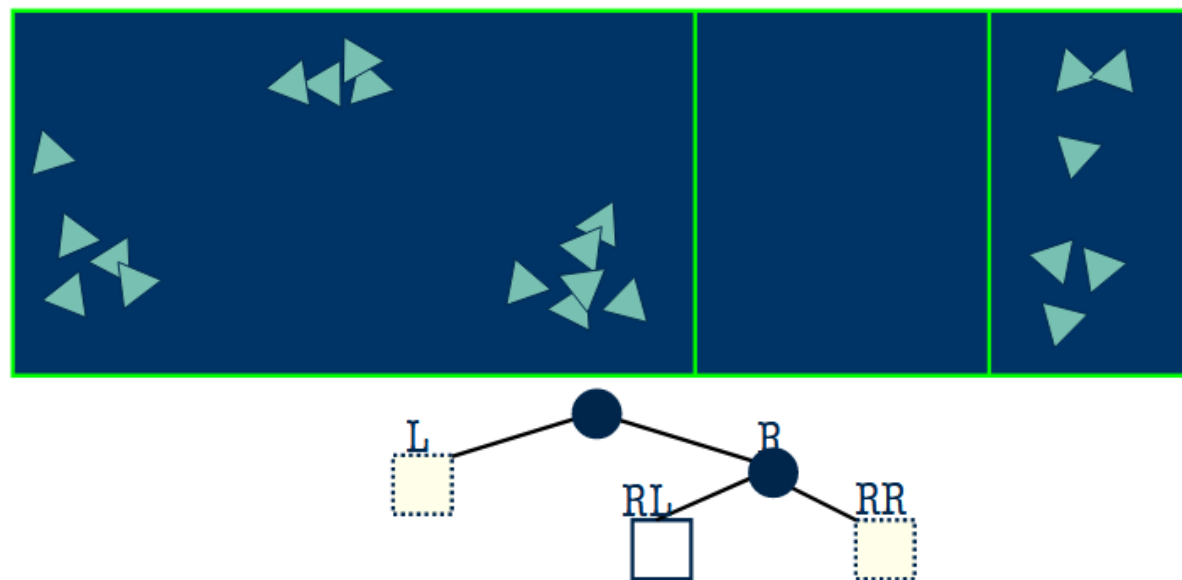
KD树

k-dimensional tree 例子 (构建过程)



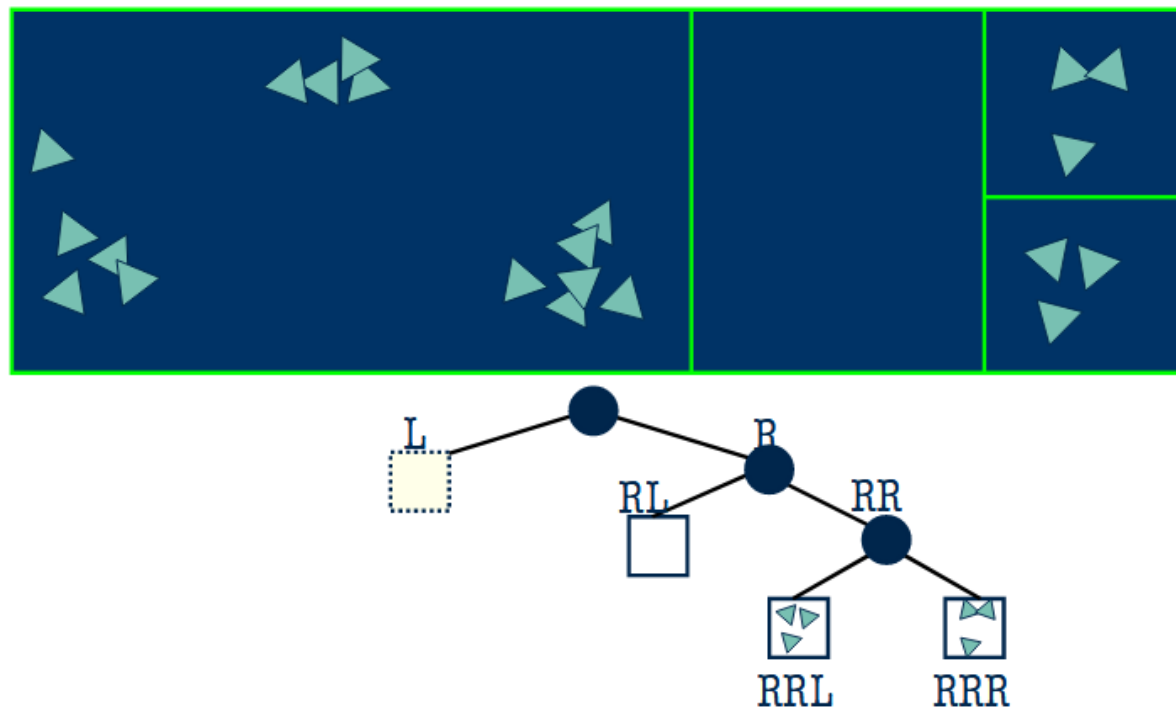
KD树

k-dimensional tree 例子 (构建过程)



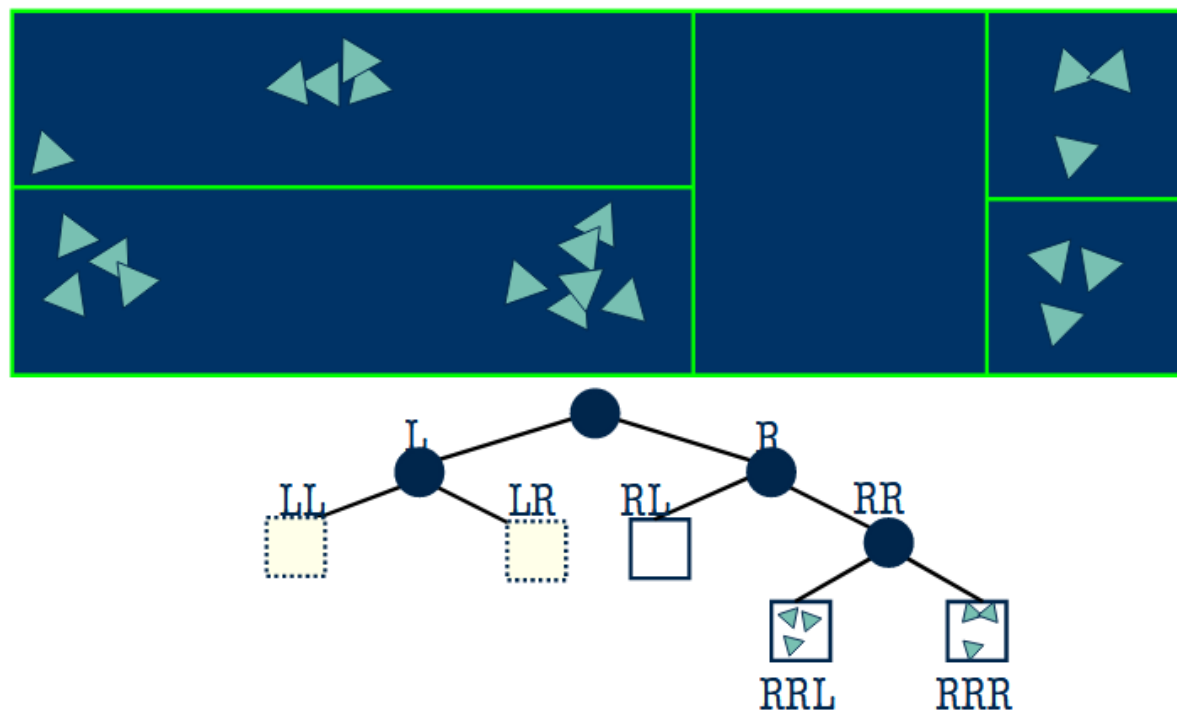
KD树

k-dimensional tree 例子 (构建过程)



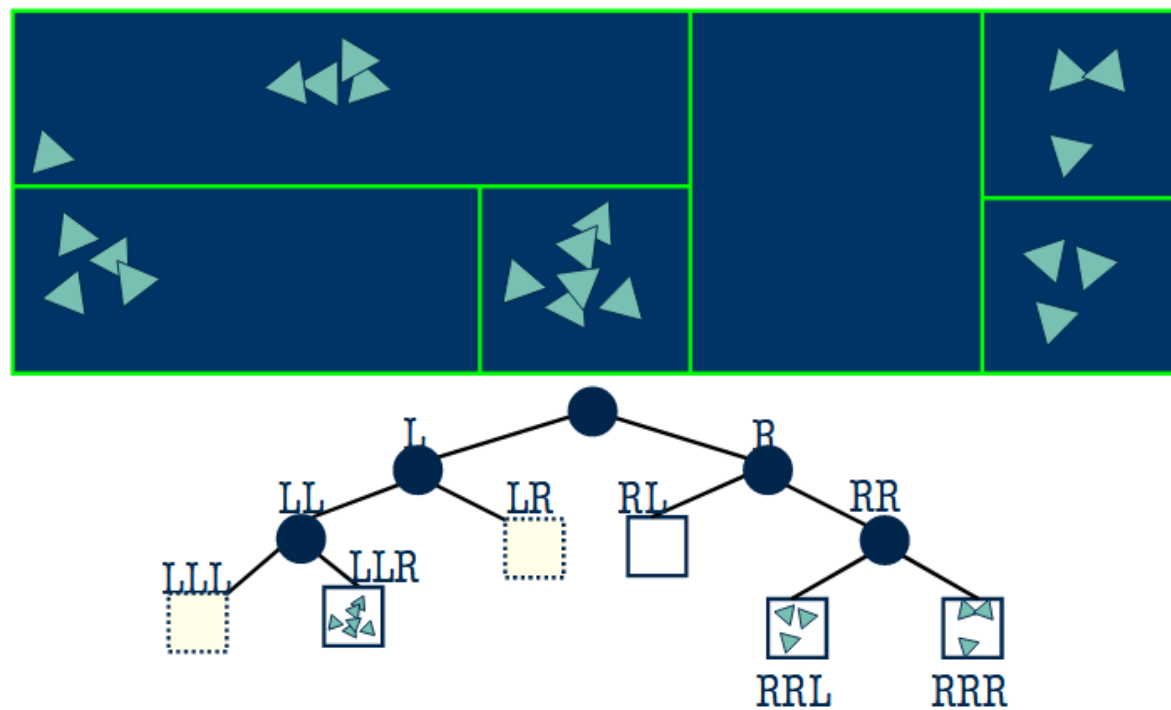
KD树

k-dimensional tree 例子 (构建过程)



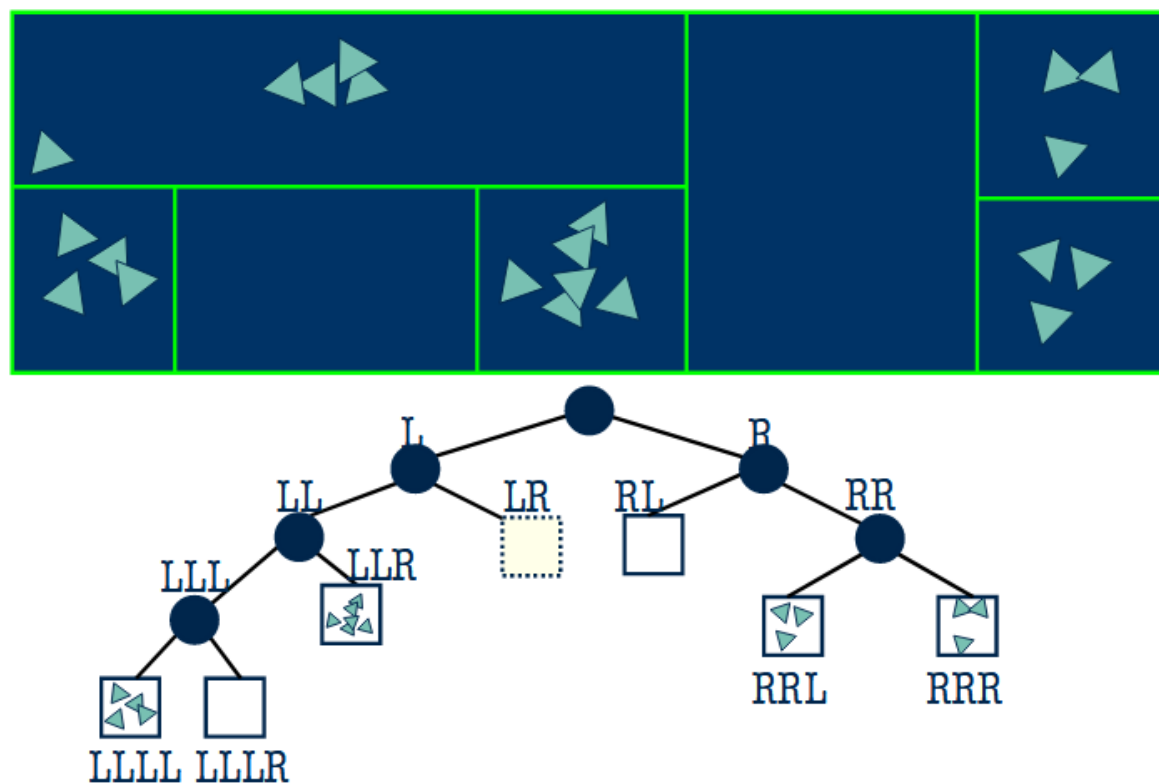
KD树

k-dimensional tree 例子 (构建过程)



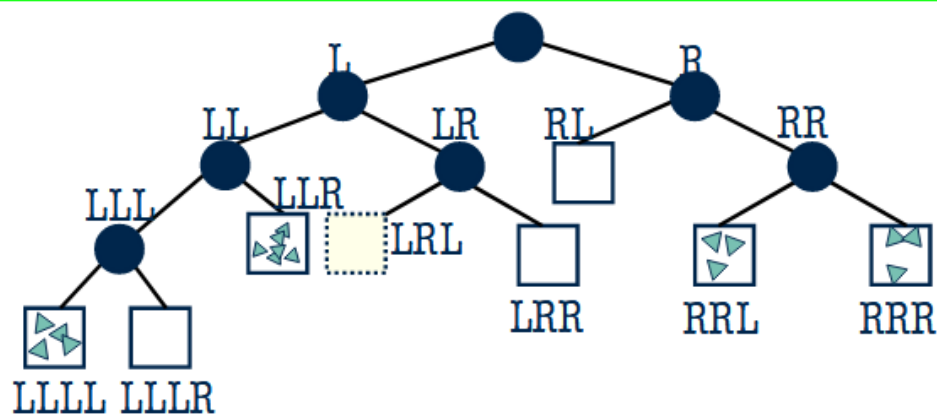
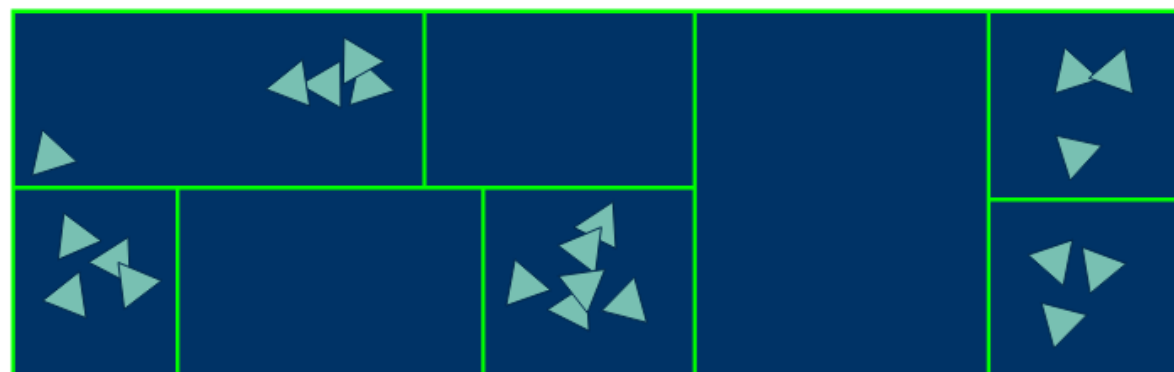
KD树

k-dimensional tree 例子 (构建过程)



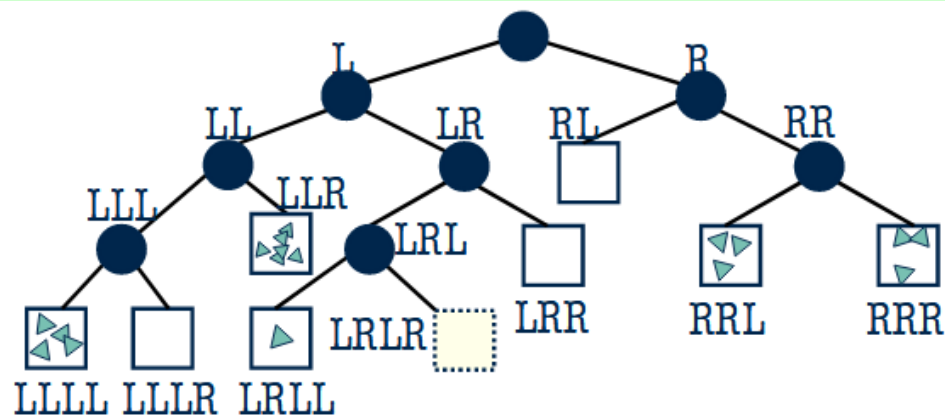
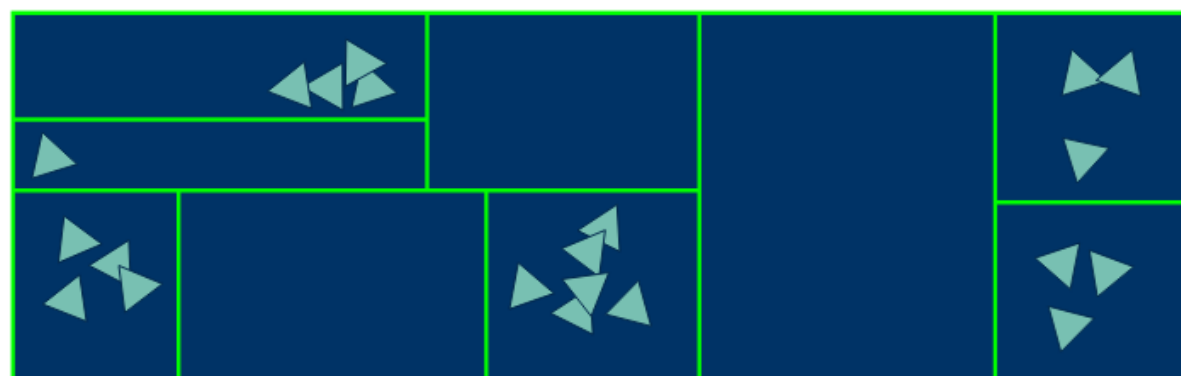
KD树

k-dimensional tree 例子 (构建过程)



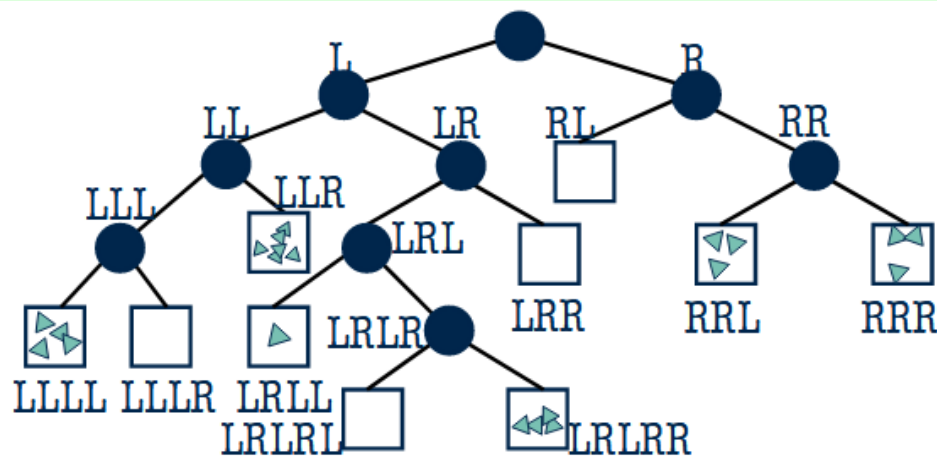
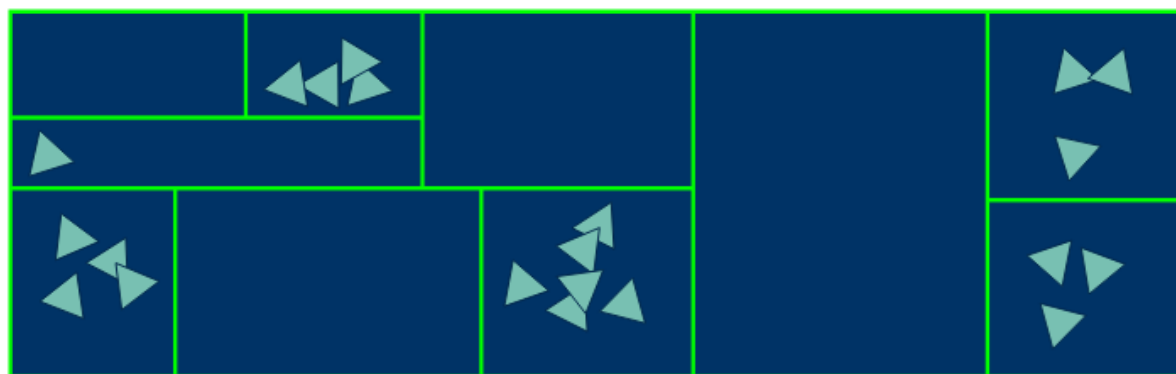
KD树

k-dimensional tree 例子 (构建过程)



KD树

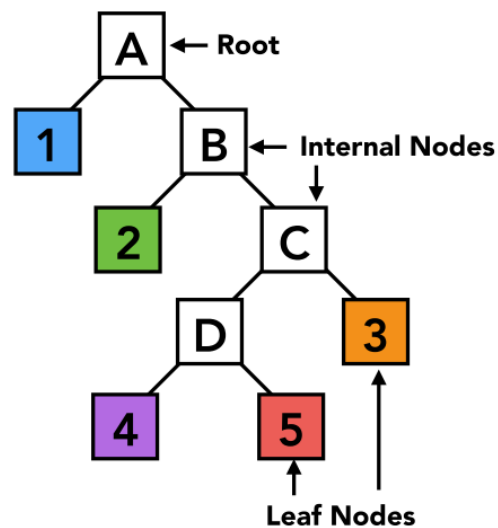
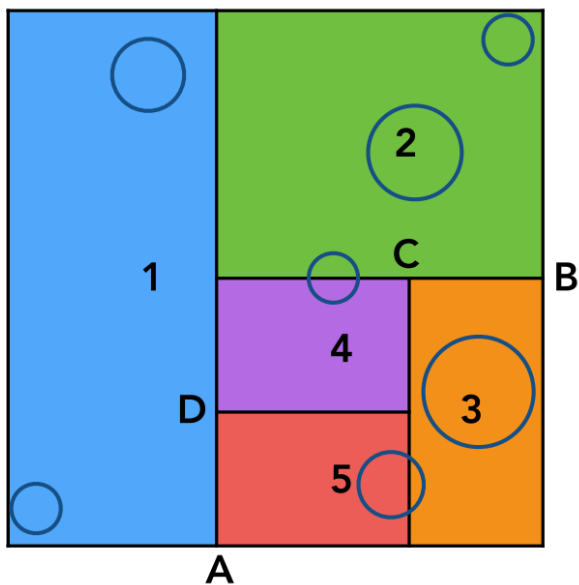
k-dimensional tree 例子 (构建过程)



KD树

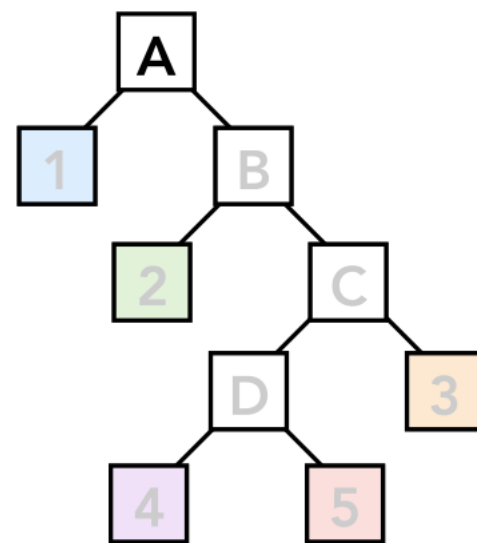
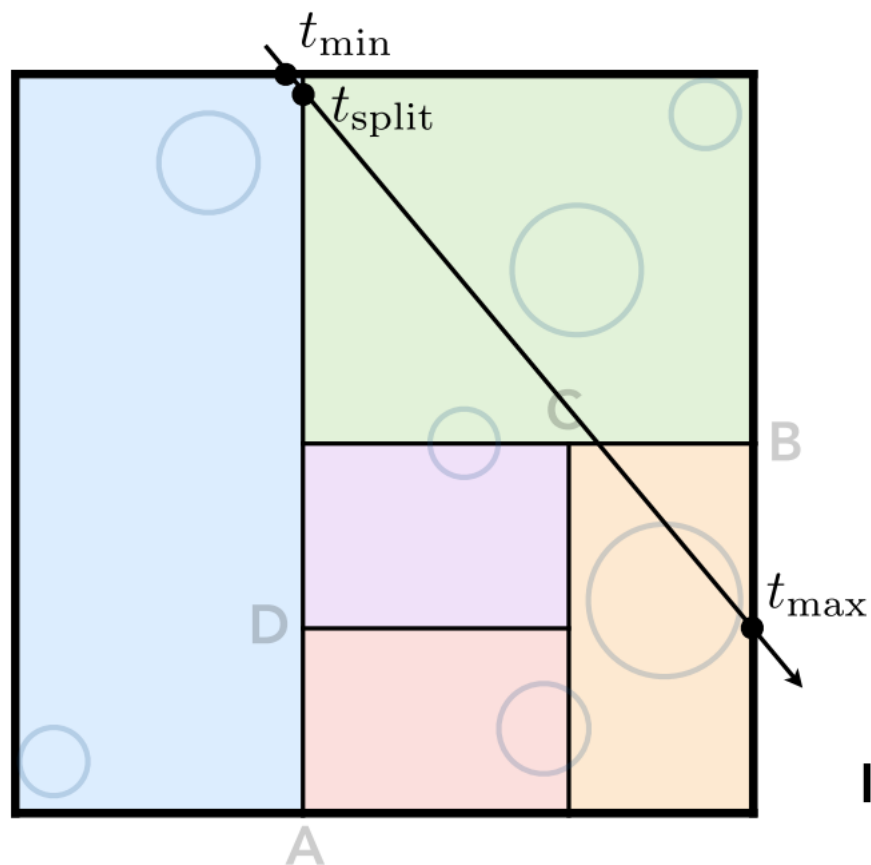
■ KD树建立过程:

- 找到包围盒
- 使用与坐标轴平行的平面，划分包围盒
- 直到满足停止划分的条件 (eg. 最大划分数、最少物体数)
- 将物体信息储存于叶子结点中



KD树-求交过程

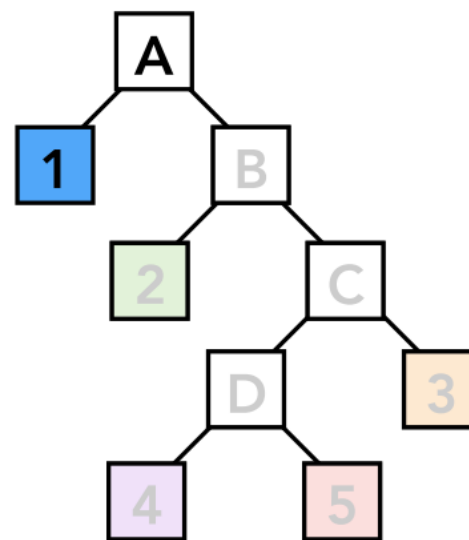
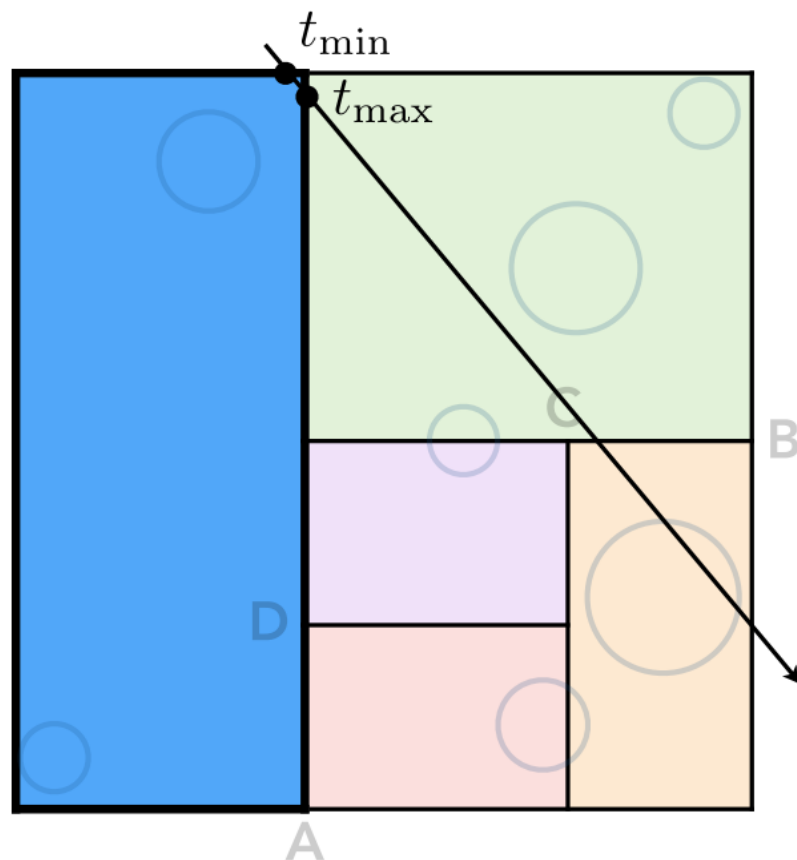
- 与格子A相交:



Internal node: split

KD树-求交过程

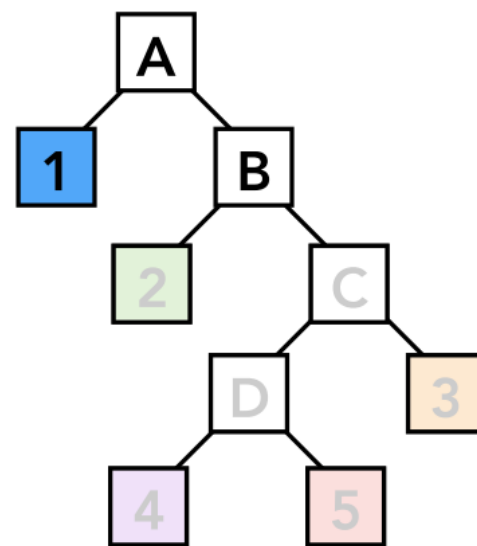
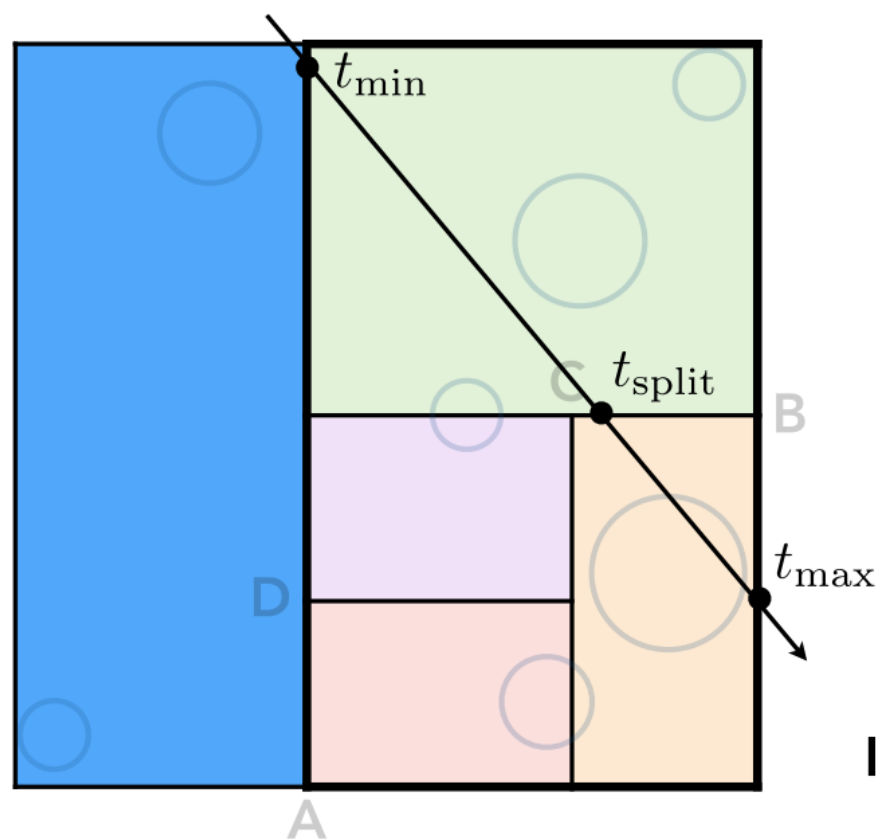
- 与包围盒1相交，但不和物体相交：



Leaf node: intersect
all objects

KD树-求交过程

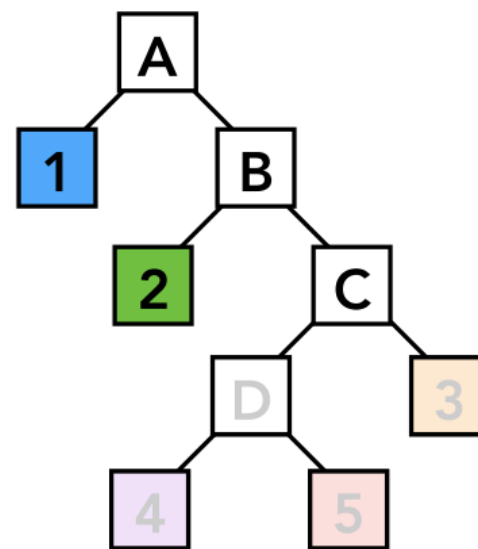
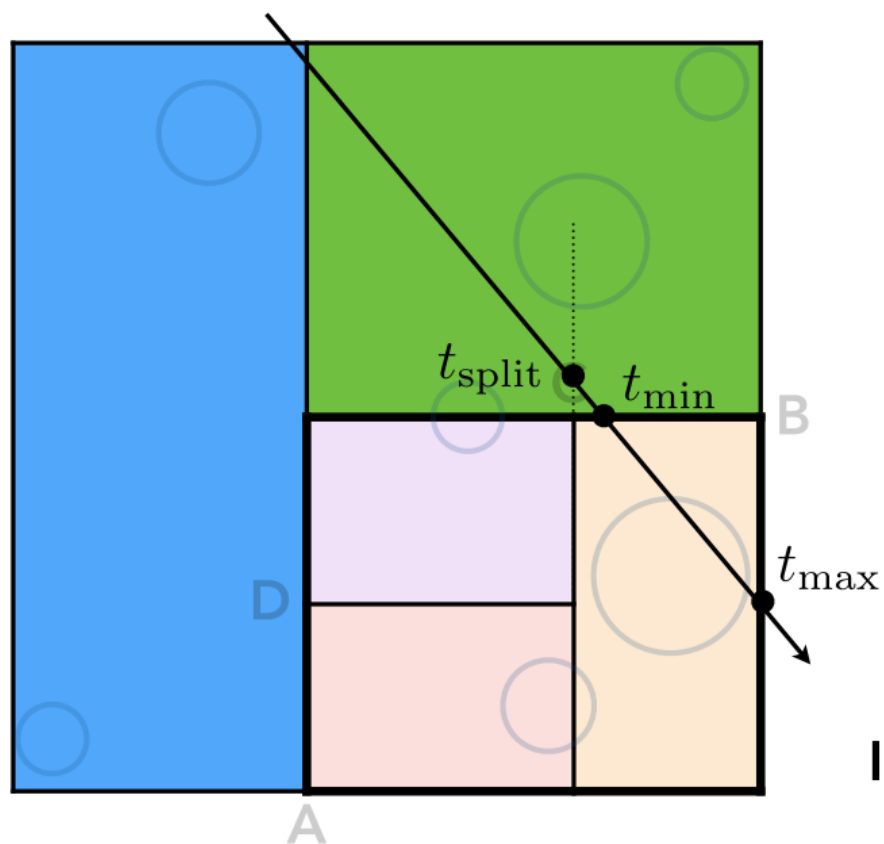
- 与格子B相交:



Internal node: split

KD树-求交过程

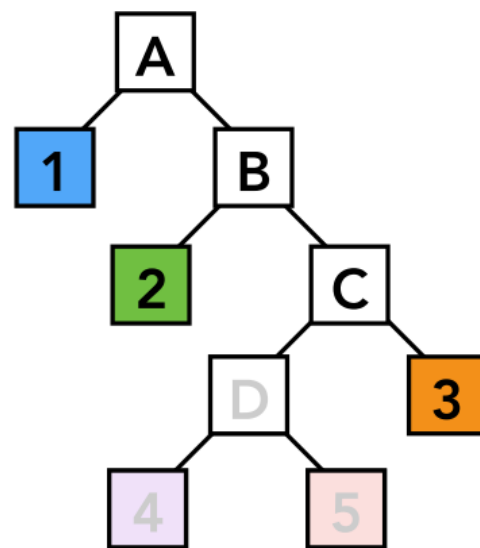
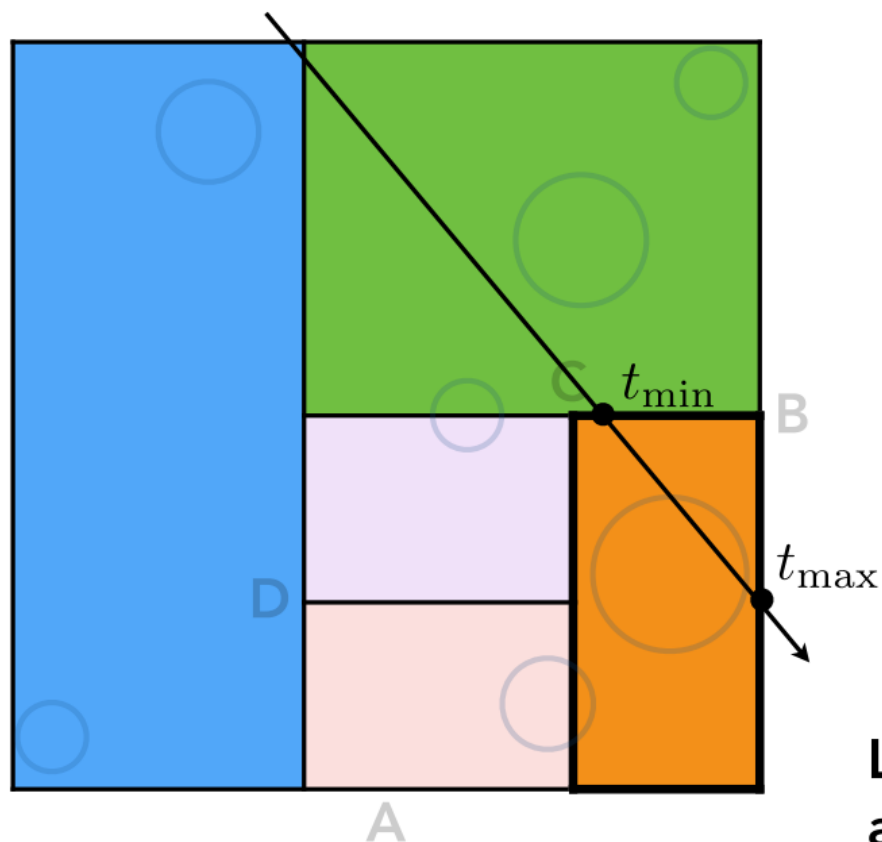
- 与包围盒2相交，但不和物体相交：



Internal node: split

KD树-求交过程

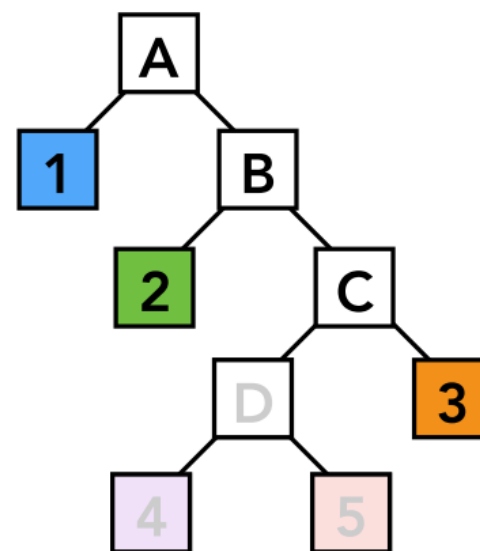
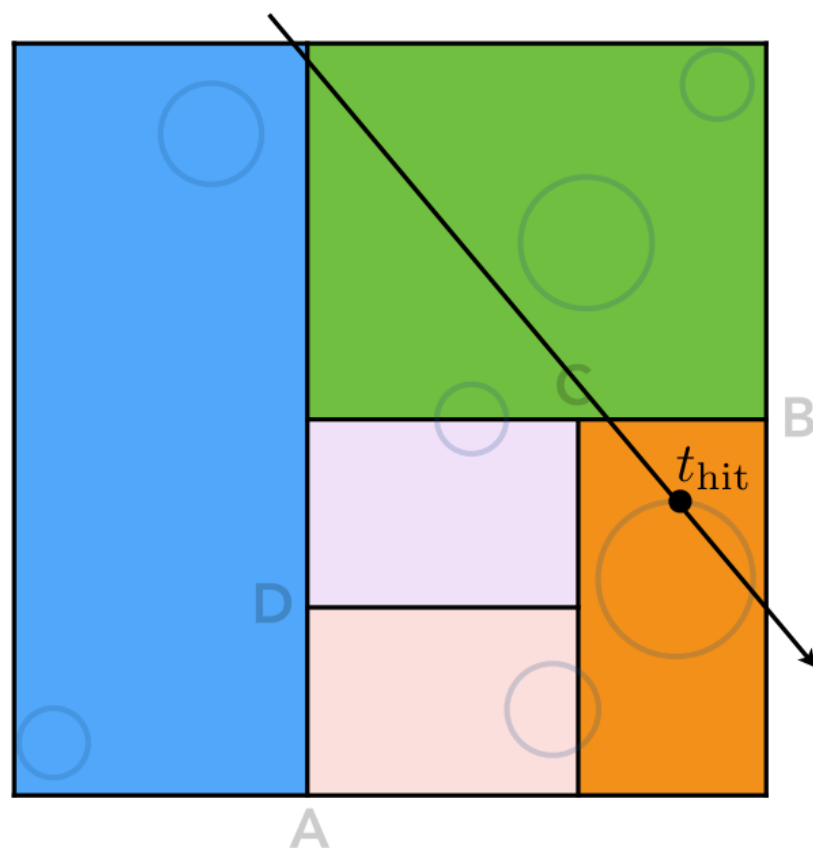
- 与包围盒3相交，判断是否物体相交：



Leaf node: intersect
all objects

KD树-求交过程

- 与包围盒3中物体相交，计算具体交点：



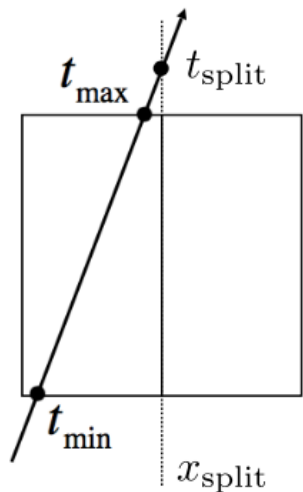
Intersection found

KD树-求交过程

■ 光线求交的递归步骤:

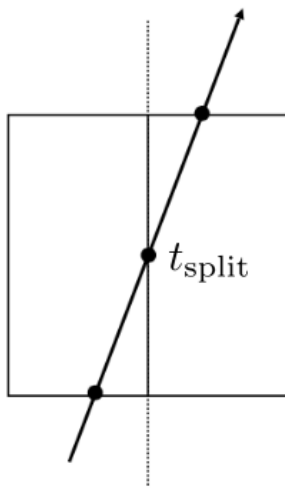
下述以x轴上的划分平面、方向向右的光线作为示例:

$$t_{\text{split}} = (x_{\text{split}} - \mathbf{o}_x) / \mathbf{d}_x$$



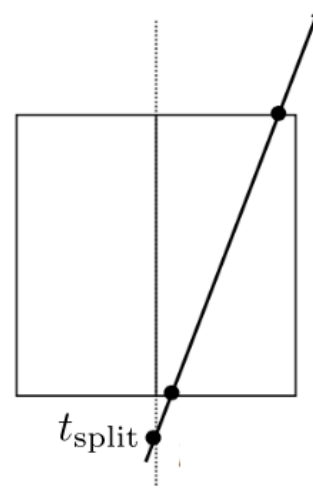
$$t_{\text{max}} < t^*$$

`Intersect(L, tmin, tmax)`



$$t_{\text{min}} < t^* < t_{\text{max}}$$

`Intersect(L, tmin, t*)`
`Intersect(R, t*, tmax)`



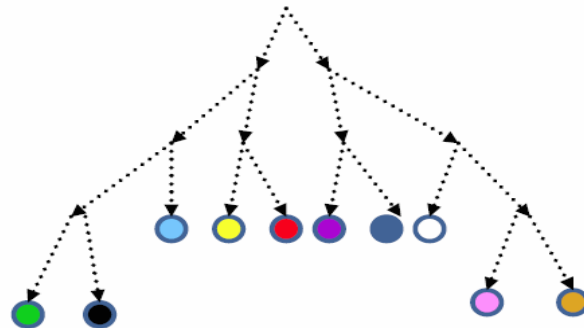
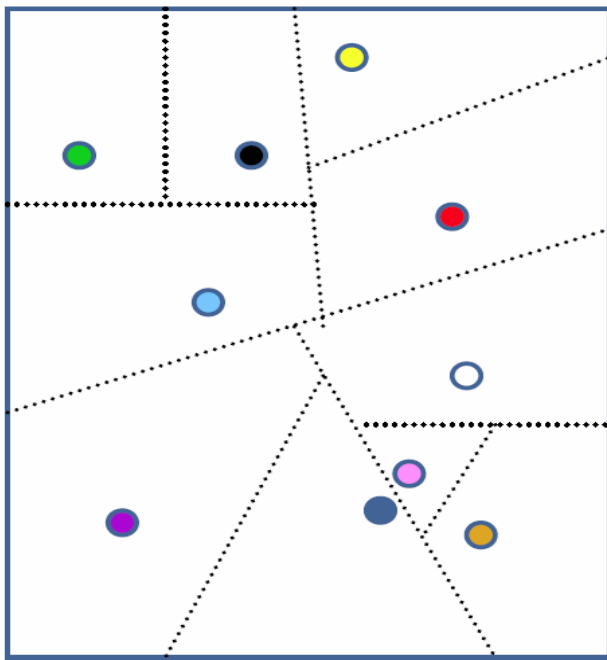
$$t^* < t_{\text{min}}$$

`Intersect(R, tmin, tmax)`

空间二分树（BSP树）

■ 空间二分树 (BSP Tree)

- 和k-d tree相比，分割平面可以取任意的方向；
- 是目前许多游戏环境里空间细分的行业标准.



谢谢！ 欢迎交流！

高林

中国科学院计算技术研究所

gaolin@ict.ac.cn